

DESKTOP ABYSS: TASKBAR CHRONICLE — FULL

CODEBASE SNAPSHOT v1.2.0

Полный снимок архитектуры и всех исходных файлов проекта на Python (PySide6).

Архитектура проекта и реестр файлов:

- config.py (100 строк)- models/stats.py (95 строк)- models/hero.py (646 строк)- models/item.py (635 строк)- models/enemy.py (239 строк)- engine/state.py (401 строк)- engine/combat_core.py (1229 строк)- engine/combat_manager.py (593 строк)- engine/cube_synth.py (219 строк)- engine/runes_tree.py (371 строк)- engine/offline_manager.py (238 строк)- engine/save_manager.py (140 строк)- gfx/animations.py (77 строк)- gfx/sprite_loader.py (447 строк)- ui/common.py (569 строк)- ui/gothic_frame.py (190 строк)- ui/detailed_stats_window.py (159 строк)- ui/battle_stage.py (968 строк)- ui/actions.py (249 строк)- ui/panels/hero_panel.py (423 строк)- ui/panels/stash_panel.py (122 строк)- ui/panels/cube_panel.py (238 строк)- ui/panels/runes_panel.py (269 строк)- ui/main_window.py (524 строк)- main.py (94 строк)

Файл: config.py

```
`python
from future import annotations

APP_VERSION = "1.2.0"

import os
import sys
from pathlib import Path
```

```

from types import MappingProxyType

APP_NAME = "Desktop Abyss: Taskbar Chronicle"
APP_SLUG = "desktop-abyss"
ORGANIZATION_NAME = "DesktopAbyss"
SAVE_SCHEMA_VERSION = 6

PROJECT_DIR = Path(file).resolve().parent
ASSETS_DIR = PROJECT_DIR / "assets"
FONTS_DIR = ASSETS_DIR / "fonts"
SPRITES_DIR = ASSETS_DIR / "sprites"
ORNAMENTS_DIR = ASSETS_DIR / "ornaments"
SOUNDS_DIR = ASSETS_DIR / "sounds"

def _user_data_dir() -> Path:
    override = os.environ.get("DESKTOP_ABYSS_DATA_DIR")
    if override:
        return Path(override).expanduser().resolve()

    if sys.platform == "win32":
        root = Path(
            os.environ.get(
                "LOCALAPPDATA",
                str(Path.home() / "AppData" / "Local"),
            )
        )
    elif sys.platform == "darwin":
        root = Path.home() / "Library" / "Application Support"
    else:
        root = Path(
            os.environ.get(
                "XDG_DATA_HOME",
                str(Path.home() / ".local" / "share"),
            )
        )

    return root / APP_SLUG

DATA_DIR = _user_data_dir()
SAVE_PATH = DATA_DIR / "save.json"

```

LOCK_PATH = DATA_DIR / "application.lock"

AUTOSAVE_INTERVAL_MS = 30_000

MAX_PARTY_SIZE = 4

DEFAULT_STASH_CAPACITY = 120

DEFAULT_BACKPACK_CAPACITY = 48

Индекс ступени хранится в сохранении.

OFFLINE_CAP_HOURS = (2, 3, 4, 6, 8, 12)

OFFLINE_CAP_SECONDS = tuple(hours * 3600 for hours in OFFLINE_CAP_HOURS)

OFFLINE_EFFICIENCIES = (0.60, 0.75, 0.90, 1.00)

Начальный баланс. Эти значения не зафиксированы в GDD.

DEFAULT_WAVE_CLEAR_SECONDS = 12.0

MIN_WAVE_CLEAR_SECONDS = 1.0

DEFAULT_GOLD_PER_WAVE = 100

DEFAULT_EXP_PER_WAVE = 30

DEFAULT_DROP_CHANCE = 0.02

Минимальный предметный DTO Модуля 1: редкость, относительный вес, базовая цена продажи.

```
LOOT_TABLE = (  
    ("common", 75, 10),  
    ("rare", 20, 50),  
    ("legendary", 5, 250),  
)
```

```
ITEM_RARITIES = frozenset({  
    "common",
```

```

"uncommon",
"rare",
"legendary",
"immortal",
"mythic",
})

PALETTE = MappingProxyType({
"basalt": "#121118",
"steel": "#23212b",
"ruby": "#8b1818",
"gold": "#c8963e",
"amethyst": "#9b4dca",
"text": "#e4dccb",
"muted_text": "#9a919f",
"border": "#514032",
"health": "#a32b36",
"mana": "#477cb8",
})

BATTLE_STAGE_WIDTH = 520
BATTLE_STAGE_HEIGHT = 144

```

Файл: models/stats.py

```

`python
from future import annotations

import math
from dataclasses import dataclass, fields

@dataclass(frozen=True)
class StatBlock:
    # Первичные атрибуты.
    strength: float = 0.0
    agility: float = 0.0

```

intelligence: float = 0.0

vitality: float = 0.0

luck: float = 0.0

Совместимые поля предыдущих модулей.

damage: float = 0.0

armor: float = 0.0

max_hp: float = 0.0

mana: float = 0.0

physical_bonus: float = 0.0

magical_bonus: float = 0.0

hp_bonus: float = 0.0

crit_chance: float = 0.0

crit_damage: float = 0.0

evasion: float = 0.0

Attack.

attack_speed: float = 0.0

basic_requirement_reduction: float = 0.0

cooldown_reduction: float = 0.0

skill_range: float = 0.0

basic_attack_range: float = 0.0

cast_speed: float = 0.0

fire_bonus: float = 0.0

cold_bonus: float = 0.0

lightning_bonus: float = 0.0

chaos_bonus: float = 0.0

multistrike: float = 0.0

projectile_count: float = 0.0

melee_bonus: float = 0.0

projectile_bonus: float = 0.0

aoe_damage: float = 0.0

projectile_speed: float = 0.0

summon_damage: float = 0.0

Defense.

fire_res: float = 0.0

cold_res: float = 0.0

lightning_res: float = 0.0

chaos_res: float = 0.0

block_chance: float = 0.0

elemental_dodge: float = 0.0

```

elemental_block: float = 0.0
hp_regen: float = 0.0
damage_absorption: float = 0.0

# Utility.
movement_speed: float = 0.0
aoe_enhancement: float = 0.0
skill_duration: float = 0.0
hp_per_hit: float = 0.0
hp_per_kill: float = 0.0
life_leech: float = 0.0
skill_heal: float = 0.0
all_skill_level: float = 0.0
exp_gain: float = 0.0
additional_exp: float = 0.0

def __post_init__(self) -> None:
    for definition in fields(self):
        value = getattr(self, definition.name)
        if type(value) not in (int, float) or not math.isfinite(value):
            raise ValueError(
                f"Некорректное значение {definition.name}: {value}"
            )

def __add__(self, other: StatBlock) -> StatBlock:
    if not isinstance(other, StatBlock):
        return NotImplemented
    return StatBlock(**{
        definition.name: (
            getattr(self, definition.name)
            + getattr(other, definition.name)
        )
        for definition in fields(self)
    })

def scaled(self, factor: float) -> StatBlock:
    if type(factor) not in (int, float) or not math.isfinite(factor):
        raise ValueError("Множитель должен быть конечным числом")
    return StatBlock(**{
        definition.name: getattr(self, definition.name) * factor
        for definition in fields(self)
    })

```

Файл: `models/hero.py`

```
`python
from future import annotations

import math
from dataclasses import dataclass, replace
from enum import Enum
from fractions import Fraction
from types import MappingProxyType
from typing import TYPE_CHECKING, Any, Sequence

from models.item import Item, ItemKind, item_from_dict
from models.stats import StatBlock

if TYPE_CHECKING:
    from engine.state import HeroState

XP_BASE = 100
STAT_POINTS_PER_LEVEL = 5
PRIMARY_STAT_KEYS = (
    "strength", "agility", "intelligence", "vitality", "luck",
)

class HeroClass(str, Enum):
    KNIGHT = "knight"
    ASSASSIN = "assassin"
    PYROMANCER = "pyromancer"
    RANGER = "ranger"
    NECROMANCER = "necromancer"
    PALADIN = "paladin"

CLASS_LIFESTEAL = {
    HeroClass.KNIGHT: 0.0,
    HeroClass.ASSASSIN: 0.03,
```

```

HeroClass.PYROMANCER: 0.0,
HeroClass.RANGER: 0.0,
HeroClass.NECROMANCER: 0.04,
HeroClass.PALADIN: 0.0,
}

CLASS_BONUSES = {
HeroClass.KNIGHT: StatBlock(
armor=10, block_chance=0.05, melee_bonus=0.15,
),
HeroClass.ASSASSIN: StatBlock(
crit_chance=0.05, attack_speed=0.15, evasion=0.03,
),
HeroClass.PYROMANCER: StatBlock(
fire_bonus=0.25, cast_speed=0.10,
),
HeroClass.RANGER: StatBlock(
projectile_bonus=0.20, attack_speed=0.15,
),
HeroClass.NECROMANCER: StatBlock(
chaos_bonus=0.20, summon_damage=0.30,
),
HeroClass.PALADIN: StatBlock(
block_chance=0.10, skill_heal=0.15, lightning_bonus=0.15,
),
}

CLASS_RANGE = {
HeroClass.KNIGHT: 140.0,
HeroClass.ASSASSIN: 140.0,
HeroClass.PYROMANCER: 300.0,
HeroClass.RANGER: 350.0,
HeroClass.NECROMANCER: 300.0,
HeroClass.PALADIN: 140.0,
}

@dataclass(frozen=True)

```



```

class Skill:
    skill_id: str
    name: str
    unlock_level: int
    cooldown_seconds: float
    mana_cost: int
    power_multiplier: float
    effect: str
    duration_seconds: float = 0.0
    target: str = "enemy"

```

```

@dataclass(frozen=True)

```

```

class ClassDefinition:

```

```

    name: str
    base: StatBlock
    growth: StatBlock
    base_hp: int
    base_armor: int
    magical: bool
    skills: tuple[Skill, ...]

```

```

    def primary(
        strength: float,
        agility: float,
        intelligence: float,
        vitality: float,
        luck: float,
    ) -> StatBlock:
        return StatBlock(
            strength=strength,
            agility=agility,
            intelligence=intelligence,
            vitality=vitality,
            luck=luck,
        )

```

```

CLASS_DEFINITIONS = MappingProxyType({

```

```

HeroClass.KNIGHT: ClassDefinition(
    "Рыцарь", primary(12, 5, 3, 14, 4),
    primary(3, 1, 0.5, 4, 0.5), 100, 10, False,
    (
        Skill("knight_slash", "Рассекающий удар", 1, 4, 0, 1.8, "damage"),
        Skill("knight_guard", "Стальная стена", 5, 15, 15, 0.30,
            "damage_reduction", 5, "self"),
        Skill("knight_shout", "Боевой клич", 15, 25, 25, 0.20,
            "party_damage_buff", 8, "party"),
    ),
),
HeroClass.ASSASSIN: ClassDefinition(
    "Ассасин", primary(7, 15, 4, 7, 8),
    primary(1.5, 4, 0.5, 2, 1.5), 60, 3, False,
    (
        Skill("assassin_stab", "Удар из тени", 1, 3, 8, 2.2, "damage"),
        Skill("assassin_poison", "Яд Бездны", 5, 9, 12, 0.5,
            "poison_per_second", 5),
        Skill("assassin_veil", "Пелена", 15, 20, 20, 0.35,
            "evasion_buff", 5, "self"),
    ),
),
HeroClass.PYROMANCER: ClassDefinition(
    "Пиромант", primary(3, 6, 16, 6, 6),
    primary(0.5, 1, 4.5, 1.5, 1), 50, 2, True,
    (
        Skill("pyro_fireball", "Огненный шар", 1, 4, 10, 2.0, "damage"),
        Skill("pyro_flame", "Пожарище", 5, 10, 20, 0.7,
            "burn_per_second", 4, "all_enemies"),
        Skill("pyro_meteor", "Метеор", 15, 24, 40, 4.0,
            "damage", 0, "all_enemies"),
    ),
),
HeroClass.RANGER: ClassDefinition(
    "Рейнджер", primary(7, 13, 5, 8, 7),

```

```

primary(1.5, 3.5, 1, 2, 1.5), 70, 4, False,
(
Skill("ranger_shot", "Прицельный выстрел", 1, 4, 5, 2.1, "damage"),
Skill("ranger_volley", "Залп", 5, 10, 15, 1.4,
"damage", 0, "all_enemies"),
Skill("ranger_mark", "Метка охотника", 15, 18, 20, 0.25,
"damage_taken_debuff", 8),
),
),
HeroClass.NECROMANCER: ClassDefinition(
"Некромант", primary(4, 4, 15, 9, 6),
primary(0.5, 0.5, 4, 2.5, 1), 65, 3, True,
(
Skill("necro_bolt", "Костяное копье", 1, 4, 10, 1.9, "damage"),
Skill("necro_drain", "Похищение жизни", 5, 10, 20, 1.4, "life_drain"),
Skill("necro_summon", "Страж могилы", 15, 30, 35, 0.8,
"summon", 15, "self"),
),
),
HeroClass.PALADIN: ClassDefinition(
"Паладин", primary(10, 4, 9, 12, 5),
primary(2.5, 0.5, 2, 3.5, 0.5), 90, 8, False,
(
Skill("paladin_smite", "Капа", 1, 5, 10, 2.0, "damage"),
Skill("paladin_heal", "Святой свет", 5, 12, 25, 1.8,
"heal", 0, "lowest_hp_ally"),
Skill("paladin_aegis", "Эгида", 15, 28, 40, 0.25,
"max_hp_shield", 8, "party"),
),
),
})

```

```

def total_exp_for_level(level: int) -> int:
if type(level) is not int or level < 1:
raise ValueError("Уровень должен быть целым >= 1")
return XP_BASE * (level - 1) * level * (2 * level - 1) // 6

```

```
def level_from_exp(total_exp: int) -> int:
    if type(total_exp) is not int or total_exp < 0:
```

```
        raise ValueError("Опыт должен быть целым >= 0")
```

```
    low, high = 1, 2
```

```
    while total_exp_for_level(high) <= total_exp:
```

```
        low, high = high, high * 2
```

```
    while low + 1 < high:
```

```
        middle = (low + high) // 2
```

```
        if total_exp_for_level(middle) <= total_exp:
```

```
            low = middle
```

```
        else:
```

```
            high = middle
```

```
    return low
```

```
@dataclass(frozen=True)
```

```
class HeroBuild:
```

```
    hero_id: str
```

```
    hero_class: HeroClass | str
```

```
    equipment: tuple[Item, ...] = ()
```

```
    allocated_stats: tuple[int, ...] = (0, 0, 0, 0, 0)
```

```
    def __post_init__(self) -> None:
```

```
        if not isinstance(self.hero_id, str) or not self.hero_id:
```

```
            raise ValueError("Необходим hero_id")
```

```
        object.__setattr__(self, "hero_class", HeroClass(self.hero_class))
```

```
        if type(self.equipment) is not tuple:
```

```
            raise ValueError("equipment должна быть tuple")
```

```
        if not all(isinstance(item, Item) for item in self.equipment):
```

```
            raise ValueError("Некорректная экипировка")
```

```
        if len({item.slot for item in self.equipment}) != len(self.equipment):
```

```
            raise ValueError("Слот занят дважды")
```

```
        if len({item.item_id for item in self.equipment}) != len(self.equipment):
```

```
            raise ValueError("Предмет экипирован дважды")
```

```
        if (
```

```
            type(self.allocated_stats) is not tuple
```

```
            or len(self.allocated_stats) != 5
```

```
            or any(type(v) is not int or v < 0 for v in self.allocated_stats)
```

```

):
    raise ValueError("Некорректное распределение характеристик")

def equip(self, item: Item) -> tuple[HeroBuild, Item | None]:
    previous = next(
        (entry for entry in self.equipment if entry.slot == item.slot),
        None,
    )
    equipment = tuple(
        entry for entry in self.equipment if entry.slot != item.slot
    ) + (item,)
    return replace(self, equipment=equipment), previous

```

@dataclass(frozen=True)

class SecondaryStats:

Поля, используемые старыми модулями.

physical_damage: float

magical_damage: float

armor: float

max_hp: int

max_mana: int

crit_chance: float

crit_multiplier: float

evasion: float

```

magic_resistance: float = 0.0
lifesteal: float = 0.0

attack_speed: float = 1.0
basic_requirement_reduction: int = 0
cooldown_reduction: float = 0.0
skill_range: float = 0.0
basic_attack_range: float = 140.0
cast_speed: float = 1.0

physical_enhancement: float = 0.0
fire_enhancement: float = 0.0
cold_enhancement: float = 0.0
lightning_enhancement: float = 0.0
chaos_enhancement: float = 0.0

```

```

multistrike: float = 0.0
projectile_count: int = 0
melee_damage: float = 0.0
projectile_damage: float = 0.0
aoe_damage: float = 0.0
projectile_speed: float = 0.0
summon_damage: float = 0.0

fire_res: float = 0.0
cold_res: float = 0.0
lightning_res: float = 0.0
chaos_res: float = 0.0
block_chance: float = 0.0
elemental_dodge: float = 0.0
elemental_block: float = 0.0
hp_regen: float = 0.0
damage_absorption: float = 0.0

movement_speed: int = 1000
aoe_enhancement: float = 0.0
skill_duration: float = 0.0
hp_per_hit: float = 0.0
hp_per_kill: float = 0.0
skill_heal: float = 0.0
all_skill_level: int = 0
exp_gain: float = 0.0
additional_exp: int = 0

def resistance(self, element: str) -> float:
    return {
        "physical": 0.0,
        "fire": self.fire_res,
        "cold": self.cold_res,
        "lightning": self.lightning_res,
        "chaos": self.chaos_res,
        "magic": self.magic_resistance,
    }[element]

def elemental_bonus(self, element: str) -> float:
    return {
        "physical": 0.0, # Уже включён в physical_damage.

```

```

        "fire": self.fire_enhancement,
        "cold": self.cold_enhancement,
        "lightning": self.lightning_enhancement,
        "chaos": self.chaos_enhancement,
        "magic": 0.0, # magical_bonus включён в magical_damage.
    }[element]

```

```
def _clamp(value: float, maximum: float) -> float:
```

```
    return max(0.0, min(maximum, value))
```

```
def experience_awards(
```

```
    base_per_wave: int,
```

```
    waves: int,
```

```
    stats: Sequence[SecondaryStats],
```

```
    cursor: int = 0,
```

```
) -> tuple[tuple[int, ...], int]:
```

```
    """
```

Распределение за несколько волн эквивалентно последовательному

начислению за каждую волну, включая округление индивидуальных бонусов.

```
    """
```

```
    if (
```

```
        type(base_per_wave) is not int or base_per_wave < 0
```

```
        or type(waves) is not int or waves < 0
```

```
        or not stats or not 0 <= cursor < len(stats)
```

```
    ):

```

```
        raise ValueError("Некорректные параметры распределения XP")
```

```
    count = len(stats)
```

```
    base, remainder = divmod(base_per_wave, count)
```

```
    extra_units = remainder * waves
```

```
    common_extra, tail = divmod(extra_units, count)
```

```
    result = []
```

```
    for index, secondary in enumerate(stats):
```

```
        extra_waves = common_extra + (
```

```
            1 if (index - cursor) % count < tail else 0
```

```
        )
```

```
        multiplier = max(
```

```
            Fraction(0),
```

```

        1 + Fraction(str(secondary.exp_gain)),
    )
    normal_award = math.floor(
        (base + secondary.additional_exp) * multiplier
    )
    extra_award = math.floor(
        (base + 1 + secondary.additional_exp) * multiplier
    )
    result.append(
        (waves - extra_waves) * normal_award
        + extra_waves * extra_award
    )

return tuple(result), (cursor + extra_units) % count

```

name, SecondaryStats attribute, display format.

```

ATTACK_ROWS = (
    ("Attack Damage", "physical_damage", "number"),
    ("Attack Speed", "attack_speed", "speed"),
    ("Critical Chance", "crit_chance", "percent"),
    ("Critical Damage", "crit_multiplier", "percent"),
    ("Basic Attack Requirement Reduction", "basic_requirement_reduction", "int"),
    ("Cooldown Reduction", "cooldown_reduction", "percent"),
    ("Skill Range", "skill_range", "percent"),
    ("Basic Attack Range", "basic_attack_range", "number"),
    ("Cast Speed", "cast_speed", "speed"),
    ("Physical Damage Enhancement", "physical_enhancement", "percent"),
    ("Fire Damage Enhancement", "fire_enhancement", "percent"),
    ("Cold Damage Enhancement", "cold_enhancement", "percent"),
    ("Lightning Damage Enhancement", "lightning_enhancement", "percent"),
    ("Chaos Damage Enhancement", "chaos_enhancement", "percent"),
    ("Multistrike Enhancement", "multistrike", "speed"),
    ("Projectile Count Enhancement", "projectile_count", "int"),
    ("Increase Melee Damage", "melee_damage", "percent"),
    ("Increase Projectile Damage", "projectile_damage", "percent"),
    ("Increase Area Of Effect Damage", "aoe_damage", "percent"),

```



```
("Increase Projectile Speed", "projectile_speed", "percent"),
("Increase Summon Damage", "summon_damage", "percent"),
)
```

```
DEFENSE_ROWS = (
("Max HP", "max_hp", "int"),
("Armor", "armor", "number"),
("Fire Resistance", "fire_res", "percent"),
("Cold Resistance", "cold_res", "percent"),
("Lightning Resistance", "lightning_res", "percent"),
("Chaos Resistance", "chaos_res", "percent"),
("Dodge Chance", "evasion", "percent"),
("Block Chance", "block_chance", "percent"),
("Elemental Dodge Chance", "elemental_dodge", "percent"),
("Elemental Block Chance", "elemental_block", "percent"),
("HP Regen Per Sec", "hp_regen", "number"),
("Damage Absorption", "damage_absorption", "number"),
)
```

```
UTILITY_ROWS = (
("Movement Speed", "movement_speed", "int"),
("Area of Effect Enhancement", "aoe_enhancement", "total_percent"),
("Skill Duration Increase", "skill_duration", "percent"),
("HP Per Hit Enhancement", "hp_per_hit", "number"),
("Add HP Per Kill", "hp_per_kill", "number"),
("Life Leech Enhancement", "lifesteal", "percent"),
("Skill Heal Increase", "skill_heal", "percent"),
("Add All Skill Level", "all_skill_level", "int"),
("Exp Gain Enhancement", "exp_gain", "total_percent"),
("Additional Exp Enhancement", "additional_exp", "int"),
)
```

```
DETAILED_REGISTRY = {
"Attack": ATTACK_ROWS,
"Defense": DEFENSE_ROWS,
"Utility": UTILITY_ROWS,
}
```

```

@dataclass(frozen=True)
class Hero:
    hero_id: str
    name: str
    hero_class: HeroClass | str
    total_exp: int = 0
    equipment: tuple[Item, ...] = ()
    allocated_stats: tuple[int, ...] = (0, 0, 0, 0, 0)

```

```

def __post_init__(self) -> None:
    object.__setattr__(self, "hero_class", HeroClass(self.hero_class))
    HeroBuild(
        self.hero_id, self.hero_class,
        self.equipment, self.allocated_stats,
    )
    if not isinstance(self.name, str) or not self.name:
        raise ValueError("Герою необходимо имя")
    level_from_exp(self.total_exp)
    if self.free_stat_points < 0:
        raise ValueError("Распределено больше очков, чем заработано")

@property
def definition(self) -> ClassDefinition:
    return CLASS_DEFINITIONS[self.hero_class]

@property
def level(self) -> int:
    return level_from_exp(self.total_exp)

@property
def earned_stat_points(self) -> int:
    return (self.level - 1) * STAT_POINTS_PER_LEVEL

@property
def free_stat_points(self) -> int:
    return self.earned_stat_points - sum(self.allocated_stats)

@property
def exp_in_level(self) -> int:
    return self.total_exp - total_exp_for_level(self.level)

```

```

@property
def exp_to_next_level(self) -> int:
    return total_exp_for_level(self.level + 1) - self.total_exp

@property
def skills(self) -> tuple[Skill, ...]:
    return tuple(
        skill for skill in self.definition.skills
        if skill.unlock_level <= self.level
    )

def gain_exp(self, amount: int) -> Hero:
    if type(amount) is not int or amount < 0:
        raise ValueError("Количество XP должно быть целым >= 0")
    return replace(self, total_exp=self.total_exp + amount)

def primary_stats(self, bonuses: StatBlock = StatBlock()) -> StatBlock:
    stats = (
        self.definition.base
        + self.definition.growth.scaled(self.level - 1)
        + CLASS_BONUSES[self.hero_class]
        + bonuses
        + StatBlock(**dict(zip(PRIMARY_STAT_KEYS, self.allocated_stats)))
    )
    for item in self.equipment:
        stats = stats + item.stats
    return stats

def secondary_stats(
    self,
    bonuses: StatBlock = StatBlock(),
) -> SecondaryStats:
    s = self.primary_stats(bonuses)
    has_shield = any(item.kind == ItemKind.SHIELD for item in self.equipment)

    physical = (
        5 + 2.0 * s.strength + 1.2 * s.agility + s.damage
    ) * max(0.0, 1 + s.physical_bonus)

    magical = (
        5 + 2.5 * s.intelligence + 0.3 * s.luck + s.damage
    ) * max(0.0, 1 + s.magical_bonus)

```

```

fire_res = _clamp(0.0015 * s.vitality + s.fire_res, 0.75)
cold_res = _clamp(0.0015 * s.vitality + s.cold_res, 0.75)
lightning_res = _clamp(0.0015 * s.vitality + s.lightning_res, 0.75)
chaos_res = _clamp(0.0010 * s.vitality + s.chaos_res, 0.75)

return SecondaryStats(
    physical_damage=max(0.0, physical),
    magical_damage=max(0.0, magical),
    armor=max(
        0.0,
        self.definition.base_armor
        + 1.5 * s.vitality + 0.5 * s.strength + s.armor,
    ),
    max_hp=max(1, math.floor(
        (self.definition.base_hp + 12 * s.vitality + s.max_hp)
        * max(0.0, 1 + s.hp_bonus)
    )),
    max_mana=max(0, math.floor(30 + 8 * s.intelligence + s.mana)),
    crit_chance=_clamp(
        0.05 + 0.001 * s.agility + 0.002 * s.luck + s.crit_chance,
        0.75,
    ),
    crit_multiplier=max(1.0, 1.5 + 0.005 * s.strength + s.crit_damage),
    evasion=_clamp(0.0015 * s.agility + s.evasion, 0.60),
    magic_resistance=(fire_res + cold_res + lightning_res) / 3,
    lifesteal=_clamp(
        CLASS_LIFESTEAL[self.hero_class] + s.life_leech, 1.0
    ),
    attack_speed=max(0.2, 1 + 0.015 * s.agility + s.attack_speed),
    basic_requirement_reduction=max(0, math.floor(s.basic_requirement_reduction)),
    cooldown_reduction=_clamp(0.002 * s.intelligence + s.cooldown_reduction, 0.75),
    skill_range=max(-0.8, s.skill_range),
    basic_attack_range=max(20.0, CLASS_RANGE[self.hero_class] + s.basic_attack_range),
    cast_speed=max(0.2, 1 + 0.01 * s.intelligence + s.cast_speed),
    physical_enhancement=s.physical_bonus,
    fire_enhancement=s.fire_bonus,
    cold_enhancement=s.cold_bonus,
    lightning_enhancement=s.lightning_bonus,
    chaos_enhancement=s.chaos_bonus,
    multistrike=max(0.0, s.multistrike),
    projectile_count=max(0, math.floor(s.projectile_count)),

```

```

        melee_damage=s.melee_bonus,
        projectile_damage=s.projectile_bonus,
        aoe_damage=s.aoe_damage,
        projectile_speed=max(-0.8, s.projectile_speed),
        summon_damage=s.summon_damage,
        fire_res=fire_res,
        cold_res=cold_res,
        lightning_res=lightning_res,
        chaos_res=chaos_res,
        block_chance=_clamp(
            (0.15 if has_shield else 0.0)
            + 0.001 * s.strength + s.block_chance, 0.75,
        ),
        elemental_dodge=_clamp(s.elemental_dodge, 0.60),
        elemental_block=_clamp(s.elemental_block, 0.75),
        hp_regen=max(0.0, 0.25 * s.vitality + s.hp_regen),
        damage_absorption=max(0.0, s.damage_absorption),
        movement_speed=max(100, 1000 + int(10 * s.agility + s.movement_speed)),
        aoe_enhancement=max(-0.9, s.aoe_enhancement),
        skill_duration=max(-0.9, s.skill_duration),
        hp_per_hit=max(0.0, s.hp_per_hit),
        hp_per_kill=max(0.0, s.hp_per_kill),
        skill_heal=max(-1.0, s.skill_heal),
        all_skill_level=max(0, math.floor(s.all_skill_level)),
        exp_gain=max(-1.0, s.exp_gain),
        additional_exp=max(0, math.floor(s.additional_exp)),
    )

def detailed_stats(
    self,
    bonuses: StatBlock = StatBlock(),
    *,
    secondary: SecondaryStats | None = None,
) -> dict[str, list[tuple[str, str]]:
    stats = secondary if secondary is not None else self.secondary_stats(bonuses)

    def formatted(value: float, mode: str) -> str:
        if mode == "percent":
            return f"{value * 100:.1f}%"
        if mode == "total_percent":
            return f"{(1 + value) * 100:.1f}%"
        if mode == "int":

```

```

        return f"{int(value):,}"
    if mode == "speed":
        return f"{value:.2f}"
    return f"{value:,.1f}"

    return {
        section: [
            (name, formatted(getattr(stats, attribute), mode))
            for name, attribute, mode in rows
        ]
        for section, rows in DETAILED_REGISTRY.items()
    }

    @classmethod
    def from_state(
        cls,
        snapshot: HeroState,
        build: HeroBuild | None = None,
    ) -> Hero:
        build = build or default_build(snapshot.hero_id)
        if snapshot.hero_id != build.hero_id:
            raise ValueError("Снимок и сборка принадлежат разным героям")
        return cls(
            snapshot.hero_id, snapshot.name, build.hero_class,
            snapshot.total_exp, build.equipment, build.allocated_stats,
        )

```

```
def default_build(hero_id: str) -> HeroBuild:
```

```
    legacy = {
```

```
        "warlock": HeroClass.NECROMANCER,
```

```
        "priest": HeroClass.PALADIN,
```

```
    }
```

```
    if hero_id in legacy:
```

```
        hero_class = legacy[hero_id]
```

```
    else:
```

```
        try:
```

```
            hero_class = HeroClass(hero_id)
```

```
        except ValueError:
```

```
            hero_class = HeroClass.KNIGHT
```

```

return HeroBuild(hero_id, hero_class)

def build_from_dict(payload: dict[str, Any]) -> HeroBuild:
    raw = dict(payload)
    raw["equipment"] = tuple(
        item_from_dict(item) for item in raw.get("equipment", ())
    )
    raw["allocated_stats"] = tuple(raw.get("allocated_stats", (0, 0, 0, 0, 0)))
    return HeroBuild(**raw)

```

Файл: models/item.py

```

`python
from future import annotations

import random
from dataclasses import dataclass, field
from enum import Enum
from typing import Any

from models.stats import StatBlock

class Rarity(str, Enum):
    COMMON = "common"
    UNCOMMON = "uncommon"
    RARE = "rare"
    LEGENDARY = "legendary"
    IMMORTAL = "immortal"
    MYTHIC = "mythic"

    RARITY_ORDER = tuple(Rarity)

    RARITY_POWER = {
        Rarity.COMMON: 1.0,
        Rarity.UNCOMMON: 1.25,
        Rarity.RARE: 1.65,
        Rarity.LEGENDARY: 2.40,

```

```
Rarity.IMMORTAL: 3.50,  
Rarity.MYTHIC: 5.00,  
}
```

```
class EquipmentSlot(str, Enum):
```

```
WEAPON = "weapon"
```

```
OFFHAND = "offhand"
```

```
HEAD = "head"
```

```
CHEST = "chest"
```

```
GLOVES = "gloves"
```

```
BOOTS = "boots"
```

```
BELT = "belt"
```

```
AMULET = "amulet"
```

```
RING_1 = "ring_1"
```

```
RING_2 = "ring_2"
```

```
class ItemKind(str, Enum):
```

```
GENERIC = "generic"
```

```
WEAPON = "weapon"
```

```
FOCUS = "focus"
```

```
SHIELD = "shield"
```

```
ACT_IDS = ("crypts", "forest", "caldera", "citadel")
```

```
ACT_NAMES = {
```

```
"crypts": "Катакомбы Падших",
```

```
"forest": "Осквернённая Чаща",
```

```
"caldera": "Пепельная Кальдера",
```

```
"citadel": "Цитадель Бездны",
```

```
}
```

```
ACT_LORE = {
```

```
"crypts": (
```

```
"На металле проступают имена тех, чьи могилы давно забыты.",
```

```
"Холод усыпальницы остался внутри, даже когда погасли погребальные свечи.",
```

```
"Хранитель катакомб слышал шёпот этой реликвии прежде собственного имени.",
```

```
),
```

```
"forest": (
```



```

"Корни оплели реликвию, но не смогли поглотить заключённую в ней волю.",
"В тишине слышно, как под кожей предмета движется ядовитый сок.",
"Чаща возвращает свои дары только тем, кого уже считает добычей.",
),
"caldera": (
"Последний удар кузнеца растворился в грохоте извержения.",
"В трещинах ещё течёт огонь, не знающий ни воздуха, ни времени.",
"Обсидиан помнит форму пламени, из которого был рождён.",
),
"citadel": (
"Эта вещь отбрасывает тень в сторону ещё не наступившего рассвета.",
"За гранью камня мерцает зал, которого нет ни на одной карте.",
"Создатель реликвии исчез из истории, но его клятва продолжает действовать.",
),
}

```

```

SLOT_NAMES = {
EquipmentSlot.WEAPON: "Клинок",
EquipmentSlot.OFFHAND: "Фокус",
EquipmentSlot.HEAD: "Венец",
EquipmentSlot.CHEST: "Доспех",
EquipmentSlot.GLOVES: "Перчатки",
EquipmentSlot.BOOTS: "Сапоги",
EquipmentSlot.BELT: "Пояс",
EquipmentSlot.AMULET: "Амулет",
EquipmentSlot.RING_1: "Кольцо",
EquipmentSlot.RING_2: "Перстень",
}

```

```

SLOT_BASE_STATS = {
EquipmentSlot.WEAPON: StatBlock(damage=8),
EquipmentSlot.OFFHAND: StatBlock(armor=4, mana=8),
EquipmentSlot.HEAD: StatBlock(armor=3, intelligence=1),
EquipmentSlot.CHEST: StatBlock(armor=8, max_hp=20),
EquipmentSlot.GLOVES: StatBlock(armor=2, agility=1),
EquipmentSlot.BOOTS: StatBlock(armor=2, agility=1),
}

```

```
EquipmentSlot.BELT: StatBlock(max_hp=15, vitality=1),
EquipmentSlot.AMULET: StatBlock(intelligence=2, mana=10),
EquipmentSlot.RING_1: StatBlock(luck=1, damage=2),
EquipmentSlot.RING_2: StatBlock(luck=1, damage=2),
}
```

```
@dataclass(frozen=True)
class AffixDefinition:
    affix_id: str
    name: str
    stats: StatBlock

    def affix(
        affix_id: str,
        name: str,
        stats: float,
    ) -> AffixDefinition:
        return AffixDefinition(affix_id, name, StatBlock(stats))
```

Идентификаторы предыдущих версий сохранены.

```
BASE_PREFIXES = (
    affix("brutal", "Жестокий", strength=2),
    affix("nimble", "Проворный", agility=2),
    affix("occult", "Оккультный", intelligence=2),
    affix("stout", "Непоколебимый", vitality=2),
    affix("lucky", "Благословенный", luck=2),
    affix("sharp", "Острый", damage=3),
    affix("flaming", "Пылающий", fire_bonus=0.04),
    affix("frozen", "Ледяной", cold_bonus=0.04),
    affix("storm", "Грозовой", lightning_bonus=0.04),
    affix("void", "Пустотный", chaos_bonus=0.04),
    affix("swift", "Стремительный", attack_speed=0.04),
    affix("echoing", "Отражённый", multistrike=0.08),
    affix("volley", "Многоликий", projectile_count=1),
    affix("far", "Дальновидный", basic_attack_range=8, skill_range=0.04),
```

```

affix("ritual", "Ритуальный", cast_speed=0.04),
affix("sweeping", "Размашистый", melee_bonus=0.04),
affix("ballistic", "Пронзающий", projectile_bonus=0.04),
affix("devastating", "Опустошающий", aoe_damage=0.04),
affix("summoner", "Призывающий", summon_damage=0.04),
affix("efficient", "Неутомимый", basic_requirement_reduction=1),
)

```

```

BASE_SUFFIXES = (
affix("of_war", "Войны", physical_bonus=0.02),
affix("of_ether", "Эфира", magical_bonus=0.02),
affix("of_life", "Жизни", max_hp=12),
affix("of_precision", "Точности", crit_chance=0.005),
affix("of_shadow", "Тени", evasion=0.005),
affix("of_ruin", "Разрушения", crit_damage=0.03),
affix("of_embers", "Углей", fire_res=0.025),
affix("of_winter", "Зимы", cold_res=0.025),
affix("of_thunder", "Грома", lightning_res=0.025),
affix("of_silence", "Безмолвия", chaos_res=0.025),
affix("of_recovery", "Восстановления", hp_regen=0.5),
affix("of_time", "Времени", cooldown_reduction=0.015),
affix("of_guard", "Стража", block_chance=0.015),
affix("of_feasting", "Пира", hp_per_hit=1.5),
affix("of_execution", "Казни", hp_per_kill=4),
affix("of_vampire", "Вампира", life_leech=0.01),
affix("of_wisdom", "Мудрости", exp_gain=0.025),
affix("of_learning", "Познания", additional_exp=2),
affix("of warding", "Оберега", damage_absorption=1.5),
affix("of_mirage", "Миража", elemental_dodge=0.01),
affix("of_aegis", "Эгиды", elemental_block=0.015),
affix("of_haste", "Спешки", movement_speed=25),
affix("of_expansion", "Расширения", aoe_enhancement=0.04),
affix("of_eternity", "Вечности", skill_duration=0.04),
affix("of_mercy", "Милосердия", skill_heal=0.04),
affix("of_mastery", "Мастерства", all_skill_level=1),
affix("of_flight", "Полёта", projectile_speed=0.05),
)

```

```

)

ACT_PREFIXES = {
"crypts": (
    affix("crypt_bone", "Костяной", armor=3, vitality=1),
    affix("crypt_funeral", "Погребальный", chaos_bonus=0.035),
    affix("crypt_catacomb", "Катакомбный", cold_res=0.03),
    affix("crypt_grave", "Могильный", hp_per_kill=3),
    affix("crypt_pale", "Бледный", cold_bonus=0.035),
    affix("crypt_ossuary", "Оссуарный", summon_damage=0.04),
),
"forest": (
    affix("forest_corrupt", "Осквернённый", chaos_bonus=0.035),
    affix("forest_thorn", "Терновый", physical_bonus=0.03),
    affix("forest_whisper", "Шепчущий", evasion=0.015),
    affix("forest_moss", "Мшистый", hp_regen=0.6),
    affix("forest_feral", "Звериный", attack_speed=0.04),
    affix("forest_spore", "Споровый", skill_duration=0.05),
),
"caldera": (
    affix("caldera_obsidian", "Обсидиановый", armor=4, crit_damage=0.02),
    affix("caldera_magma", "Магматический", fire_bonus=0.045),
    affix("caldera_fireborn", "Огнерождённый", fire_res=0.035),
    affix("caldera_cinder", "Угольный", damage=3),
    affix("caldera_furnace", "Горновой", melee_bonus=0.04),
    affix("caldera_scorched", "Опалённый", aoe_damage=0.04),
),
"citadel": (
    affix("citadel_ether", "Эфирный", magical_bonus=0.04),
    affix("citadel_ancient", "Предвечный", all_skill_level=1),
    affix("citadel_astral", "Астральный", lightning_bonus=0.04),
    affix("citadel_rift", "Разломный", projectile_count=1),
    affix("citadel_timeless", "Вневременной", cooldown_reduction=0.02),
    affix("citadel_sovereign", "Державный", block_chance=0.02),
),
}

```

```

ACT_SUFFIXES = {
"crypts": (
    affix("crypt_decay", "Тлена", chaos_res=0.03),
    affix("crypt_tomb", "Усыпальницы", max_hp=16),
    affix("crypt_coldblood", "Хладной крови", life_leech=0.012),
    affix("crypt_requiem", "Реквиема", skill_heal=0.04),
    affix("crypt_silence", "Мёртвой тишины", elemental_dodge=0.015),
    affix("crypt_ancestors", "Предков", exp_gain=0.03),
),
"forest": (
    affix("forest_rot", "Гнили", hp_per_hit=2),
    affix("forest_dew", "Ядовитой росы", chaos_bonus=0.04),
    affix("forest_predator", "Хищника", crit_chance=0.008),
    affix("forest_roots", "Глубоких корней", damage_absorption=2),
    affix("forest_hunt", "Дикой охоты", projectile_bonus=0.04),
    affix("forest_moon", "Бледной луны", movement_speed=35),
),
"caldera": (
    affix("caldera_ash", "Пепла", fire_res=0.03),
    affix("caldera_eruption", "Извержения", aoe_enhancement=0.06),
    affix("caldera_lava", "Жгучей лавы", fire_bonus=0.04),
    affix("caldera_sparks", "Искр", projectile_speed=0.06),
    affix("caldera_crucible", "Тигля", block_chance=0.02),
    affix("caldera_slag", "Чёрного шлака", armor=5),
),
"citadel": (
    affix("citadel_abyss", "Бездны", chaos_bonus=0.045),
    affix("citadel_fracture", "Разлома", elemental_block=0.02),
    affix("citadel_eternity", "Вечности", skill_duration=0.06),
    affix("citadel_stars", "Погасших звёзд", mana=18),
    affix("citadel_dominion", "Владычества", summon_damage=0.05),
    affix("citadel_memory", "Последней памяти", additional_exp=3),
),
}

PREFIXES = BASE_PREFIXES + tuple(

```

```

entry for entry in ACT_PREFIXES.values() for entry in entries
)

```

```

SUFFIXES = BASE_SUFFIXES + tuple(
entry for entry in ACT_SUFFIXES.values() for entry in entries
)

```

```

PREFIX_BY_ID = {entry.affix_id: entry for entry in PREFIXES}
SUFFIX_BY_ID = {entry.affix_id: entry for entry in SUFFIXES}

```

```

@dataclass(frozen=True)

```

```

class ItemArchetype:

```

```

    archetype_id: str

```

```

    name: str

```

```

    slot: EquipmentSlot

```

```

    kind: ItemKind

```

```

    visual: str

```

```

    stats: StatBlock

```

```

    favored_act: str | None = None

```

```

    gender: str = "m"

```

```

    def archetype(

```

```

        key: str,

```

```

        name: str,

```

```

        slot: EquipmentSlot,

```

```

        visual: str,

```

```

        ,

```

```

        kind: ItemKind = ItemKind.GENERIC,

```

```

        act: str | None = None,

```

```

        gender: str = "m",

```

```

        stats: float,

```

```

    ) -> ItemArchetype:

```

```

        return ItemArchetype(

```

```

            key, name, slot, kind, visual,

```

```

            StatBlock(*stats), act, gender,

```

```

        )

```

```

W = EquipmentSlot.WEAPON

```

O = EquipmentSlot.OFFHAND

```
ARCHETYPES = (  
    archetype("knight_sword", "Меч рыцаря", W, "sword",  
kind=ItemKind.WEAPON, damage=8, strength=1),  
    archetype("espadon", "Двуручный эспадон", W, "greatsword",  
kind=ItemKind.WEAPON, damage=12, melee_bonus=0.02),  
    archetype("falchion", "Изогнутый фальшион", W, "sabre",  
kind=ItemKind.WEAPON, damage=8, crit_chance=0.005),  
    archetype("assassin_dagger", "Кинжал убийцы", W, "dagger",  
kind=ItemKind.WEAPON, damage=5, attack_speed=0.04),  
    archetype("abyss_stiletto", "Стилет Бездны", W, "dagger",  
kind=ItemKind.WEAPON, act="citadel", damage=5, crit_damage=0.05),  
    archetype("twin_blades", "Парные клинки", W, "dual",  
kind=ItemKind.WEAPON, gender="p", damage=6, multistrike=0.05),  
    archetype("archmage_staff", "Посох архимара", W, "staff",  
kind=ItemKind.WEAPON, intelligence=3, damage=5, mana=10),  
    archetype("occult_wand", "Оккультный жезл", W, "wand",  
kind=ItemKind.WEAPON, damage=4, cast_speed=0.05),  
    archetype("soul_scythe", "Коса жнеца душ", W, "scythe",  
kind=ItemKind.WEAPON, act="crypts", gender="f",  
damage=7, chaos_bonus=0.03, summon_damage=0.03),  
    archetype("willow_bow", "Ивовый лук", W, "bow",  
kind=ItemKind.WEAPON, act="forest", damage=7, agility=1),  
    archetype("composite_bow", "Композитный лук", W, "bow",  
kind=ItemKind.WEAPON, damage=9, projectile_bonus=0.02),  
    archetype("heavy_crossbow", "Тяжёлый арбалет", W, "crossbow",  
kind=ItemKind.WEAPON, damage=11, projectile_speed=0.04),  
    archetype("war_hammer", "Боевой молот", W, "hammer",  
kind=ItemKind.WEAPON, act="caldera", damage=10, strength=2),  
    archetype("rosewood_mace", "Палисандровая булава", W, "mace",  
kind=ItemKind.WEAPON, gender="f", damage=7, skill_heal=0.03),  
    archetype("righteous_flail", "Праведный цеп", W, "flail",  
kind=ItemKind.WEAPON, damage=8, lightning_bonus=0.03),  
  
    archetype("knight_targe", "Рыцарский тарч", O, "shield",  
kind=ItemKind.SHIELD, armor=8),
```

```

archetype("tower_shield", "Башенный щит", 0, "tower",
    kind=ItemKind.SHIELD, armor=12, damage_absorption=1),
archetype("steel_buckler", "Стальной баклер", 0, "buckler",
    kind=ItemKind.SHIELD, armor=5, block_chance=0.025),
archetype("grimoire", "Гримуар заклинателя", 0, "book",
    kind=ItemKind.FOCUS, intelligence=2, mana=14),
archetype("ancestor_skull", "Череп предка", 0, "skull",
    kind=ItemKind.FOCUS, act="crypts", summon_damage=0.04, mana=8),
archetype("storm_orb", "Сфера бури", 0, "orb",
    kind=ItemKind.FOCUS, gender="f", lightning_bonus=0.04, mana=10),
archetype("hunter_quiver", "Охотничий колчан", 0, "quiver",
    act="forest", projectile_bonus=0.04, attack_speed=0.02),
archetype("void_arrows", "Стрелы пустоты", 0, "arrows",
    act="citadel", gender="p", projectile_count=1),

archetype("plate_armor", "Тяжёлый панцирь", EquipmentSlot.CHEST, "plate",
    armor=12, block_chance=0.01, damage_absorption=1),
archetype("leather_armor", "Кожаный доспех", EquipmentSlot.CHEST, "leather",
    act="forest", armor=5, evasion=0.01,
    attack_speed=0.02, projectile_bonus=0.02),
archetype("silk_robe", "Тканевая мантия", EquipmentSlot.CHEST, "robe",
    gender="f", armor=2, mana=20,
    cooldown_reduction=0.01, cast_speed=0.03),

archetype("horned_helm", "Рогатый шлем", EquipmentSlot.HEAD, "helm",
    armor=5, strength=1),
archetype("shadow_hood", "Капюшон теней", EquipmentSlot.HEAD, "hood",
    evasion=0.015, crit_chance=0.005),
archetype("ritual_crown", "Ритуальная корона", EquipmentSlot.HEAD, "crown",
    gender="f", intelligence=2, mana=10),

archetype("steel_gauntlets", "Стальные рукавицы", EquipmentSlot.GLOVES,
    "gloves", gender="p", armor=4, strength=1),
archetype("hunter_gloves", "Перчатки охотника", EquipmentSlot.GLOVES,
    "gloves", gender="p", agility=2, attack_speed=0.02),

archetype("iron_boots", "Железные сапоги", EquipmentSlot.BOOTS,
    "boots", gender="p", armor=4, vitality=1),
archetype("shadow_boots", "Сапоги теневого пути", EquipmentSlot.BOOTS,
    "boots", gender="p", movement_speed=40, evasion=0.01),

archetype("war_belt", "Пояс воителя", EquipmentSlot.BELT,

```



```

        "belt", max_hp=18, strength=1),
archetype("runic_belt", "Рунический кушак", EquipmentSlot.BELT,
        "belt", mana=12, damage_absorption=1),
archetype("blood_amulet", "Кровавый амулет", EquipmentSlot.AMULET,
        "amulet", act="crypts", life_leech=0.01, max_hp=12),
archetype("star_amulet", "Звёздный медальон", EquipmentSlot.AMULET,
        "amulet", act="citadel", intelligence=2, exp_gain=0.02),
archetype("runic_ring", "Рунический перстень", EquipmentSlot.RING_1,
        "ring", luck=1, crit_chance=0.005),
archetype("ember_ring", "Перстень углей", EquipmentSlot.RING_2,
        "ring", act="caldera", fire_bonus=0.025, fire_res=0.02),
archetype("oath_ring", "Кольцо клятвы", EquipmentSlot.RING_2,
        "signet", gender="n", block_chance=0.01, skill_heal=0.02),
)

```

ARCHETYPE_BY_ID = {entry.archetype_id: entry for entry in ARCHETYPES}

ELITE_RARITY_WEIGHTS = (15, 30, 40, 12, 2.7, 0.3)

BOSS_RARITY_WEIGHTS = (0, 15, 48, 29, 7, 1)

def positive_int(name: str, value: int, minimum: int = 1) -> None:

if type(value) is not int or value < minimum:

raise ValueError(f"{name} должен быть целым >= {minimum}")

def act_from_wave(wave: int) -> str:

positive_int("wave", wave)

return ACT_IDS[min((wave - 1) // 30, 3)]

def inflect_prefix(name: str, gender: str) -> str:

if gender == "m":

return name

```

if name.endswith("ий"):
    stem = name[:-2]
    if name.endswith(("ский", "цкий")):
        return stem + {"f": "ая", "n": "ое", "p": "ие"}[gender]
    return stem + {"f": "яя", "n": "ее", "p": "ие"}[gender]
if name.endswith(("ый", "ой")):

```

```

    stem = name[:-2]
    return stem + {"f": "ая", "n": "ое", "p": "ье"}[gender]
return name

```

```
@dataclass(frozen=True)
```

```
class Item:
```

```
    item_id: str
```

```
    rarity: Rarity | str
```

```
    source_wave: int
```

```
    sell_value: int
```

```

    slot: EquipmentSlot | str = EquipmentSlot.WEAPON
    level: int = 1
    base_name: str = "Реликвия Бездны"
    prefix_id: str | None = None
    suffix_id: str | None = None
    stats: StatBlock = field(default_factory=StatBlock)
    kind: ItemKind | str = ItemKind.GENERIC

    archetype_id: str | None = None
    origin_act: str | None = None
    lore: str = ""

    def __post_init__(self) -> None:
        if not isinstance(self.item_id, str) or not self.item_id:
            raise ValueError("Предмету необходим item_id")

        object.__setattr__(self, "rarity", Rarity(self.rarity))
        object.__setattr__(self, "slot", EquipmentSlot(self.slot))
        object.__setattr__(self, "kind", ItemKind(self.kind))

        positive_int("source_wave", self.source_wave)
        positive_int("sell_value", self.sell_value, 0)
        positive_int("level", self.level)

        if not isinstance(self.base_name, str) or not self.base_name:
            raise ValueError("Предмету необходимо имя")
        if not isinstance(self.stats, StatBlock):
            raise ValueError("stats должен быть StatBlock")
        if not isinstance(self.lore, str):

```

```

        raise ValueError("lore должна быть строкой")

    if self.prefix_id is not None and self.prefix_id not in PREFIX_BY_ID:
        raise ValueError("Неизвестный префикс")
    if self.suffix_id is not None and self.suffix_id not in SUFFIX_BY_ID:
        raise ValueError("Неизвестный суффикс")
    if self.origin_act is not None and self.origin_act not in ACT_IDS:
        raise ValueError("Неизвестный акт")
    if self.kind in (ItemKind.SHIELD, ItemKind.FOCUS):
        if self.slot != EquipmentSlot.OFFHAND:
            raise ValueError("Щит/фокус должен занимать OFFHAND")

    if self.archetype_id is not None:
        archetype_data = ARCHETYPE_BY_ID.get(self.archetype_id)
        if archetype_data is None:
            raise ValueError("Неизвестный архетип")
        if archetype_data.slot != self.slot:
            raise ValueError("Архетип не соответствует слоту")
        if archetype_data.kind != self.kind:
            raise ValueError("Архетип не соответствует типу предмета")

@property
def display_name(self) -> str:
    archetype_data = ARCHETYPE_BY_ID.get(self.archetype_id)
    gender = archetype_data.gender if archetype_data else "m"

    parts = []
    if self.prefix_id:
        parts.append(
            inflect_prefix(
                PREFIX_BY_ID[self.prefix_id].name, gender
            )
        )
    parts.append(self.base_name)
    if self.suffix_id:
        parts.append(SUFFIX_BY_ID[self.suffix_id].name)
    return " ".join(parts)

@property
def visual_key(self) -> str:
    archetype_data = ARCHETYPE_BY_ID.get(self.archetype_id)
    if archetype_data:
        return archetype_data.visual
    if self.kind == ItemKind.SHIELD:

```

```

        return "shield"
    return {
        EquipmentSlot.WEAPON: "sword",
        EquipmentSlot.OFFHAND: "orb",
        EquipmentSlot.HEAD: "helm",
        EquipmentSlot.CHEST: "plate",
        EquipmentSlot.GLOVES: "gloves",
        EquipmentSlot.BOOTS: "boots",
        EquipmentSlot.BELT: "belt",
        EquipmentSlot.AMULET: "amulet",
        EquipmentSlot.RING_1: "ring",
        EquipmentSlot.RING_2: "signet",
    }[self.slot]

```

```

def generate_item(
    level: int,
    rarity: Rarity | str,
    *,
    slot: EquipmentSlot | str | None = None,
    source_wave: int = 1,
    rng: random.Random | None = None,
    act: str | None = None,
    archetype_id: str | None = None,
) -> Item:
    positive_int("level", level)
    positive_int("source_wave", source_wave)

```

```

    generator = rng if rng is not None else random.Random()
    rarity = Rarity(rarity)
    act = act or act_from_wave(source_wave)

    if act not in ACT_IDS:
        raise ValueError("Неизвестный акт")

    requested_slot = EquipmentSlot(slot) if slot is not None else None

    if archetype_id is not None:
        base = ARCHETYPE_BY_ID[archetype_id]
        if requested_slot is not None and base.slot != requested_slot:
            raise ValueError("Архетип несовместим с запрошенным слотом")

```

```

else:
    if requested_slot is None:
        requested_slot = generator.choice(tuple(EquipmentSlot))

    candidates = [
        entry for entry in ARCHETYPES
        if entry.slot == requested_slot
    ]
    base = generator.choices(
        candidates,
        weights=[
            3.0 if entry.favored_act == act else 1.0
            for entry in candidates
        ],
        k=1,
    )[0]

    tier = RARITY_ORDER.index(rarity)
    power = RARITY_POWER[rarity]
    quality = generator.uniform(0.90, 1.10)

    stats = base.stats.scaled(
        (1 + (level - 1) * 0.12) * power * quality
    )

    prefix = None
    suffix = None

    if tier >= 1:
        pool = ACT_PREFIXES[act] if generator.random() < 0.75 else BASE_PREFIXES
        prefix = generator.choice(pool)

    if tier >= 2:
        pool = ACT_SUFFIXES[act] if generator.random() < 0.75 else BASE_SUFFIXES
        suffix = generator.choice(pool)

    affix_scale = power * (1 + (level - 1) * 0.025)
    if prefix:
        stats = stats + prefix.stats.scaled(affix_scale)
    if suffix:
        stats = stats + suffix.stats.scaled(affix_scale)

```

```

return Item(
    item_id=f"item-{generator.getrandbits(128):032x}",
    rarity=rarity,
    source_wave=source_wave,
    sell_value=max(1, int(10 * level * power)),
    slot=base.slot,
    level=level,
    base_name=base.name,
    prefix_id=prefix.affix_id if prefix else None,
    suffix_id=suffix.affix_id if suffix else None,
    stats=stats,
    kind=base.kind,
    archetype_id=base.archetype_id,
    origin_act=act,
    lore=generator.choice(ACT_LORE[act]),
)

```

```

def roll_enemy_loot(
    wave: int,
    rank: str,
    rng: random.Random,
) -> tuple[Item, ...]:
    positive_int("wave", wave)
    rank = getattr(rank, "value", rank)

```

```

if rank == "boss":
    count = rng.randint(2, 4)
    weights = BOSS_RARITY_WEIGHTS
elif rank == "elite":
    if rng.random() >= 0.60:
        return ()
    count = 1
    weights = ELITE_RARITY_WEIGHTS
else:
    return ()

# Отдельный баланс предметов за элитников и боссов.
level = 1 + (wave - 1) // 5
return tuple(
    generate_item(
        level,

```

```

    rng.choices(RARITY_ORDER, weights=weights, k=1)[0],
    source_wave=wave,
    act=act_from_wave(wave),
    rng=rng,
)
for _ in range(count)
)

```

```
def item_from_dict(payload: dict[str, Any]) -> Item:
```

```
raw = dict(payload)
```

```
raw["stats"] = StatBlock(raw.get("stats", {}))
```

```
return Item(raw)
```

```
,
```

Файл: `models/enemy.py`

```
`python
```

```
from future import annotations
```

```
import random
```

```
from dataclasses import dataclass
```

```
from enum import Enum
```

```
WAVES_PER_ACT = 30
```

```
BOSS_EVERY = 10
```

```
BOSS_TIME_LIMIT = 45.0
```

```
class Act(str, Enum):
```

```
CRYPTS = "crypts"
```

```
FOREST = "forest"
```

```
CALDERA = "caldera"
```

```
CITADEL = "citadel"
```

```
ACT_NAMES = {
```

```
Act.CRYPTS: "Крипты",
```

```
Act.FOREST: "Проклятый лес",
```

```
Act.CALDERA: "Кальдера",
```

```
Act.CITADEL: "Цитадель",
```

```

}

class EnemyRank(str, Enum):
    NORMAL = "normal"
    ELITE = "elite"
    BOSS = "boss"

class EnemyAffix(str, Enum):
    VAMPIRIC = "vampiric"
    STONE_SKIN = "stone_skin"
    EXTRA_FAST = "extra_fast"
    REFLECTIVE = "reflective"

@dataclass(frozen=True)
class EnemyTemplate:
    template_id: str
    name: str
    act: Act
    hp: float
    damage: float
    armor: float
    attack_interval: float
    speed: float
    magical: bool = False

BESTIARY = {
    Act.CRYPTS: (
        EnemyTemplate("skeleton", "Костяной страж", Act.CRYPTS,
            150, 20, 10, 1.7, 48),
        EnemyTemplate("wraith", "Призрак", Act.CRYPTS,
            110, 24, 3, 1.9, 58, True),
    ),
    Act.FOREST: (
        EnemyTemplate("wolf", "Теневой волк", Act.FOREST,
            160, 25, 6, 1.25, 76),
        EnemyTemplate("dryad", "Осквернённая дриада", Act.FOREST,
            180, 24, 12, 1.8, 44, True),
    ),
}

```



```

),
Act.CALDERA: (
    EnemyTemplate("imp", "Пепельный бес", Act.CALDERA,
        145, 32, 9, 1.4, 65, True),
    EnemyTemplate("golem", "Обсидиановый голем", Act.CALDERA,
        280, 35, 32, 2.2, 32),
),
Act.CITADEL: (
    EnemyTemplate("sentinel", "Страж Цитадели", Act.CITADEL,
        260, 38, 28, 1.6, 48),
    EnemyTemplate("inquisitor", "Инквизитор Бездны", Act.CITADEL,
        200, 43, 15, 1.8, 50, True),
),
}

BESTIARY[Act.CRYPTS] += (
    EnemyTemplate(
        "bat", "Могильная нетопырь", Act.CRYPTS,
        85, 15, 2, 1.15, 82,
    ),
)

BESTIARY[Act.FOREST] += (
    EnemyTemplate(
        "spider", "Паук-погребальщик", Act.FOREST,
        125, 23, 8, 1.30, 65,
    ),
)

BOSS_NAMES = {
    Act.CRYPTS: "Костяной Архонт",
    Act.FOREST: "Сердце Гнили",
    Act.CALDERA: "Владыка Пепла",
    Act.CITADEL: "Хранитель Разлома",
}

@dataclass

```

class Enemy:

"""Изменяемая боевая сущность. В save.json напрямую не сохраняется."""

```
enemy_id: str
template_id: str
name: str
act: Act
rank: EnemyRank
affixes: tuple[EnemyAffix, ...]

max_hp: float
hp: float
damage: float
armor: float
attack_interval: float
speed: float
magical: bool

x: float
y: float
attack_timer: float = 0.5
rage: float = 0.0
rage_max: float = 100.0

animation: str = "run"
animation_left: float = 0.0
corpse_left: float = 0.65

marked_until: float = 0.0
marked_bonus: float = 0.0

@property
def alive(self) -> bool:
    return self.hp > 0

@property
def is_boss(self) -> bool:
    return self.rank == EnemyRank.BOSS

@property
def enraged(self) -> bool:
```

```

    return self.is_boss and self.rage >= self.rage_max

@property
def effective_damage(self) -> float:
    return self.damage * (1.6 if self.enraged else 1.0)

@property
def effective_interval(self) -> float:
    return self.attack_interval * (0.7 if self.enraged else 1.0)

def act_for_wave(wave: int) -> Act:
    if type(wave) is not int or wave < 1:
        raise ValueError("Номер волны должен быть целым >= 1")
    index = min((wave - 1) // WAVES_PER_ACT, len(Act) - 1)
    return tuple(Act)[index]

def spawn_wave(
    wave: int,
    rng: random.Random | None = None,
    *,
    generation: int = 0,
) -> list[Enemy]:
    act = act_for_wave(wave)
    generator = rng if rng is not None else random.Random()

    boss_wave = wave % BOSS_EVERY == 0
    count = 1 if boss_wave else min(6, 3 + (wave - 1) // 20)

    # Общая прогрессия сложности, без сброса при переходе акта.
    hp_scale = (1.0 + 0.09 * (wave - 1)) ** 1.35
    damage_scale = (1.0 + 0.06 * (wave - 1)) ** 1.15

    result = []

    for index in range(count):
        template = generator.choice(BESTIARY[act])

        if boss_wave:
            rank = EnemyRank.BOSS
            affixes = ()

```

```

elif generator.random() < min(0.35, 0.12 + wave * 0.002):
    rank = EnemyRank.ELITE
    amount = 2 if wave >= 40 else 1
    affixes = tuple(generator.sample(tuple(EnemyAffix), amount))
else:
    rank = EnemyRank.NORMAL
    affixes = ()

hp_multiplier = {
    EnemyRank.NORMAL: 1.0,
    EnemyRank.ELITE: 1.9,
    EnemyRank.BOSS: 7.0,
}[rank]

attack_multiplier = {
    EnemyRank.NORMAL: 1.0,
    EnemyRank.ELITE: 1.35,
    EnemyRank.BOSS: 1.8,
}[rank]

armor = template.armor + wave * 0.7
interval = template.attack_interval
speed = template.speed

if EnemyAffix.STONE_SKIN in affixes:
    armor = armor * 2 + 25

if EnemyAffix.EXTRA_FAST in affixes:
    interval *= 0.70
    speed *= 1.55

max_hp = template.hp * hp_scale * hp_multiplier

result.append(Enemy(
    enemy_id=f"enemy-{generation}-{wave}-{index}",
    template_id=template.template_id,
    name=BOSS_NAMES[act] if boss_wave else template.name,
    act=act,
    rank=rank,
    affixes=affixes,
    max_hp=max_hp,
    hp=max_hp,

```

```

        damage=template.damage * damage_scale * attack_multiplier,
        armor=armor,
        attack_interval=interval,
        speed=speed,
        magical=template.magical,
        x=460.0 + index * 48,
        y=99.0 + (index % 2) * 12,
    ))
    return result

```

Файл: engine/state.py

```

`python
from future import annotations

import math
import time

from dataclasses import dataclass, field, replace
from fractions import Fraction
from typing import ClassVar

from PySide6.QtCore import QObject, QThread, Signal

import config as C
from models.hero import HeroBuild
from models.item import Item as ItemDrop

def require_int(name: str, value: int, minimum: int = 0) -> None:
    if type(value) is not int or value < minimum:
        raise ValueError(f'{name}: ожидается целое число >= {minimum}')

def require_number(
    name: str,
    value: float,
    minimum: float = 0.0,
    maximum: float | None = None,

```

) -> None:

if type(value) not in (int, float):

raise ValueError(f"{name}: ожидается число")

```
if not math.isfinite(value) or value < minimum:  
    raise ValueError(f"{name}: недопустимое значение {value}")
```

```
if maximum is not None and value > maximum:  
    raise ValueError(f"{name}: значение больше {maximum}")
```

@dataclass(frozen=True)

class HeroState:

hero_id: str

name: str

max_hp: int = 100

hp: int = 100

attack: int = 10

defense: int = 0

total_exp: int = 0

```
def __post_init__(self) -> None:  
    if not isinstance(self.hero_id, str) or not self.hero_id:  
        raise ValueError("У героя должен быть непустой hero_id")  
    if not isinstance(self.name, str) or not self.name:  
        raise ValueError("У героя должно быть имя")
```

```
    require_int("max_hp", self.max_hp, 1)  
    require_int("hp", self.hp)  
    require_int("attack", self.attack)  
    require_int("defense", self.defense)  
    require_int("total_exp", self.total_exp)
```

```
    if self.hp > self.max_hp:  
        raise ValueError("hp не может превышать max_hp")
```

@dataclass(frozen=True)

class FarmProfile:

average_wave_clear_seconds: float = C.DEFAULT_WAVE_CLEAR_SECONDS

gold_per_wave: int = C.DEFAULT_GOLD_PER_WAVE

```
exp_per_wave: int = C.DEFAULT_EXP_PER_WAVE
```

```
# 0.25 означает +25%, а не итоговый множитель x0.25.
```

```
gold_bonus: float = 0.0
```

```
sale_bonus: float = 0.0
```

```
drop_chance: float = C.DEFAULT_DROP_CHANCE
```

```
def __post_init__(self) -> None:
```

```
    require_number(
```

```
        "average_wave_clear_seconds",
```

```
        self.average_wave_clear_seconds,
```

```
        C.MIN_WAVE_CLEAR_SECONDS,
```

```
    )
```

```
    require_int("gold_per_wave", self.gold_per_wave)
```

```
    require_int("exp_per_wave", self.exp_per_wave)
```

```
    require_number("gold_bonus", self.gold_bonus)
```

```
    require_number("sale_bonus", self.sale_bonus)
```

```
    require_number("drop_chance", self.drop_chance, 0.0, 1.0)
```

```
def default_party() -> tuple[HeroState, ...]:
```

```
    return (
```

```
        HeroState("knight", "Рыцарь", 250, 250, 14, 20),
```

```
        HeroState("assassin", "Ассасин", 130, 130, 28, 5),
```

```
        HeroState("warlock", "Колдун", 110, 110, 32, 3),
```

```
        HeroState("priest", "Жрец", 150, 150, 12, 7),
```

```
    )
```

```
@dataclass(frozen=True)
```

```
class GameData:
```

```
    gold: int = 0
```

```
    cleared_waves: int = 0
```

```
# Рациональная доля незавершённой волны: "0", "1/2", ...
```

```
# Строка нужна для точного JSON round-trip.
```

```
wave_fraction: str = "0"
```

```
party: tuple[HeroState, ...] = field(default_factory=default_party)
```

```
stash: tuple[ItemDrop, ...] = ()
```

```
stash_capacity: int = C.DEFAULT_STASH_CAPACITY
```

```
farm: FarmProfile = field(default_factory=FarmProfile)
```

```

offline_cap_level: int = 0
offline_efficiency_level: int = 0

# Следующий герой, которому достанется остаточная единица XP.
exp_cursor: int = 0
last_active_timestamp: float = field(default_factory=time.time)

hero_builds: tuple[HeroBuild, ...] = ()
unlocked_runes: tuple[str, ...] = ()
backpack: tuple[ItemDrop, ...] = ()
reward_claims: tuple[str, ...] = ()

def __post_init__(self) -> None:
    require_int("gold", self.gold)
    require_int("cleared_waves", self.cleared_waves)
    require_int("stash_capacity", self.stash_capacity)
    require_int("offline_cap_level", self.offline_cap_level)
    require_int("offline_efficiency_level", self.offline_efficiency_level)
    require_int("exp_cursor", self.exp_cursor)
    require_number("last_active_timestamp", self.last_active_timestamp)

    if self.offline_cap_level >= len(C.OFFLINE_CAP_SECONDS):
        raise ValueError("Неизвестная ступень потолка оффлайна")
    if self.offline_efficiency_level >= len(C.OFFLINE_EFFICIENCIES):
        raise ValueError("Неизвестная ступень эффективности")

    if not isinstance(self.wave_fraction, str):
        raise ValueError("wave_fraction должна быть строкой")
    try:
        fraction = Fraction(self.wave_fraction)
    except (ValueError, ZeroDivisionError) as exc:
        raise ValueError("Некорректная доля волны") from exc
    if not 0 <= fraction < 1:
        raise ValueError("Доля волны должна находиться в [0, 1)")

    if type(self.party) is not tuple:
        raise ValueError("party должна быть tuple")
    if not 1 <= len(self.party) <= C.MAX_PARTY_SIZE:
        raise ValueError("Отряд должен содержать от 1 до 4 героев")
    if not all(isinstance(hero, HeroState) for hero in self.party):
        raise ValueError("Некорректные данные героев")
    if len({hero.hero_id for hero in self.party}) != len(self.party):

```



```

        raise ValueError("Повторяющиеся hero_id")
    if self.exp_cursor >= len(self.party):
        raise ValueError("exp_cursor вне состава отряда")

    if type(self.stash) is not tuple:
        raise ValueError("stash должна быть tuple")
    if not all(isinstance(item, ItemDrop) for item in self.stash):
        raise ValueError("Некорректные предметы")
    if len(self.stash) > self.stash_capacity:
        raise ValueError("Тайник переполнен")
    if len({item.item_id for item in self.stash}) != len(self.stash):
        raise ValueError("Повторяющиеся item_id")

    if not isinstance(self.farm, FarmProfile):
        raise ValueError("Некорректный профиль фарма")

    if type(self.hero_builds) is not tuple:
        raise ValueError("hero_builds должна быть tuple")
    if not all(isinstance(build, HeroBuild) for build in self.hero_builds):
        raise ValueError("Некорректные сборки героев")

    build_ids = [build.hero_id for build in self.hero_builds]
    if len(build_ids) != len(set(build_ids)):
        raise ValueError("Повторяющиеся сборки героев")

    if type(self.backpack) is not tuple:
        raise ValueError("backpack должна быть tuple")

    if not all(isinstance(item, ItemDrop) for item in self.backpack):
        raise ValueError("Некорректные предметы рюкзака")

    if len(self.backpack) > C.DEFAULT_BACKPACK_CAPACITY:
        raise ValueError("Рюкзак переполнен")

    all_item_ids = [
        item.item_id
        for item in (*self.stash, *self.backpack)
    ]
    all_item_ids.extend(
        item.item_id
        for build in self.hero_builds
        for item in build.equipment

```

```

    )
    if len(all_item_ids) != len(set(all_item_ids)):
        raise ValueError(
            "Предмет одновременно находится в нескольких контейнерах"
        )

    if type(self.unlocked_runes) is not tuple:
        raise ValueError("unlocked_runes должна быть tuple")
    if not all(
        isinstance(node_id, str) and node_id
        for node_id in self.unlocked_runes
    ):
        raise ValueError("Некорректные идентификаторы рун")
    if len(self.unlocked_runes) != len(set(self.unlocked_runes)):
        raise ValueError("Повторяющиеся руны")

@property
def current_wave(self) -> int:
    return self.cleared_waves + 1

@property
def offline_cap_seconds(self) -> int:
    return C.OFFLINE_CAP_SECONDS[self.offline_cap_level]

@property
def offline_efficiency(self) -> float:
    return C.OFFLINE_EFFICIENCIES[self.offline_efficiency_level]

@property
def computed_squad(self):
    # Локальный импорт исключает циклический импорт модулей.
    from engine.runes_tree import RunesTree
    return RunesTree().squad(self)

@property
def squad_attack(self) -> int:
    return int(sum(
        member.stats.magical_damage
        if member.hero.definition.magical
        else member.stats.physical_damage
        for member in self.computed_squad
    ))

```

```

@property
def squad_max_hp(self) -> int:
    return sum(
        member.stats.max_hp
        for member in self.computed_squad
    )

@property
def squad_hp(self) -> int:
    return sum(
        snapshot.hp * member.stats.max_hp // snapshot.max_hp
        for snapshot, member in zip(self.party, self.computed_squad)
    )

```

```

class _SingletonQObject(type(QObject)):
    _instances: ClassVar[dict[type, QObject]] = {}

```

```

def __call__(cls, *args, **kwargs):
    if cls not in cls._instances:
        cls._instances[cls] = super().__call__(*args, **kwargs)
    return cls._instances[cls]

```

```

class GameState(QObject, metaclass=_SingletonQObject):
    gold_changed = Signal(object) # Новый баланс: Python int.
    exp_gained = Signal(str, object) # hero_id, начисленный XP.
    item_dropped = Signal(object) # ItemDrop, помещённый в тайник.

```

```

# Пакетный контракт: первая пройденная волна, количество волн.
# Для активного боя количество будет равно 1.
wave_cleared = Signal(object, object)

party_changed = Signal(object)
stash_changed = Signal(object)
backpack_changed = Signal(object)
runes_changed = Signal(object)
builds_changed = Signal(object)
state_changed = Signal(object) # Полный GameData.
offline_completed = Signal(object) # OfflineResult.
state_loaded = Signal(object)

```

```

def __init__(self) -> None:
    super().__init__()
    self._data = GameData()

@classmethod
def instance(cls) -> GameState:
    return cls()

def assert_owner_thread(self) -> None:
    if QThread.currentThread() != self.thread():
        raise RuntimeError(
            "GameState разрешено изменять только в его Qt-потоме"
        )

@property
def data(self) -> GameData:
    return self._data

def load(self, data: GameData) -> None:
    """Вызывается при загрузке, а не для выдачи игровых наград."""
    self.assert_owner_thread()
    if not isinstance(data, GameData):
        raise TypeError("Ожидается GameData")

    self._data = data
    self.state_loaded.emit(data)
    self.state_changed.emit(data)

def commit(
    self,
    expected: GameData,
    updated: GameData,
    offline_result: object | None = None,
) -> None:
    """Применить снимок, только если исходное состояние ещё актуально."""
    self.assert_owner_thread()

    if self._data is not expected:
        raise RuntimeError("Попытка применить устаревший снимок")
    if not isinstance(updated, GameData):
        raise TypeError("Ожидается GameData")

```

```

previous = self._data

# Сначала заменяем все данные, только затем рассылаем события.
self._data = updated

if previous.gold != updated.gold:
    self.gold_changed.emit(updated.gold)

old_exp = {hero.hero_id: hero.total_exp for hero in previous.party}
for hero in updated.party:
    gained = hero.total_exp - old_exp.get(hero.hero_id, hero.total_exp)
    if gained > 0:
        self.exp_gained.emit(hero.hero_id, gained)

if previous.party != updated.party:
    self.party_changed.emit(updated.party)

if previous.stash != updated.stash:
    old_ids = {item.item_id for item in previous.stash}
    for item in updated.stash:
        if item.item_id not in old_ids:
            self.item_dropped.emit(item)
    self.stash_changed.emit(updated.stash)

if previous.backpack != updated.backpack:
    self.backpack_changed.emit(updated.backpack)

waves = updated.cleared_waves - previous.cleared_waves
if waves > 0:
    self.wave_cleared.emit(previous.current_wave, waves)

if previous.unlocked_runes != updated.unlocked_runes:
    self.runes_changed.emit(updated.unlocked_runes)

if previous.hero_builds != updated.hero_builds:
    self.builds_changed.emit(updated.hero_builds)

self.state_changed.emit(updated)

if offline_result is not None:
    self.offline_completed.emit(offline_result)

```

```

def add_gold(self, amount: int) -> None:
    require_int("amount", amount)
    before = self.data
    self.commit(before, replace(before, gold=before.gold + amount))

def spend_gold(self, amount: int) -> bool:
    self.assert_owner_thread()
    require_int("amount", amount)

    before = self.data
    if before.gold < amount:
        return False

    self.commit(before, replace(before, gold=before.gold - amount))
    return True

def set_farm_profile(self, profile: FarmProfile) -> None:
    before = self.data
    self.commit(before, replace(before, farm=profile))

def set_offline_upgrades(
    self,
    cap_level: int,
    efficiency_level: int,
) -> None:
    """Применение уже рассчитанных бонусов. Покупки – задача дерева рун."""
    before = self.data
    self.commit(
        before,
        replace(
            before,
            offline_cap_level=cap_level,
            offline_efficiency_level=efficiency_level,
        ),
    )

def set_party(self, party: tuple[HeroState, ...]) -> None:
    before = self.data
    self.commit(before, replace(before, party=party, exp_cursor=0))

```

Файл: engine/combat_core.py

```
`python
from future import annotations

import math
import random
from dataclasses import dataclass, field, replace
from fractions import Fraction

from PySide6.QtCore import QObject, Signal

import config as C
from engine.runes_tree import RunesTree
from engine.state import GameState
from models.enemy import (
    Act, BOSS_TIME_LIMIT, Enemy, EnemyAffix,
    act_for_wave, spawn_wave,
)
from models.hero import (
    Hero, HeroClass, SecondaryStats, Skill, experience_awards,
)
from models.item import generate_item

FIXED_STEP = 0.05
HERO_IFRAMES = 0.18
BARRIER_RATIO = 0.20
BETWEEN_WAVES = 1.0
DEFEAT_DELAY = 3.0
BASE_ATTACK_COST = 2.0
BASE_PROJECTILE_SPEED = 320.0

HERO_POSITIONS = (
    (78.0, 104.0), (108.0, 116.0),
    (136.0, 97.0), (164.0, 113.0),
)

BASIC_INTERVALS = {
```

```

HeroClass.KNIGHT: 1.20,
HeroClass.ASSASSIN: 0.75,
HeroClass.PYROMANCER: 1.40,
HeroClass.RANGER: 0.95,
HeroClass.NECROMANCER: 1.45,
HeroClass.PALADIN: 1.30,
}

ELEMENTS = {"physical", "fire", "cold", "lightning", "chaos", "magic"}

CLASS_ELEMENT = {
HeroClass.KNIGHT: "physical",
HeroClass.ASSASSIN: "physical",
HeroClass.PYROMANCER: "fire",
HeroClass.RANGER: "physical",
HeroClass.NECROMANCER: "chaos",
HeroClass.PALADIN: "physical",
}

ENEMY_ELEMENT = {
"wraith": "cold",
"dryad": "chaos",
"imp": "fire",
"inquisitor": "lightning",
}

def combined_chance(first: float, second: float) -> float:
    first = max(0.0, min(1.0, first))
    second = max(0.0, min(1.0, second))
    return 1 - (1 - first) * (1 - second)

@dataclass(frozen=True)
class Hit:
    damage: float
    critical: bool = False
    dodged: bool = False

    def roll_damage(

```



```

power: float,
armor: float,
*,
magical: bool = False,
crit_chance: float = 0.0,
crit_multiplier: float = 1.5,
evasion: float = 0.0,
rng: random.Random | None = None,
) -> Hit:
values = (power, armor, crit_chance, crit_multiplier, evasion)
if not all(math.isfinite(value) for value in values):
raise ValueError("Параметры урона должны быть конечными")
if power < 0 or armor < 0 or crit_multiplier < 1:
raise ValueError("Некорректные параметры урона")
if not 0 <= crit_chance <= 1 or not 0 <= evasion <= 1:
raise ValueError("Вероятности должны находиться в [0, 1]")

```

```

generator = rng if rng is not None else random.Random()
if generator.random() < evasion:
    return Hit(0.0, dodged=True)

critical = generator.random() < crit_chance
effective_armor = armor * (0.5 if magical else 1.0)
damage = power * 100 / (100 + effective_armor)
if critical:
    damage *= crit_multiplier
return Hit(damage, critical=critical)

```

```
@dataclass(frozen=True)
```

```
class DamageEvent:
```

```
target_id: str
```

```
x: float
```

```
y: float
```

```
amount: float
```

```
kind: str
```

```
critical: bool = False
```

```
@dataclass(frozen=True)
```

```

class ActorFrame:
    actor_id: str
    sprite_key: str
    name: str
    x: float
    y: float
    hp: float
    max_hp: float
    animation: str
    enemy: bool
    elite: bool = False
    boss: bool = False
    rage: float = 0.0
    mana: float = 0.0
    max_mana: float = 0.0

    @dataclass(frozen=True)
    class CombatFrame:
        wave: int
        act: Act
        phase: str
        time: float
        progress: float
        barrier: float
        barrier_max: float
        boss_remaining: float | None
        actors: tuple[ActorFrame, ...]

    @dataclass
    class HeroRuntime:
        hero: Hero
        stats: SecondaryStats
        hp: float
        mana: float
        x: float
        y: float

```

```

basic_timer: float = 0.25
skill_cooldowns: dict[str, float] = field(default_factory=dict)
invulnerable_until: float = 0.0

animation: str = "idle"
animation_left: float = 0.0
buffs: dict[str, tuple[float, float]] = field(default_factory=dict)

pending_skill: Skill | None = None
cast_left: float = 0.0

summon_left: float = 0.0
summon_timer: float = 0.0
summon_power: float = 0.0

shield: float = 0.0
shield_until: float = 0.0
second_wind_ready: float = 0.0

@property
def alive(self) -> bool:
    return self.hp > 0

def buff(self, name: str, now: float) -> float:
    until, amount = self.buffs.get(name, (0.0, 0.0))
    return amount if until > now else 0.0

```

```

@dataclass
class PendingImpact:
    owner_id: str
    enemy_id: str
    power: float
    element: str
    tags: frozenset[str]
    remaining: float
    drain: float = 0.0

```

```

@dataclass
class DamageOverTime:
    owner_id: str

```

```

enemy_id: str
power: float
remaining: float
magical: bool
timer: float = 1.0
element: str = "chaos"
tags: frozenset[str] = frozenset()

```

```

class CombatManager(QObject):
    frame_changed = Signal(object)
    damage_event = Signal(object)
    skill_used = Signal(str, str)
    wave_finished = Signal(object)
    party_defeated = Signal(object)

```

```

def __init__(
    self,
    state: GameState,
    *,
    rng: random.Random | None = None,
    parent: QObject | None = None,
) -> None:
    super().__init__(parent)
    self.state = state
    self.rng = rng if rng is not None else random.Random()
    self.runes = RunesTree()

    self.running = False
    self.phase = "stopped"
    self.time = 0.0

    self._accumulator = 0.0
    self._phase_left = 0.0
    self._generation = 0
    self._wave = state.data.current_wave
    self._wave_elapsed = 0.0
    self._wave_real_elapsed = 0.0
    self._wave_total_hp = 1.0
    self._continued_wave = False

    self._heroes: dict[str, HeroRuntime] = {}

```

```

self._enemies: list[Enemy] = []
self._dots: list[DamageOverTime] = []
self._impacts: list[PendingImpact] = []

self.barrier = 0.0
self.barrier_max = 0.0
self._effects = self.runes.effects(state.data.unlocked_runes)

@property
def heroes(self) -> tuple[HeroRuntime, ...]:
    return tuple(self._heroes.values())

@property
def enemies(self) -> tuple[Enemy, ...]:
    return tuple(self._enemies)

def start(self) -> None:
    self.state.assert_owner_thread()
    if self.running:
        return
    self._sync_heroes()
    self.running = True
    self._accumulator = 0.0
    self._spawn()
    self.frame_changed.emit(self.frame())

def stop(self) -> None:
    self.state.assert_owner_thread()
    if self.running:
        self.flush()
    self.running = False
    self.phase = "stopped"
    self._impacts.clear()
    self._dots.clear()

def _sync_heroes(self) -> None:
    members = self.runes.squad(self.state.data)
    if not 3 <= len(members) <= 4:
        raise ValueError("Боевой отряд должен содержать 3-4 героев")

    snapshots = {hero.hero_id: hero for hero in self.state.data.party}
    updated: dict[str, HeroRuntime] = {}

```

```

    for index, member in enumerate(members):
        hero_id = member.hero.hero_id
        runtime = self._heroes.get(hero_id)

        if runtime is None:
            snapshot = snapshots[hero_id]
            runtime = HeroRuntime(
                member.hero, member.stats,
                member.stats.max_hp * snapshot.hp / snapshot.max_hp,
                float(member.stats.max_mana),
                *HERO_POSITIONS[index],
            )
        else:
            ratio = runtime.hp / runtime.stats.max_hp
            runtime.hero = member.hero
            runtime.stats = member.stats
            runtime.hp = max(0.0, min(member.stats.max_hp, ratio * member.stats.max_hp))
            runtime.mana = min(runtime.mana, member.stats.max_mana)

        for skill in member.hero.skills:
            runtime.skill_cooldowns.setdefault(skill.skill_id, 0.0)
            updated[hero_id] = runtime

    old_max = self.barrier_max
    self._heroes = updated
    self._effects = self.runes.effects(self.state.data.unlocked_runes)

    self.barrier_max = BARRIER_RATIO * sum(
        hero.stats.max_hp for hero in self._heroes.values()
    )
    self.barrier = (
        self.barrier_max if old_max == 0
        else min(self.barrier_max, self.barrier * self.barrier_max / old_max)
    )

def _spawn(self) -> None:
    self._sync_heroes()
    self._generation += 1
    self._wave = self.state.data.current_wave
    self._enemies = spawn_wave(
        self._wave, self.rng, generation=self._generation
    )

```

```

)
self._wave_total_hp = sum(enemy.max_hp for enemy in self._enemies)

partial = float(Fraction(self.state.data.wave_fraction))
self._continued_wave = partial > 0

for enemy in self._enemies:
    enemy.hp = max(0.001, enemy.max_hp * (1 - partial))

for hero in self._heroes.values():
    hero.pending_skill = None
    hero.cast_left = 0.0

self._dots.clear()
self._impacts.clear()
self._wave_elapsed = 0.0
self._wave_real_elapsed = 0.0
self.phase = "battle"

def advance(self, seconds: float) -> None:
    self.state.assert_owner_thread()
    if not math.isfinite(seconds) or not 0 <= seconds <= 60:
        raise ValueError("advance принимает от 0 до 60 секунд")
    if not self.running:
        return

    self._sync_heroes()
    speed = 1 + self._effects.active_speed_bonus
    self._accumulator += seconds * speed

    while self._accumulator + 1e-12 >= FIXED_STEP:
        self._accumulator = max(0.0, self._accumulator - FIXED_STEP)
        self._step(FIXED_STEP, FIXED_STEP / speed)

    self.frame_changed.emit(self.frame())

@staticmethod
def _animate(actor: HeroRuntime | Enemy, state: str, duration: float) -> None:
    if actor.animation == "die":
        return

    if state == "hurt" and actor.animation in ("attack", "cast"):
        return

```

```

    if actor.animation == state:
        return

    actor.animation = state
    actor.animation_left = duration

def _step(self, dt: float, real_dt: float) -> None:
    self.time += dt

    for actor in (*self._heroes.values(), *self._enemies):
        if actor.animation != "die" and actor.animation_left > 0:
            actor.animation_left = max(0.0, actor.animation_left - dt)
            if actor.animation_left == 0:
                actor.animation = "idle"

    for hero in self._heroes.values():
        hero.basic_timer = max(0.0, hero.basic_timer - dt)
        for skill_id in hero.skill_cooldowns:
            hero.skill_cooldowns[skill_id] = max(
                0.0, hero.skill_cooldowns[skill_id] - dt
            )
        if hero.alive:
            hero.mana = min(
                hero.stats.max_mana,
                hero.mana + (3 + hero.stats.max_mana * 0.01) * dt,
            )
            if self.time >= hero.shield_until:
                hero.shield = 0.0

    if self.phase != "battle":
        self._phase_left -= dt
        if self._phase_left <= 0:
            if self.phase == "defeat":
                for index, hero in enumerate(self._heroes.values()):
                    hero.hp = float(hero.stats.max_hp)
                    hero.mana = float(hero.stats.max_mana)
                    hero.x, hero.y = HERO_POSITIONS[index]
                    hero.invulnerable_until = 0.0
                    hero.animation = "idle"
                    hero.animation_left = 0.0
                    hero.buffs.clear()
                    hero.shield = 0.0
                    hero.summon_left = 0.0

```



```

        self.barrier = self.barrier_max
        self.flush()
    else:
        self.barrier = min(
            self.barrier_max, self.barrier + self.barrier_max * 0.25
        )
        self._spawn()
    return

    self._wave_elapsed += dt
    self._wave_real_elapsed += real_dt

    for hero in self._heroes.values():
        if hero.alive:
            hero.hp = min(
                hero.stats.max_hp, hero.hp + hero.stats.hp_regen * dt
            )

    self._tick_impacts(dt)
    self._tick_dots(dt)
    self._tick_heroes(dt)

    if not any(hero.alive for hero in self._heroes.values()):
        self._defeat("Отряд пал")
        return

    if not any(enemy.alive for enemy in self._enemies):
        self._finish_wave()
        return

    self._tick_enemies(dt)

    if not any(hero.alive for hero in self._heroes.values()):
        self._defeat("Отряд пал")
    elif (
        any(enemy.is_boss and enemy.alive for enemy in self._enemies)
        and self._wave_elapsed >= BOSS_TIME_LIMIT
    ):
        self._defeat("Истекло время боя с боссом")

def _target(self) -> Enemy | None:
    return min(
        (enemy for enemy in self._enemies if enemy.alive),

```

```

        key=lambda enemy: enemy.x,
        default=None,
    )

    @staticmethod
    def basic_interval(hero: HeroRuntime) -> float:
        return BASIC_INTERVALS[hero.hero.hero_class] / max(0.2, hero.stats.attack_speed)

    @staticmethod
    def skill_cooldown(hero: HeroRuntime, skill: Skill) -> float:
        return skill.cooldown_seconds * (
            1 - min(0.75, hero.stats.cooldown_reduction)
        )

    @staticmethod
    def skill_duration(hero: HeroRuntime, skill: Skill) -> float:
        return skill.duration_seconds * (1 + hero.stats.skill_duration)

    @staticmethod
    def skill_power(hero: HeroRuntime, skill: Skill) -> float:
        return skill.power_multiplier * (1 + 0.10 * hero.stats.all_skill_level)

    @staticmethod
    def _distance(hero: HeroRuntime, enemy: Enemy) -> float:
        return math.hypot(hero.x - enemy.x, hero.y - enemy.y)

    def _skill_cost(self, hero: HeroRuntime, skill: Skill) -> float:
        berserk = (
            "berserker" in self._effects.keystones
            and hero.hp < hero.stats.max_hp * 0.40
        )
        return skill.mana_cost * (2 if berserk else 1)

    def _can_cast(self, hero: HeroRuntime, skill: Skill, target: Enemy) -> bool:
        if skill.effect == "heal":
            return any(
                ally.alive and ally.hp < ally.stats.max_hp * 0.9
                for ally in self._heroes.values()
            )
        if skill.effect == "max_hp_shield":
            return any(
                ally.alive and ally.shield <= 0

```

```

        for ally in self._heroes.values():
    )
    if skill.target in ("self", "party", "lowest_hp_ally"):
        return True

    reach = (hero.stats.basic_attack_range + 80) * (1 + hero.stats.skill_range)
    return self._distance(hero, target) <= reach

def _tick_heroes(self, dt: float) -> None:
    for index, hero in enumerate(tuple(self._heroes.values())):
        if not hero.alive:
            hero.pending_skill = None
            continue

        if hero.pending_skill is not None:
            hero.cast_left = max(0.0, hero.cast_left - dt)
            if hero.cast_left <= 1e-9:
                skill = hero.pending_skill
                hero.pending_skill = None
                self._cast(hero, skill)
            continue

        target = self._target()
        if target is None:
            break

        if hero.summon_left > 0:
            hero.summon_left = max(0.0, hero.summon_left - dt)
            hero.summon_timer -= dt
            if hero.summon_timer <= 1e-9:
                hero.summon_timer += 1.0
                self._hit_enemy(
                    hero, target, hero.summon_power, True,
                    element="chaos", tags=frozenset({"summon"}),
                )

        target = self._target()
        if target is None:
            break

        if hero.animation in ("attack", "cast", "hurt"):
            continue

```

```

cast = False
for skill in hero.hero.skills:
    cost = self._skill_cost(hero, skill)
    if (
        hero.skill_cooldowns[skill.skill_id] <= 1e-9
        and hero.mana >= cost
        and self._can_cast(hero, skill, target)
    ):
        hero.mana -= cost
        hero.skill_cooldowns[skill.skill_id] = self.skill_cooldown(hero, skill)
        hero.pending_skill = skill
        hero.cast_left = 0.45 / hero.stats.cast_speed
        self._animate(hero, "cast", hero.cast_left)
        self.skill_used.emit(hero.hero.hero_id, skill.skill_id)
        cast = True
        break

if cast:
    continue

reach = hero.stats.basic_attack_range
if self._distance(hero, target) > reach:
    destination = min(195.0, max(HERO_POSITIONS[index][0], target.x - reach + 2))
    movement = hero.stats.movement_speed * 0.06 * dt
    hero.x += max(-movement, min(movement, destination - hero.x))
    if hero.animation in ("idle", "run"):
        hero.animation = "run"
    continue

if hero.animation == "run":
    hero.animation = "idle"

cost = max(0.0, BASE_ATTACK_COST - hero.stats.basic_requirement_reduction)
if hero.basic_timer <= 1e-9 and hero.mana >= cost:
    hero.mana -= cost
    hero.basic_timer = self.basic_interval(hero)
    self._animate(hero, "attack", min(0.35, hero.basic_timer * 0.8))

element = CLASS_ELEMENT[hero.hero.hero_class]
ranged = hero.hero.hero_class in (
    HeroClass.PYROMANCER, HeroClass.RANGER, HeroClass.NECROMANCER,

```

```

    )
    power = (
        hero.stats.physical_damage
        if element == "physical" else hero.stats.magical_damage
    )
    tags = frozenset({"projectile" if ranged else "melee"})
    self._dispatch(hero, target, power, element, tags)

def _repeat_count(self, enhancement: float) -> int:
    whole = math.floor(enhancement)
    return 1 + whole + int(self.rng.random() < enhancement - whole)

def _dispatch(
    self,
    hero: HeroRuntime,
    target: Enemy,
    power: float,
    element: str,
    tags: frozenset[str],
    *,
    drain: float = 0.0,
    repeat: bool = True,
) -> None:
    repetitions = self._repeat_count(hero.stats.multistrike) if repeat else 1
    projectiles = 1 + hero.stats.projectile_count if "projectile" in tags else 1

    travel = 0.0
    if "projectile" in tags:
        speed = BASE_PROJECTILE_SPEED * (1 + hero.stats.projectile_speed)
        travel = self._distance(hero, target) / speed

    for repetition in range(repetitions):
        for projectile in range(projectiles):
            self._impacts.append(PendingImpact(
                hero.hero_id,
                target.enemy_id,
                power,
                element,
                tags,
                travel + repetition * 0.10 + projectile * 0.025,
                drain,
            ))

```

```

def _tick_impacts(self, dt: float) -> None:
    enemies = {enemy.enemy_id: enemy for enemy in self._enemies}
    pending: list[PendingImpact] = []

    for impact in self._impacts:
        impact.remaining -= dt
        owner = self._heroes.get(impact.owner_id)
        target = enemies.get(impact.enemy_id)

        if owner is None or target is None or not target.alive:
            continue
        if impact.remaining > 1e-9:
            pending.append(impact)
            continue

        # Уже выпущенный снаряд может попасть после смерти владельца.
        if not owner.alive and "projectile" not in impact.tags:
            continue

        dealt = self._hit_enemy(
            owner, target, impact.power,
            impact.element != "physical",
            element=impact.element,
            tags=impact.tags,
        )
        if impact.drain > 0:
            self._heal(owner, dealt * impact.drain * (1 + owner.stats.skill_heal))

    self._impacts = pending

def _area_targets(self, hero: HeroRuntime, target: Enemy) -> list[Enemy]:
    radius = 80 * (1 + hero.stats.aoe_enhancement)
    return [
        enemy for enemy in self._enemies
        if enemy.alive
        and math.hypot(enemy.x - target.x, enemy.y - target.y) <= radius
    ]

def _cast(self, hero: HeroRuntime, skill: Skill) -> bool:
    if not hero.alive:
        return False

```

```

effect = skill.effect
magnitude = self.skill_power(hero, skill)
duration = self.skill_duration(hero, skill)

if effect == "heal":
    allies = [ally for ally in self._heroes.values() if ally.alive]
    if not allies:
        return False
    target_hero = min(allies, key=lambda ally: ally.hp / ally.stats.max_hp)
    self._heal(
        target_hero,
        hero.stats.magical_damage * magnitude * (1 + hero.stats.skill_heal),
    )
    return True

if effect in ("damage_reduction", "evasion_buff"):
    key = "reduction" if effect == "damage_reduction" else "evasion"
    hero.buffs[key] = (self.time + duration, magnitude)
    return True

if effect == "party_damage_buff":
    for ally in self._heroes.values():
        ally.buffs["damage"] = (self.time + duration, magnitude)
    return True

if effect == "max_hp_shield":
    for ally in self._heroes.values():
        if ally.alive:
            ally.shield = max(ally.shield, ally.stats.max_hp * magnitude)
            ally.shield_until = self.time + duration
    return True

if effect == "summon":
    hero.summon_left = duration
    hero.summon_timer = 0.0
    hero.summon_power = hero.stats.magical_damage * magnitude
    return True

target = self._target()
if target is None:
    return False

```

```

    if effect == "damage_taken_debuff":
        target.marked_until = self.time + duration
        target.marked_bonus = magnitude
        return True

    area = skill.target == "all_enemies"
    targets = self._area_targets(hero, target) if area else [target]

    element = CLASS_ELEMENT[hero.hero.hero_class]
    if skill.skill_id == "paladin_smite":
        element = "lightning"
    elif effect == "poison_per_second":
        element = "chaos"
    elif effect == "burn_per_second":
        element = "fire"

    ranged = (
        hero.hero.hero_class in (
            HeroClass.PYROMANCER, HeroClass.RANGER, HeroClass.NECROMANCER,
        )
        and effect not in ("burn_per_second", "poison_per_second")
    )
    tags = set()
    if area:
        tags.add("aoe")
    if ranged:
        tags.add("projectile")
    elif element == "physical":
        tags.add("melee")

    base = (
        hero.stats.physical_damage
        if element == "physical" else hero.stats.magical_damage
    )
    power = base * magnitude

    if effect in ("poison_per_second", "burn_per_second"):
        for enemy in targets:
            self._dots.append(DamageOverTime(
                hero.hero.hero_id, enemy.enemy_id,
                power, duration, True,

```



```

        element=element, tags=frozenset(tags),
    ))
    return True

    if effect not in ("damage", "life_drain"):
        raise ValueError(f"Неизвестный эффект навыка: {effect}")

    for enemy in targets:
        self._dispatch(
            hero, enemy, power, element, frozenset(tags),
            drain=0.35 if effect == "life_drain" else 0.0,
            repeat=False,
        )
    return True

def _tick_dots(self, dt: float) -> None:
    enemies = {enemy.enemy_id: enemy for enemy in self._enemies}
    remaining: list[DamageOverTime] = []

    for dot in self._dots:
        owner = self._heroes.get(dot.owner_id)
        enemy = enemies.get(dot.enemy_id)
        if owner is None or enemy is None or not enemy.alive:
            continue

        # Пропорциональная последняя доля секунды не теряется.
        portion = min(dt, dot.remaining)
        dot.remaining -= portion

        self._hit_enemy(
            owner, enemy, dot.power * portion, True,
            element=dot.element,
            tags=dot.tags,
            allow_crit=False,
            allow_splash=False,
            on_hit=False,
            show_event=False,
        )

    if dot.remaining > 1e-9 and enemy.alive:
        remaining.append(dot)

```

```

self._dots = remaining
def _hit_enemy(
    self,
    hero: HeroRuntime,
    enemy: Enemy,
    power: float,
    magical: bool,
    *,
    element: str | None = None,
    tags: frozenset[str] = frozenset(),
    allow_crit: bool = True,
    allow_splash: bool = True,
    on_hit: bool = True,
    show_event: bool = True,
) -> float:
    if not enemy.alive:
        return 0.0

    element = element or ("magic" if magical else "physical")
    if element not in ELEMENTS:
        raise ValueError("Неизвестный тип урона")

    s = hero.stats
    adjusted = power * max(0.0, 1 + s.elemental_bonus(element))

    for tag, bonus in (
        ("melee", s.melee_damage),
        ("projectile", s.projectile_damage),
        ("aoe", s.aoe_damage),
        ("summon", s.summon_damage),
    ):
        if tag in tags:
            adjusted *= max(0.0, 1 + bonus)

    adjusted *= 1 + hero.buff("damage", self.time)
    if (
        "berserker" in self._effects.keystones
        and hero.hp < s.max_hp * 0.40
    ):
        adjusted *= 1.8

```

```

    if enemy.marked_until > self.time:
        adjusted *= 1 + enemy.marked_bonus

    crit_multiplier = s.crit_multiplier
    if element != "physical" and "supernova" in self._effects.keystones:
        crit_multiplier *= 1.5

    hit = roll_damage(
        adjusted, enemy.armor,
        magical=element != "physical",
        crit_chance=s.crit_chance if allow_crit else 0.0,
        crit_multiplier=crit_multiplier,
        rng=self.rng,
    )

    dealt = min(enemy.hp, hit.damage)
    enemy.hp = max(0.0, enemy.hp - dealt)

    if show_event and dealt > 0:
        self.damage_event.emit(DamageEvent(
            enemy.enemy_id, enemy.x, enemy.y - 32,
            dealt, "physical" if element == "physical" else "magical",
            hit.critical,
        ))

    if enemy.is_boss:
        enemy.rage = min(
            enemy.rage_max,
            enemy.rage + dealt / enemy.max_hp * 30,
        )

    killed = not enemy.alive
    if killed:
        enemy.animation = "die"
        enemy.animation_left = 0.0
    elif on_hit:
        self._animate(enemy, "hurt", 0.20)

    if dealt > 0:
        healing = dealt * s.lifesteal
        if on_hit:
            healing += s.hp_per_hit

```

```

        if killed:
            healing += s.hp_per_kill
            self._heal(hero, healing, show=show_event)

        if (
            on_hit and dealt > 0 and hero.alive
            and EnemyAffix.REFLECTIVE in enemy.affixes
        ):
            self._hit_hero(hero, dealt * 0.12, True, element="chaos")

        if (
            allow_splash and on_hit and "melee" in tags
            and "titan_sweep" in self._effects.keystones
            and hero.alive
        ):
            radius = 80 * (1 + s.aoe_enhancement)
            neighbor = next(
                (
                    entry for entry in self._enemies
                    if entry.alive and entry is not enemy
                    and abs(entry.x - enemy.x) <= radius
                ),
                None,
            )
            if neighbor:
                self._hit_enemy(
                    hero, neighbor, power * 0.40, False,
                    element="physical",
                    tags=frozenset({"melee", "aoe"}),
                    allow_crit=False, allow_splash=False,
                )

    return dealt

def _hit_hero(
    self,
    hero: HeroRuntime,
    power: float,
    magical: bool,
    *,
    element: str | None = None,
) -> float:

```

```
if not hero.alive or self.time < hero.invulnerable_until:
```

```
    return 0.0
```

```
if not math.isfinite(power) or power < 0:
```

```
    raise ValueError("Некорректный входящий урон")
```

```
element = element or ("magic" if magical else "physical")
```

```
if element not in ELEMENTS:
```

```
    raise ValueError("Неизвестный тип урона")
```

```
elemental = element != "physical"
```

```
s = hero.stats
```

```
dodge = min(0.95, s.evasion + hero.buff("evasion", self.time))
```

```
block = s.block_chance
```

```
if elemental:
```

```
    dodge = combined_chance(dodge, s.elemental_dodge)
```

```
    block = combined_chance(block, s.elemental_block)
```

```
hero.invulnerable_until = self.time + HERO_IFRAMES
```

```
if self.rng.random() < dodge:
```

```
    self.damage_event.emit(DamageEvent(
```

```
        hero.hero.hero_id, hero.x, hero.y - 32, 0, "dodge",
```

```
    ))
```

```
    return 0.0
```

```
blocked = self.rng.random() < block
```

```
damage = power * (0.40 if blocked else 1.0)
```

```
if blocked:
```

```
    self.damage_event.emit(DamageEvent(
```

```
        hero.hero.hero_id, hero.x, hero.y - 39,
```

```
        power * 0.60, "block",
```

```
    ))
```

```
if elemental:
```

```
    damage *= 1 - s.resistance(element)
```

```
else:
```

```
    damage *= 100 / (100 + s.armor)
```

```
damage *= 1 - min(0.90, hero.buff("reduction", self.time))
```

```
damage = max(0.0, damage - s.damage_absorption)
```

```

    personal = min(hero.shield, damage)
    hero.shield -= personal
    damage -= personal

    shared = min(self.barrier, damage)
    self.barrier -= shared
    damage -= shared

    if "ether_shield" in self._effects.keystones:
        mana_absorb = min(hero.mana, damage * 0.30)
        hero.mana -= mana_absorb
        damage -= mana_absorb

    dealt = min(hero.hp, damage)
    hero.hp = max(0.0, hero.hp - dealt)

    self.damage_event.emit(DamageEvent(
        hero.hero_id, hero.x, hero.y - 32,
        dealt,
        ("magical" if elemental else "physical") if dealt > 0 else "barrier",
    ))

    if not hero.alive:
        hero.animation = "die"
        hero.animation_left = 0.0
        hero.pending_skill = None
    elif dealt > 0:
        self._animate(hero, "hurt", 0.20)

    if (
        hero.alive
        and "second_wind" in self._effects.keystones
        and hero.hp < s.max_hp * 0.20
        and self.time >= hero.second_wind_ready
    ):
        hero.second_wind_ready = self.time + 60
        self._heal(hero, s.max_hp * 0.30)

    return dealt

def _heal(self, hero: HeroRuntime, amount: float, *, show: bool = True) -> None:
    if not hero.alive or amount <= 0:

```

```

        return
    restored = min(hero.stats.max_hp - hero.hp, amount)
    if restored <= 0:
        return
    hero.hp += restored

    if show:
        self.damage_event.emit(DamageEvent(
            hero.hero_id, hero.x, hero.y - 32,
            restored, "heal",
        ))

def _tick_enemies(self, dt: float) -> None:
    for index, enemy in enumerate(self._enemies):
        if not enemy.alive:
            enemy.corpse_left -= dt
            continue

        if enemy.is_boss:
            enemy.rage = min(
                enemy.rage_max,
                enemy.rage + dt * enemy.rage_max / 30,
            )

        enemy.attack_timer = max(0.0, enemy.attack_timer - dt)
        destination = 225 + (index % 3) * 20

        if enemy.x > destination:
            enemy.x = max(destination, enemy.x - enemy.speed * dt)
            if enemy.animation in ("idle", "run"):
                enemy.animation = "run"
            continue

        if enemy.animation == "run":
            enemy.animation = "idle"

    allies = [hero for hero in self._heroes.values() if hero.alive]
    if not allies:
        return

    if enemy.attack_timer <= 1e-9:
        target = self.rng.choice(allies) if enemy.magical else allies[0]

```

```

        self._animate(enemy, "attack", 0.35)
        enemy.attack_timer = enemy.effective_interval

    element = (
        ENEMY_ELEMENT.get(enemy.template_id, "chaos")
        if enemy.magical else "physical"
    )

    dealt = self._hit_hero(
        target, enemy.effective_damage, enemy.magical,
        element=element,
    )

    if EnemyAffix.VAMPIRIC in enemy.affixes and dealt > 0:
        restored = min(enemy.max_hp - enemy.hp, dealt * 0.25)
        enemy.hp += restored
        if restored > 0:
            self.damage_event.emit(DamageEvent(
                enemy.enemy_id, enemy.x, enemy.y - 32,
                restored, "heal",
            ))

def _party_snapshots(self):
    result = []
    for snapshot in self.state.data.party:
        runtime = self._heroes.get(snapshot.hero_id)
        if runtime is None:
            result.append(snapshot)
            continue
        hp = (
            0 if not runtime.alive
            else max(1, min(
                snapshot.max_hp,
                math.ceil(runtime.hp / runtime.stats.max_hp * snapshot.max_hp),
            ))
        )
        result.append(replace(snapshot, hp=hp))
    return tuple(result)

def _progress(self) -> float:
    if self.phase == "between":
        return 1.0
    if self.phase != "battle":

```



```

        return 0.0
    return max(0.0, min(
        1.0,
        1 - sum(enemy.hp for enemy in self._enemies) / self._wave_total_hp,
    ))

def flush(self) -> None:
    self.state.assert_owner_thread()
    if not self._heroes:
        return
    before = self.state.data
    progress = self._progress() if self.phase == "battle" else 0.0

    self.state.commit(before, replace(
        before,
        party=self._party_snapshots(),
        wave_fraction=str(Fraction(str(min(progress, 0.999999999)))),
    ))

def _finish_wave(self) -> None:
    self.phase = "between"
    self._phase_left = BETWEEN_WAVES
    self._dots.clear()
    self._impacts.clear()

    before = self.state.data
    profile = before.farm

    gold = math.floor(
        profile.gold_per_wave * (1 + Fraction(str(profile.gold_bonus)))
    )

    if self._wave % 10 == 0 and "gold_fever" in self._effects.keystones:
        gold *= 4

    secondary = [
        self._heroes[snapshot.hero_id].stats
        for snapshot in before.party
    ]

    gains, cursor = experience_awards(
        profile.exp_per_wave, 1, secondary, before.exp_cursor,
    )

    party = tuple(

```

```

        replace(snapshot, total_exp=snapshot.total_exp + gains[index])
    for index, snapshot in enumerate(self._party_snapshots())
    )
    stash = list(before.stash)
    if self.rng.random() < profile.drop_chance:
        rarity = self.rng.choices(
            [entry[0] for entry in C.LOOT_TABLE],
            weights=[entry[1] for entry in C.LOOT_TABLE],
            k=1,
        )[0]
        price = {entry[0]: entry[2] for entry in C.LOOT_TABLE}[rarity]
        if len(stash) >= before.stash_capacity:
            gold += math.floor(
                price * (1 + Fraction(str(profile.sale_bonus)))
            )
        else:
            item = generate_item(
                1 + (self._wave - 1) // 100,
                rarity, source_wave=self._wave, rng=self.rng,
            )
            known = {
                entry.item_id
                for entry in (*before.stash, *before.backpack)
            }
            known.update(
                entry.item_id
                for build in before.hero_builds
                for entry in build.equipment
            )
            while item.item_id in known:
                item = replace(
                    item, item_id=f"item-{self.rng.getrandbits(128):032x}"
                )
            stash.append(replace(item, sell_value=price))
    farm = profile
    if not self._continued_wave:
        measured = max(C.MIN_WAVE_CLEAR_SECONDS, self._wave_real_elapsed)
        farm = replace(
            profile,

```

```

        average_wave_clear_seconds=max(
            C.MIN_WAVE_CLEAR_SECONDS,
            profile.average_wave_clear_seconds * 0.85 + measured * 0.15,
        ),
    )

    self.state.commit(before, replace(
        before,
        gold=before.gold + gold,
        party=party,
        stash=tuple(stash),
        cleared_waves=before.cleared_waves + 1,
        wave_fraction="0",
        exp_cursor=cursor,
        farm=farm,
    ))

    self.wave_finished.emit(self._wave)

def _defeat(self, reason: str) -> None:
    self.phase = "defeat"
    self._phase_left = DEFEAT_DELAY
    self._dots.clear()
    self._impacts.clear()
    for hero in self._heroes.values():
        hero.pending_skill = None

    before = self.state.data
    self.state.commit(before, replace(
        before,
        party=self._party_snapshots(),
        wave_fraction="0",
    ))
    self.party_defeated.emit(reason)

def frame(self) -> CombatFrame:
    actors = [
        ActorFrame(
            hero.hero_id, hero.hero_class.value, hero.hero.name,
            hero.x, hero.y, hero.hp, hero.stats.max_hp,
            hero.animation, False,
            mana=hero.mana, max_mana=hero.stats.max_mana,
        )
    ]

```

```

        for hero in self._heroes.values():
    ]

    for enemy in self._enemies:
        if not enemy.alive and enemy.corpse_left <= 0:
            continue
        actors.append(ActorFrame(
            enemy.enemy_id, enemy.template_id, enemy.name,
            enemy.x, enemy.y, enemy.hp, enemy.max_hp,
            enemy.animation, True,
            elite=bool(enemy.affixes),
            boss=enemy.is_boss,
            rage=enemy.rage / enemy.rage_max,
        ))

    boss = any(enemy.is_boss for enemy in self._enemies)
    return CombatFrame(
        self._wave, act_for_wave(self._wave), self.phase, self.time,
        self._progress(), self.barrier, self.barrier_max,
        max(0.0, BOSS_TIME_LIMIT - self._wave_elapsed),
        if boss and self.phase == "battle" else None,
        tuple(actors),
    )

```

Файл: engine/combat_manager.py

```

`python
from future import annotations

import math
import random
import time

from dataclasses import dataclass, replace
from fractions import Fraction

from PySide6.QtCore import QObject, Signal

from engine import combat_core as core
from engine.combat_core import (

```

```

ActorFrame,
DamageEvent,
HeroRuntime,
Hit,
PendingImpact,
combined_chance,
roll_damage,
)
from engine.save_manager import SaveError, SaveManager
from engine.state import GameState
from models.enemy import Enemy
from models.hero import HeroClass
from models.item import Item, roll_enemy_loot

```

Совместимые публичные константы.

```

FIXED_STEP = core.FIXED_STEP
HERO_IFRAMES = core.HERO_IFRAMES
BASIC_INTERVALS = core.BASIC_INTERVALS
BARRIER_RATIO = core.BARRIER_RATIO
BETWEEN_WAVES = core.BETWEEN_WAVES
DEFEAT_DELAY = core.DEFEAT_DELAY
BASE_ATTACK_COST = core.BASE_ATTACK_COST
BASE_PROJECTILE_SPEED = core.BASE_PROJECTILE_SPEED

HERO_POSITIONS = (
(155.0, 112.0),
(135.0, 95.0),
(105.0, 115.0),
(75.0, 100.0),
)

CLASS_POSITIONS = {
HeroClass.KNIGHT: (155.0, 112.0),
HeroClass.PALADIN: (155.0, 112.0),
HeroClass.ASSASSIN: (135.0, 95.0),
}

```

```
HeroClass.RANGER: (105.0, 115.0),
HeroClass.PYROMANCER: (75.0, 100.0),
HeroClass.NECROMANCER: (75.0, 100.0),
}
```

```
@dataclass(frozen=True)
```

```
class ProjectileFrame:
```

```
    projectile_id: int
```

```
    owner_id: str
```

```
    target_id: str
```

```
    kind: str
```

```
    x: float
```

```
    y: float
```

```
    target_x: float
```

```
    target_y: float
```

```
    progress: float
```

```
    angle: float
```

```
@dataclass(frozen=True)
```

```
class CombatFrame(core.CombatFrame):
```

```
    projectiles: tuple[ProjectileFrame, ...] = ()
```

```
@dataclass(frozen=True)
```

```
class VfxEvent:
```

```
    kind: str
```

```
    x: float
```

```
    y: float
```

```
    element: str = "physical"
```

```
    critical: bool = False
```

```
@dataclass(frozen=True)
```

```
class LootEvent:
```

```
    item: Item
```

```
    x: float
```

```
    y: float
```

```
    stored: bool
```

```
    sale_gold: int = 0
```

```

@dataclass
class VisualImpact(PendingImpact):
    visual_id: int = 0
    visual_kind: str = ""
    start_x: float = 0.0
    start_y: float = 0.0
    launch_time: float = 0.0
    travel_time: float = 0.0

@dataclass(frozen=True)
class PendingLoot:
    key: str
    wave: int
    x: float
    y: float
    items: tuple[Item, ...]

class CombatManager(core.CombatManager):
    vfx_event = Signal(object)
    loot_event = Signal(object)
    combat_error = Signal(str)

```

```

def __init__(
    self,
    state: GameState,
    *,
    rng: random.Random | None = None,
    saves: SaveManager | None = None,
    parent: QObject | None = None,
) -> None:
    super().__init__(state, rng=rng, parent=parent)
    self.saves = saves

    self._visual_serial = 0
    self._pending_loot: list[PendingLoot] = []
    self._formation_signature = None
    self._home_positions: dict[str, tuple[float, float]] = {}
    self._last_save_error = ""

def _sync_heroes(self) -> None:

```

```

previous_ids = set(self._heroes)
super()._sync_heroes()

signature = tuple(
    (hero.hero.hero_id, hero.hero.hero_class)
    for hero in self._heroes.values()
)
if signature == self._formation_signature:
    return

self._formation_signature = signature
self._home_positions.clear()

occupied: dict[tuple[float, float], int] = {}
for hero in self._heroes.values():
    base = CLASS_POSITIONS[hero.hero.hero_class]
    duplicate = occupied.get(base, 0)
    occupied[base] = duplicate + 1

    # Два танка или два мага не занимают один пиксель.
    x = base[0] - duplicate * 25
    y = base[1] - duplicate * 20
    home = (max(38.0, x), max(69.0, y))

    self._home_positions[hero.hero.hero_id] = home
    if hero.hero.hero_id not in previous_ids:
        hero.x, hero.y = home
    else:
        hero.x, hero.y = home

def _spawn(self) -> None:
    super()._spawn()
    for hero in self._heroes.values():
        hero.x, hero.y = self._home_positions[hero.hero.hero_id]

def _tick_heroes(self, dt: float) -> None:
    """
    Сохраняет правила core: AS, CDR, стоимость атак, касты,
    призыв и дальность. Меняет только привязку движения к построению.
    """
    for hero in tuple(self._heroes.values()):
        if not hero.alive:

```



```

        hero.pending_skill = None
        continue

    if hero.pending_skill is not None:
        hero.cast_left = max(0.0, hero.cast_left - dt)
        if hero.cast_left <= 1e-9:
            skill = hero.pending_skill
            hero.pending_skill = None
            self._cast(hero, skill)
            continue

    target = self._target()
    if target is None:
        break

    if hero.summon_left > 0:
        hero.summon_left = max(0.0, hero.summon_left - dt)
        hero.summon_timer -= dt
        if hero.summon_timer <= 1e-9:
            hero.summon_timer += 1.0
            self._hit_enemy(
                hero, target, hero.summon_power, True,
                element="chaos",
                tags=frozenset({"summon"}),
            )

    target = self._target()
    if target is None:
        break

    if hero.animation in ("attack", "cast", "hurt"):
        continue

    cast = False
    for skill in hero.hero.skills:
        cost = self._skill_cost(hero, skill)
        if (
            hero.skill_cooldowns[skill.skill_id] <= 1e-9
            and hero.mana >= cost
            and self._can_cast(hero, skill, target)
        ):
            hero.mana -= cost

```

```

        hero.skill_cooldowns[skill.skill_id] = self.skill_cooldown(
            hero, skill
        )
        hero.pending_skill = skill
        hero.cast_left = 0.45 / hero.stats.cast_speed
        self._animate(hero, "cast", hero.cast_left)
        self.skill_used.emit(hero.hero_id, skill.skill_id)
        cast = True
        break

    if cast:
        continue

    home_x, _ = self._home_positions[hero.hero_id]
    reach = hero.stats.basic_attack_range

    if self._distance(hero, target) > reach:
        destination = min(
            195.0,
            max(home_x, target.x - reach + 2),
        )
        movement = hero.stats.movement_speed * 0.06 * dt
        hero.x += max(
            -movement,
            min(movement, destination - hero.x),
        )
        if hero.animation in ("idle", "run"):
            hero.animation = "run"
            continue

    if hero.animation == "run":
        hero.animation = "idle"

    cost = max(
        0.0,
        BASE_ATTACK_COST - hero.stats.basic_requirement_reduction,
    )

    if hero.basic_timer <= 1e-9 and hero.mana >= cost:
        hero.mana -= cost
        hero.basic_timer = self.basic_interval(hero)
        self._animate(

```

```

        hero, "attack",
        min(0.35, hero.basic_timer * 0.8),
    )

    element = core.CLASS_ELEMENT[hero.hero.hero_class]
    ranged = hero.hero.hero_class in (
        HeroClass.PYROMANCER,
        HeroClass.RANGER,
        HeroClass.NECROMANCER,
    )
    power = (
        hero.stats.physical_damage
        if element == "physical"
        else hero.stats.magical_damage
    )
    self._dispatch(
        hero,
        target,
        power,
        element,
        frozenset({"projectile" if ranged else "melee"}),
    )

def _dispatch(
    self,
    hero: HeroRuntime,
    target: Enemy,
    power: float,
    element: str,
    tags: frozenset[str],
    *,
    drain: float = 0.0,
    repeat: bool = True,
) -> None:
    repetitions = (
        self._repeat_count(hero.stats.multistrike)
        if repeat else 1
    )
    projectiles = (
        1 + hero.stats.projectile_count
        if "projectile" in tags else 1
    )

```

```

        ranged = "projectile" in tags
        travel = 0.0

    if ranged:
        speed = BASE_PROJECTILE_SPEED * (
            1 + hero.stats.projectile_speed
        )
        travel = self._distance(hero, target) / speed

    kind = {
        HeroClass.RANGER: "arrow",
        HeroClass.PYROMANCER: "fireball",
        HeroClass.NECROMANCER: "necrotic",
    }.get(hero.hero.hero_class, "arcane")

    for repetition in range(repetitions):
        for projectile in range(projectiles):
            delay = repetition * 0.10 + projectile * 0.025
            self._visual_serial += 1

            self._impacts.append(VisualImpact(
                owner_id=hero.hero.hero_id,
                enemy_id=target.enemy_id,
                power=power,
                element=element,
                tags=tags,
                remaining=travel + delay,
                drain=drain,
                visual_id=self._visual_serial,
                visual_kind=kind if ranged else "",
                start_x=hero.x + 9,
                start_y=hero.y - 23,
                launch_time=self.time + delay,
                travel_time=max(travel, 0.001),
            ))

def _hit_enemy(
    self,
    hero: HeroRuntime,
    enemy: Enemy,
    power: float,

```

```

magical: bool,
*,
element: str | None = None,
tags: frozenset[str] = frozenset(),
allow_crit: bool = True,
allow_splash: bool = True,
on_hit: bool = True,
show_event: bool = True,
) -> float:
    was_alive = enemy.alive
    if not was_alive:
        return 0.0

    dealt = super()._hit_enemy(
        hero, enemy, power, magical,
        element=element,
        tags=tags,
        allow_crit=allow_crit,
        allow_splash=allow_splash,
        on_hit=on_hit,
        show_event=show_event,
    )

    resolved_element = element or (
        "magic" if magical else "physical"
    )

    if dealt > 0 and on_hit:
        if (
            hero.hero.hero_class == HeroClass.PALADIN
            and resolved_element == "lightning"
        ):
            self.vfx_event.emit(VfxEvent(
                "holy", enemy.x, enemy.y - 18, "lightning"
            ))
        elif "melee" in tags:
            self.vfx_event.emit(VfxEvent(
                "slash", enemy.x, enemy.y - 20, resolved_element
            ))

        self.vfx_event.emit(VfxEvent(
            "impact", enemy.x, enemy.y - 20, resolved_element

```

```

    ))
    if was_alive and not enemy.alive:
        self._queue_death_loot(enemy)
    return dealt

def _hit_hero(
    self,
    hero: HeroRuntime,
    power: float,
    magical: bool,
    *,
    element: str | None = None,
) -> float:
    barrier_before = self.barrier
    result = super()._hit_hero(
        hero, power, magical, element=element
    )
    if self.barrier < barrier_before:
        self.vfx_event.emit(VfxEvent(
            "barrier", hero.x, hero.y - 18, "magic"
        ))
    return result

def _queue_death_loot(self, enemy: Enemy) -> None:
    index = enemy.enemy_id.rsplit("-", 1)[-1]
    key = f"{self._wave}:{index}"
    if key in self.state.data.reward_claims:
        return
    if any(record.key == key for record in self._pending_loot):
        return
    items = roll_enemy_loot(
        self._wave,
        enemy.rank.value,
        self.rng,
    )
    # Пустой ролл тоже фиксируется: перезапуск не даёт новый шанс.

```

```

self._pending_loot.append(PendingLoot(
    key=key,
    wave=self._wave,
    x=enemy.x,
    y=enemy.y,
    items=items,
))

def _settle_loot(self) -> None:
    if not self._pending_loot:
        return

    before = self.state.data
    claims = set(before.reward_claims)
    stash = list(before.stash)
    gold = before.gold
    events: list[LootEvent] = []

    occupied_ids = {
        item.item_id
        for item in (*before.stash, *before.backpack)
    }
    occupied_ids.update(
        item.item_id
        for build in before.hero_builds
        for item in build.equipment
    )

    for record in self._pending_loot:
        if record.key in claims:
            continue

        claims.add(record.key)

        for original_item in record.items:
            item = original_item
            while item.item_id in occupied_ids:
                item = replace(
                    item,
                    item_id=f"item-{self.rng.getrandbits(128):032x}",
                )
            occupied_ids.add(item.item_id)

```

```

        stored = len(stash) < before.stash_capacity
        sale = 0

        if stored:
            stash.append(item)
        else:
            sale = math.floor(
                item.sell_value
                * (1 + Fraction(str(before.farm.sale_bonus)))
            )
            gold += sale

        events.append(LootEvent(
            item, record.x, record.y, stored, sale
        ))

        # Нужны текущая и предыдущая волны; история не растёт бесконечно.
        oldest = max(1, before.current_wave - 1)
        claims = {
            key for key in claims
            if int(key.split(":")[0]) >= oldest
        }

        progress = self._progress() if self.phase == "battle" else 0.0

        after = replace(
            before,
            gold=gold,
            stash=tuple(stash),
            party=self._party_snapshots(),
            wave_fraction=str(Fraction(str(min(progress, 0.999999999)))),
            reward_claims=tuple(sorted(claims)),
        )

        if self.saves is not None:
            after = replace(
                after,
                last_active_timestamp=max(
                    time.time(), after.last_active_timestamp
                ),
            )

```



```

        self.saves.write(after)

    self.state.commit(before, after)
    self._pending_loot.clear()
    self._last_save_error = ""

    for event in events:
        self.loot_event.emit(event)

def _step(self, dt: float, real_dt: float) -> None:
    try:
        # При отказе диска сначала завершаем прошлую выдачу.
        self._settle_loot()
    except SaveError as exc:
        self._report_save_error(exc)
    return

    super()._step(dt, real_dt)

    try:
        self._settle_loot()
    except SaveError as exc:
        self._report_save_error(exc)

def _report_save_error(self, exc: Exception) -> None:
    message = str(exc)
    if message != self._last_save_error:
        self._last_save_error = message
        self.combat_error.emit(
            "Бой приостановлен: не удалось сохранить лут. " + message
        )

def flush(self) -> None:
    self._settle_loot()
    super().flush()

def frame(self) -> CombatFrame:
    base = super().frame()
    enemies = {enemy.enemy_id: enemy for enemy in self._enemies}
    projectiles = []

    for impact in self._impacts:

```

```

        if not isinstance(impact, VisualImpact):
            continue
        if not impact.visual_kind:
            continue
        if self.time < impact.launch_time:
            continue

        target = enemies.get(impact.enemy_id)
        if target is None or not target.alive:
            continue

        progress = max(
            0.0,
            min(1.0, 1 - impact.remaining / impact.travel_time),
        )
        tx, ty = target.x, target.y - 20
        x = impact.start_x + (tx - impact.start_x) * progress
        y = impact.start_y + (ty - impact.start_y) * progress
        angle = math.atan2(ty - impact.start_y, tx - impact.start_x)

        projectiles.append(ProjectileFrame(
            impact.visual_id,
            impact.owner_id,
            impact.enemy_id,
            impact.visual_kind,
            x, y, tx, ty, progress, angle,
        ))

    return CombatFrame(
        wave=base.wave,
        act=base.act,
        phase=base.phase,
        time=base.time,
        progress=base.progress,
        barrier=base.barrier,
        barrier_max=base.barrier_max,
        boss_remaining=base.boss_remaining,
        actors=base.actors,
        projectiles=tuple(projectiles),
    )

```

Файл: engine/cube_synth.py

```
`python
from future import annotations

import random

from dataclasses import dataclass, replace
from types import MappingProxyType
from typing import Mapping

from engine.runes_tree import RunesTree
from engine.save_manager import SaveManager
from engine.state import GameData, GameState
from models.item import Item, Rarity, RARITY_ORDER, generate_item

MIN_INGREDIENTS = 4
MAX_INGREDIENTS = 12
MAX_LEVEL_SPREAD = 15

BASE_CRITICAL_CHANCE = 0.024

class SynthesisError(ValueError):
    pass

@dataclass(frozen=True)
class SynthesisResult:
    before: GameData
    after: GameData
    consumed_ids: tuple[str, ...]
    item: Item
    probabilities: Mapping[Rarity, float]

class CubeSynth:
    def __init__(self, runes: RunesTree | None = None) -> None:
        self.runes = runes if runes is not None else RunesTree()

    def validate(self, ingredients: tuple[Item, ...]) -> None:
        if not MIN_INGREDIENTS <= len(ingredients) <= MAX_INGREDIENTS:
            raise SynthesisError("Нужно от 4 до 12 предметов")
        if not all(isinstance(item, Item) for item in ingredients):
```

```

        raise SynthesisError("Некорректные ингредиенты")
    if len({item.item_id for item in ingredients}) != len(ingredients):
        raise SynthesisError("Один предмет указан несколько раз")

    rarities = {item.rarity for item in ingredients}
    if len(rarities) != 1:
        raise SynthesisError("Ингредиенты должны иметь одну редкость")

    rarity = ingredients[0].rarity
    if rarity == Rarity.MYTHIC:
        raise SynthesisError("Mythic – максимальная редкость")

    levels = [item.level for item in ingredients]
    if max(levels) - min(levels) > MAX_LEVEL_SPREAD:
        raise SynthesisError("Слишком большой диапазон уровней")

def probabilities(
    self,
    ingredients: tuple[Item, ...],
    critical_bonus: float = 0.0,
) -> Mapping[Rarity, float]:
    self.validate(ingredients)

    if not 0.0 <= critical_bonus <= 1.0:
        raise SynthesisError("Некорректный бонус синтеза")

    count = len(ingredients)
    tier = RARITY_ORDER.index(ingredients[0].rarity)

    # 4 предмета: 40% оставить ранг.
    # 12 предметов: 0% оставить ранг.
    same = 0.40 * (MAX_INGREDIENTS - count) / (
        MAX_INGREDIENTS - MIN_INGREDIENTS
    )

    critical = min(
        1.0 - same,
        BASE_CRITICAL_CHANCE * count / MAX_INGREDIENTS + critical_bonus,
    )

    upgrade = 1.0 - same - critical

    probabilities: dict[Rarity, float] = {}

```

```

        for offset, chance in ((0, same), (1, upgrade), (2, critical)):
            result_tier = min(tier + offset, len(RARITY_ORDER) - 1)
            rarity = RARITY_ORDER[result_tier]
            probabilities[rarity] = probabilities.get(rarity, 0.0) + chance

        return MappingProxyType(probabilities)

def autofill(
    self,
    stash: tuple[Item, ...],
    *,
    rarity: Rarity | str | None = None,
    excluded_ids: frozenset[str] = frozenset(),
) -> tuple[str, ...]:
    """
    Детерминированно выбирает крупнейшую подходящую группу.
    При равенстве: меньшая редкость, меньший уровень, item_id.

    excluded_ids предназначен для будущих блокировок/избранных предметов.
    """
    rarity_filter = None if rarity is None else Rarity(rarity)
    best: tuple[Item, ...] = ()

    for tier in RARITY_ORDER[::-1]:
        if rarity_filter is not None and tier != rarity_filter:
            continue

        candidates = sorted(
            (
                item for item in stash
                if item.rarity == tier
                and item.item_id not in excluded_ids
            ),
            key=lambda item: (item.level, item.item_id),
        )

        for start, first in enumerate(candidates):
            group = tuple(
                item
                for item in candidates[start:start + MAX_INGREDIENTS]
                if item.level <= first.level + MAX_LEVEL_SPREAD

```

```

    )
    if len(group) >= MIN_INGREDIENTS and len(group) > len(best):
        best = group
    return tuple(item.item_id for item in best)

def calculate(
    self,
    data: GameData,
    ingredient_ids: tuple[str, ...],
    *,
    rng: random.Random | None = None,
) -> SynthesisResult:
    if len(set(ingredient_ids)) != len(ingredient_ids):
        raise SynthesisError("Повторяющийся идентификатор")

    stash_by_id = {item.item_id: item for item in data.stash}

    try:
        ingredients = tuple(
            stash_by_id[item_id] for item_id in ingredient_ids
        )
    except KeyError as exc:
        raise SynthesisError("Предмет отсутствует в тайнике") from exc

    bonus = self.runes.effects(data.unlocked_runes).synthesis_bonus
    probabilities = self.probabilities(ingredients, bonus)

    generator = rng if rng is not None else random.Random()

    rarities = tuple(probabilities)
    result_rarity = generator.choices(
        rarities,
        weights=tuple(probabilities.values()),
        k=1,
    )[0]

    level = sum(item.level for item in ingredients) // len(ingredients)
    slot = generator.choice(ingredients).slot

    item = generate_item(

```

```

        level,
        result_rarity,
        slot=slot,
        source_wave=max(entry.source_wave for entry in ingredients),
        rng=generator,
    )

    # Не допускаем конфликт даже с экипированными предметами.
    known_ids = set(stash_by_id)
    known_ids.update(
        entry.item_id
        for build in data.hero_builds
        for entry in build.equipment
    )
    known_ids.update(item.item_id for item in data.backpack)
    while item.item_id in known_ids:
        item = replace(
            item,
            item_id=f"item-{generator.getrandbits(128):032x}",
        )

    consumed = set(ingredient_ids)
    stash = tuple(
        entry for entry in data.stash
        if entry.item_id not in consumed
    ) + (item,)

    after = replace(data, stash=stash)

    return SynthesisResult(
        before=data,
        after=after,
        consumed_ids=ingredient_ids,
        item=item,
        probabilities=probabilities,
    )

def transmute(
    self,
    state: GameState,
    saves: SaveManager,
    ingredient_ids: tuple[str, ...],

```

```

    *,
    rng: random.Random | None = None,
) -> SynthesisResult:
    state.assert_owner_thread()
    result = self.calculate(state.data, ingredient_ids, rng=rng)

    saves.write(result.after)
    state.commit(result.before, result.after)

    return result

```

Файл: engine/runes_tree.py

```

`python
from future import annotations

import math
from dataclasses import dataclass, field, replace
from enum import Enum
from types import MappingProxyType

import config as C
from engine.save_manager import SaveManager
from engine.state import GameData, GameState
from models.hero import Hero, SecondaryStats
from models.stats import StatBlock

class RuneError(ValueError):
    pass

class Sector(str, Enum):
    WAR = "war"
    ETHER = "ether"
    VITALITY = "vitality"
    GREED = "greed"
    CHRONOMANCY = "chronomancy"

```



```

SECTOR_NAMES = {
Sector.WAR: "Война",
Sector.ETHER: "Эфир",
Sector.VITALITY: "Живучесть",
Sector.GREED: "Алчность",
Sector.CHRONOMANCY: "Хрономантия",
}

```

```

@dataclass(frozen=True)
class RuneEffects:
stats: StatBlock = field(default_factory=StatBlock)
gold_bonus: float = 0.0
sale_bonus: float = 0.0
drop_bonus: float = 0.0

```

```

cap_steps: int = 0
efficiency_steps: int = 0
synthesis_bonus: float = 0.0
active_speed_bonus: float = 0.0
keystones: tuple[str, ...] = ()
def __add__(self, other: RuneEffects) -> RuneEffects:
    return RuneEffects(
        stats=self.stats + other.stats,
        gold_bonus=self.gold_bonus + other.gold_bonus,
        sale_bonus=self.sale_bonus + other.sale_bonus,
        drop_bonus=self.drop_bonus + other.drop_bonus,
        cap_steps=self.cap_steps + other.cap_steps,
        efficiency_steps=self.efficiency_steps + other.efficiency_steps,
        synthesis_bonus=self.synthesis_bonus + other.synthesis_bonus,
        active_speed_bonus=self.active_speed_bonus + other.active_speed_bonus,
        keystones=tuple(sorted(set(self.keystones + other.keystones))),
    )

```

```

@dataclass(frozen=True)
class RuneNode:
node_id: str

```

```

name: str
sector: Sector | None
cost: int
prerequisites: tuple[str, ...]
effects: RuneEffects
position: tuple[float, float]
keystone: bool = False

@dataclass(frozen=True)
class SquadMember:
    hero: Hero
    stats: SecondaryStats

```

```
def node_cost(index: int, keystone: bool = False) -> int:
```

```
    """
```

```
     $C(i) = \text{ceil\_to\_1000}(5000 * 1.45^i)$ .
```

```
    Расчёт через целые числа, без float в экономике.
```

```
    """
```

```
    numerator = 5000 * 145index
```

```
    denominator = 100index
```

```

cost = (
    (numerator + denominator * 1000 - 1)
    // (denominator * 1000)
    * 1000
)
return max(cost, 250_000_000) if keystone else cost

```

```
def normal_effect(sector: Sector, index: int) -> RuneEffects:
```

```
    if sector == Sector.WAR:
```

```
        options = (
```

```
            RuneEffects(stats=StatBlock(physical_bonus=0.02)),
```

```
            RuneEffects(stats=StatBlock(strength=2)),
```

```
            RuneEffects(stats=StatBlock(crit_damage=0.05)),
```

```
            RuneEffects(stats=StatBlock(crit_chance=0.01)),
```

```
        )
```

```
    elif sector == Sector.ETHER:
```

```

options = (
    RuneEffects(stats=StatBlock(magical_bonus=0.02)),
    RuneEffects(stats=StatBlock(intelligence=2)),
    RuneEffects(stats=StatBlock(mana=10)),
)
elif sector == Sector.VITALITY:
    options = (
        RuneEffects(stats=StatBlock(hp_bonus=0.03)),
        RuneEffects(stats=StatBlock(armor=4)),
        RuneEffects(stats=StatBlock(vitality=2)),
        RuneEffects(stats=StatBlock(evasion=0.008)),
    )
elif sector == Sector.GREED:
    options = (
        RuneEffects(gold_bonus=0.03),
        RuneEffects(sale_bonus=0.02),
        RuneEffects(drop_bonus=0.002),
    )
else:
    options = (
        RuneEffects(stats=StatBlock(mana=5)),
        RuneEffects(stats=StatBlock(luck=1)),
        RuneEffects(stats=StatBlock(max_hp=5)),
    )

```

```

return options[index % len(options)]

```

```

def build_nodes() -> dict[str, RuneNode]:
    nodes = {
        "nexus": RuneNode(
            "nexus",
            "Сердце Бездны",
            None,
            0,
            (),
            RuneEffects(),

```

```
(0.0, 0.0),  
)  
}
```

```
# Цепь родителей в двоичном дереве:  
# 0 -> 1 -> 3 -> 7 -> 15.  
cap_nodes = {  
    0: "Сон Бездны I – 3 часа",  
    1: "Сон Бездны II – 4 часа",  
    3: "Сон Бездны III – 6 часов",  
    7: "Вневременной Покой – 8 часов",  
    15: "Вечность Бездны – 12 часов",  
}  
  
# Вторая последовательная ветка: 0 -> 2 -> 5 -> 11.  
efficiency_nodes = {  
    2: "Ясновидение I – 75%",  
    5: "Ясновидение II – 90%",  
    11: "Ясновидение III – 100%",  
}  
  
great_runes = {  
    Sector.WAR: (  
        ("Берсерк Бездны", "berserker"),  
        ("Титанический Размах", "titan_sweep"),  
    ),  
    Sector.ETHER: (  
        ("Сверхновая", "supernova"),  
        ("Эфирный Щит", "ether_shield"),  
    ),  
    Sector.VITALITY: (  
        ("Железный Монолит", "iron_monolith"),  
        ("Второе Дыхание", "second_wind"),  
    ),  
    Sector.GREED: (  
        ("Золотая Лихорадка", "gold_fever"),  
        ("Алхимический Синтез", "alchemical_synthesis"),  
    ),  
    Sector.CHRONOMANCY: (  
        ("Ускорение I", "active_speed_1"),  
        ("Ускорение II", "active_speed_2"),  
    ),  
}
```

```

}

for sector_index, sector in enumerate(Sector):
    angle = sector_index * math.tau / len(Sector) - math.pi / 2

    for index in range(30):
        node_id = f"{sector.value}.{index:02d}"
        parent = (
            "nexus"
            if index == 0
            else f"{sector.value}.{(index - 1) // 2:02d}"
        )

        depth = (index + 1).bit_length() - 1
        first_at_depth = 2**depth - 1
        position_in_depth = index - first_at_depth
        count_at_depth = 2**depth

        spread = (
            (position_in_depth + 0.5) / count_at_depth - 0.5
        ) * 0.95
        radius = 160 + depth * 120

        position = (
            math.cos(angle + spread) * radius,
            math.sin(angle + spread) * radius,
        )

        effect = normal_effect(sector, index)
        name = f"{SECTOR_NAMES[sector]} - узел {index + 1}"
        is_keystone = index in (28, 29)

        if sector == Sector.CHRONOMANCY:
            if index in cap_nodes:
                name = cap_nodes[index]
                effect = RuneEffects(cap_steps=1)
            elif index in efficiency_nodes:
                name = efficiency_nodes[index]
                effect = RuneEffects(efficiency_steps=1)

        if is_keystone:
            name, flag = great_runes[sector][index - 28]

```

```

        effect = RuneEffects(keystones=(flag,))

    if flag == "alchemical_synthesis":
        effect = RuneEffects(
            synthesis_bonus=0.05,
            keystones=(flag,),
        )
    elif flag in ("active_speed_1", "active_speed_2"):
        effect = RuneEffects(
            active_speed_bonus=0.25,
            keystones=(flag,),
        )

    nodes[node_id] = RuneNode(
        node_id=node_id,
        name=name,
        sector=sector,
        cost=node_cost(index, is_keystone),
        prerequisites=(parent,),
        effects=effect,
        position=position,
        keystone=is_keystone,
    )

return nodes

```

class RunesTree:

def **init**(self) -> None:

self.nodes = MappingProxyType(build_nodes())

```

@property
def edges(self) -> tuple[tuple[str, str], ...]:
    return tuple(
        (parent, node.node_id)
        for node in self.nodes.values()
        for parent in node.prerequisites
    )

def validate_unlocked(self, unlocked: tuple[str, ...]) -> None:
    if type(unlocked) is not tuple:
        raise RuneError("Список рун должен быть tuple")
    if len(unlocked) != len(set(unlocked)):

```

```

        raise RuneError("Повторяющиеся руны")
    if "nexus" in unlocked:
        raise RuneError("Центр открыт автоматически и не сохраняется")

    owned = set(unlocked) | {"nexus"}

    for node_id in unlocked:
        if node_id not in self.nodes:
            raise RuneError(f"Неизвестная руна: {node_id}")
        if not set(self.nodes[node_id].prerequisites) <= owned:
            raise RuneError(f"Нарушены связи руны {node_id}")

def effects(self, unlocked: tuple[str, ...]) -> RuneEffects:
    self.validate_unlocked(unlocked)
    result = RuneEffects()

    # Одинаковый порядок суммирования для любого порядка покупок.
    for node_id in sorted(unlocked):
        result = result + self.nodes[node_id].effects

    return result

def calculate_purchase(self, data: GameData, node_id: str) -> GameData:
    self.validate_unlocked(data.unlocked_runes)

    if node_id not in self.nodes or node_id == "nexus":
        raise RuneError("Эту руну нельзя купить")
    if node_id in data.unlocked_runes:
        raise RuneError("Руна уже куплена")

    node = self.nodes[node_id]
    owned = set(data.unlocked_runes) | {"nexus"}

    if not set(node.prerequisites) <= owned:
        raise RuneError("Сначала откройте связанные предыдущие узлы")
    if data.gold < node.cost:
        raise RuneError("Недостаточно золота")

    effect = node.effects

    # Прибавляем только эффект новой руны.
    # Полный набор рун повторно в farm не применяется.

```

```

    farm = replace(
        data.farm,
        gold_bonus=data.farm.gold_bonus + effect.gold_bonus,
        sale_bonus=data.farm.sale_bonus + effect.sale_bonus,
        drop_chance=min(1.0, data.farm.drop_chance + effect.drop_bonus),
    )

    return replace(
        data,
        gold=data.gold - node.cost,
        farm=farm,
        unlocked_runes=tuple(sorted((*data.unlocked_runes, node_id))),
        offline_cap_level=min(
            len(C.OFFLINE_CAP_SECONDS) - 1,
            data.offline_cap_level + effect.cap_steps,
        ),
        offline_efficiency_level=min(
            len(C.OFFLINE_EFFICIENCIES) - 1,
            data.offline_efficiency_level + effect.efficiency_steps,
        ),
    )

def buy(
    self,
    state: GameState,
    saves: SaveManager,
    node_id: str,
) -> GameData:
    state.assert_owner_thread()
    before = state.data
    after = self.calculate_purchase(before, node_id)

    saves.write(after)
    state.commit(before, after)
    return after

def squad(self, data: GameData) -> tuple[SquadMember, ...]:
    """Вычисленные характеристики активного отряда с экипировкой и рунами."""
    effects = self.effects(data.unlocked_runes)
    builds = {build.hero_id: build for build in data.hero_builds}
    members = []

```



```

    for snapshot in data.party:
        hero = Hero.from_state(snapshot, builds.get(snapshot.hero_id))
        stats = hero.secondary_stats(effects.stats)

        # Этот Keystone безусловный – его можно применить уже сейчас.
        if "iron_monolith" in effects.keystones:
            stats = replace(
                stats,
                armor=stats.armor * 1.5,
                evasion=0.0,
            )

        members.append(SquadMember(hero, stats))

    return tuple(members)

```

Файл: engine/offline_manager.py

```

`python
from future import annotations

from engine.runes_tree import RunesTree
from models.hero import experience_awards

import math
import random
import time

from dataclasses import dataclass, replace
from fractions import Fraction

import config as C
from engine.save_manager import SaveManager
from engine.state import GameData, GameState, ItemDrop, require_number
from models.item import generate_item

def rational(value: int | float | str) -> Fraction:
    """Без двоичных float-артефактов при преобразовании коэффициентов."""
    return Fraction(str(value))

```

```
@dataclass(frozen=True)
```

```
class OfflineResult:
```

```
    before: GameData
```

```
    after: GameData
```

```
    elapsed_seconds: float
```

```
    credited_seconds: float
```

```
    effective_seconds: float
```

```
    discarded_seconds: float
```

```
    waves_cleared: int
```

```
    wave_gold: int
```

```
    sale_gold: int
```

```
    total_exp: int
```

```
    items_stored: int
```

```
    items_sold: int
```

```
    clock_rollback: bool
```

```
    @property
```

```
    def total_gold(self) -> int:
```

```
        return self.wave_gold + self.sale_gold
```

```
    @property
```

```
    def items_rolled(self) -> int:
```

```
        return self.items_stored + self.items_sold
```

```
    @property
```

```
    def cap_reached(self) -> bool:
```

```
        return self.elapsed_seconds >= self.before.offline_cap_seconds
```

```
class OfflineManager:
```

```
    def calculate(
```

```
        self,
```

```
        data: GameData,
```

```
        now: float | None = None,
```

```
        rng: random.Random | None = None,
```

```
    ) -> OfflineResult:
```

```
    """
```

Рассчитать новый снимок, не изменяя GameState и файл сохранения.

```
rng можно передать для воспроизводимых тестов.
```

```
Профиль фарма фиксируется на весь оффлайн-интервал.
```

```
"""
```

```
timestamp = time.time() if now is None else now
```

```
require_number("now", timestamp)
```

```
generator = rng if rng is not None else random.Random()
```

```
previous_time = rational(data.last_active_timestamp)
```

```
current_time = rational(timestamp)
```

```
raw_seconds = max(Fraction(0), current_time - previous_time)
```

```
credited_seconds = min(
```

```
    raw_seconds,
```

```
    Fraction(data.offline_cap_seconds),
```

```
)
```

```
effective_seconds = (
```

```
    credited_seconds * rational(data.offline_efficiency)
```

```
)
```

```
progress = (
```

```
    Fraction(data.wave_fraction)
```

```
    + effective_seconds
```

```
    / rational(data.farm.average_wave_clear_seconds)
```

```
)
```

```
waves = math.floor(progress)
```

```
remainder = progress - waves
```

```
# Единое правило округления, независимое от размера оффлайн-пакета.
```

```
gold_per_wave = math.floor(
```

```
    data.farm.gold_per_wave
```

```
    * (1 + rational(data.farm.gold_bonus))
```

```
)
```

```
wave_gold = waves * gold_per_wave
```

```
members = RunesTree().squad(data)
```

```
gains, exp_cursor = experience_awards(
```

```
    data.farm.exp_per_wave,
```

```
    waves,
```

```
    [member.stats for member in members],
```

```
    data.exp_cursor,
```

```
)
```

```

total_exp = sum(gains)
party = tuple(
    replace(
        hero,
        total_exp=hero.total_exp + gains[index],
    )
    for index, hero in enumerate(data.party)
)

stash = list(data.stash)
known_item_ids = {item.item_id for item in stash}
known_item_ids.update(
    item.item_id
    for build in data.hero_builds
    for item in build.equipment
)
known_item_ids.update(item.item_id for item in data.backpack)
items_stored = 0
items_sold = 0
sale_gold = 0

rarity_names = [row[0] for row in C.LOOT_TABLE]
rarity_weights = [row[1] for row in C.LOOT_TABLE]
base_prices = {row[0]: row[2] for row in C.LOOT_TABLE}

for wave_number in range(
    data.current_wave,
    data.current_wave + waves,
):
    if generator.random() >= data.farm.drop_chance:
        continue

    rarity = generator.choices(
        rarity_names,
        weights=rarity_weights,
        k=1,
    )[0]

    base_sell_value = base_prices[rarity]

    if len(stash) >= data.stash_capacity:

```

```

        # Начальное правило утилизации:
        # любой избыточный предмет автоматически продаётся.
        items_sold += 1
        sale_gold += math.floor(
            base_sell_value
            * (1 + rational(data.farm.sale_bonus))
        )
        continue

    item_id = f"offline-{generator.getrandbits(128):032x}"
    while item_id in known_item_ids:
        item_id = f"offline-{generator.getrandbits(128):032x}"

    # Стартовый баланс: один уровень предмета на каждые 100 волн.
    # Цену продажи сохраняем прежней, чтобы не менять экономику Модуля 1.
    item = generate_item(
        level=1 + (wave_number - 1) // 100,
        rarity=rarity,
        source_wave=wave_number,
        rng=generator,
    )
    item = replace(
        item,
        item_id=item_id,
        sell_value=base_sell_value,
    )
    known_item_ids.add(item_id)
    stash.append(item)
    items_stored += 1

    after = replace(
        data,
        gold=data.gold + wave_gold + sale_gold,
        cleared_waves=data.cleared_waves + waves,
        wave_fraction=str(remainder),
        party=party,
        exp_cursor=exp_cursor,
        stash=tuple(stash),
    )

    # Обрабатываем весь интервал, включая отброшенное сверх cap.
    # Иначе повторный запуск мог бы получить ещё одну порцию наград.
    last_active_timestamp=max(

```

```

        timestamp,
        data.last_active_timestamp,
    ),
)

return OfflineResult(
    before=data,
    after=after,
    elapsed_seconds=float(raw_seconds),
    credited_seconds=float(credited_seconds),
    effective_seconds=float(effective_seconds),
    discarded_seconds=float(raw_seconds - credited_seconds),
    waves_cleared=waves,
    wave_gold=wave_gold,
    sale_gold=sale_gold,
    total_exp=total_exp,
    items_stored=items_stored,
    items_sold=items_sold,
    clock_rollback=current_time < previous_time,
)

def claim(
    self,
    state: GameState,
    saves: SaveManager,
    now: float | None = None,
    rng: random.Random | None = None,
) -> OfflineResult:
    """
    Вызывается один раз после загрузки сохранения.

    1. Рассчитать кандидат.
    2. Атомарно записать награды и timestamp.
    3. Обновить GameState и послать сигналы.

    Если запись не удалась, состояние в памяти остаётся прежним.
    """
    state.assert_owner_thread()
    result = self.calculate(state.data, now=now, rng=rng)

    saves.write(result.after)
    state.commit(

```

```

        result.before,
        result.after,
        offline_result=result,
    )

    return result

```

Файл: engine/save_manager.py

```

`python
from future import annotations

import json
import time
from dataclasses import asdict, replace
from pathlib import Path

from PySide6.QtCore import QIODevice, QSaveFile

import config as C
from engine.state import (
    FarmProfile,
    GameData,
    GameState,
    HeroState,
    ItemDrop,
    require_number,
)
from models.hero import build_from_dict
from models.item import item_from_dict

class SaveError(RuntimeError):
    pass

class SaveManager:
    def init(self, path: Path = C.SAVE_PATH) -> None:
        self.path = Path(path)

```

```

def load(self) -> GameData:
    if not self.path.exists():
        return GameData()

    try:
        payload = json.loads(self.path.read_text(encoding="utf-8"))
        version = payload["schema_version"]

        if type(version) is not int:
            raise ValueError("Некорректная версия сохранения")
        if version not in (1, 2, 3, 4, 5, C.SAVE_SCHEMA_VERSION):
            raise ValueError(f"Неподдерживаемая версия: {version}")

        raw = dict(payload["state"])

        raw["party"] = tuple(
            HeroState(**hero) for hero in raw["party"]
        )
        raw["stash"] = tuple(
            item_from_dict(item) for item in raw["stash"]
        )
        raw["backpack"] = tuple(
            item_from_dict(item)
            for item in raw.get("backpack", ())
        )
        raw["farm"] = FarmProfile(**raw["farm"])

        raw["hero_builds"] = tuple(
            build_from_dict(build)
            for build in raw.get("hero_builds", ())
        )
        raw["unlocked_runes"] = tuple(
            raw.get("unlocked_runes", ())
        )

        raw["reward_claims"] = tuple(raw.get("reward_claims", ()))
        data = GameData(**raw)

        # Импорт внутри метода: runes_tree использует SaveManager.
        from engine.runes_tree import RunesTree
        RunesTree().validate_unlocked(data.unlocked_runes)

```



```

        return data

    except (
        OSError,
        UnicodeError,
        ValueError,
        TypeError,
        KeyError,
        OverflowError,
    ) as exc:
        raise SaveError(
            f"Не удалось загрузить {self.path}: {exc}"
        ) from exc

def write(self, data: GameData) -> None:
    payload = {
        "schema_version": C.SAVE_SCHEMA_VERSION,
        "state": asdict(data),
    }

    try:
        encoded = json.dumps(
            payload,
            ensure_ascii=False,
            indent=2,
            allow_nan=False,
        ).encode("utf-8")
        self.path.parent.mkdir(parents=True, exist_ok=True)
    except (OSError, ValueError, TypeError) as exc:
        raise SaveError(f"Ошибка подготовки сохранения: {exc}") from exc

    file = QSaveFile(str(self.path))
    file.setDirectWriteFallback(False)

    if not file.open(QIODevice.OpenModeFlag.WriteOnly):
        raise SaveError(file.errorString())

    if file.write(encoded) != len(encoded):
        message = file.errorString()
        file.cancelWriting()
        raise SaveError(f"Ошибка записи: {message}")

```

```

    if not file.commit():
        raise SaveError(f"Ошибка фиксации сохранения: {file.errorString()}")

def checkpoint(
    self,
    state: GameState,
    now: float | None = None,
) -> None:
    """Сохранить активную сессию без начисления оффлайн-наград."""
    state.assert_owner_thread()

    timestamp = time.time() if now is None else now
    require_number("now", timestamp)

    before = state.data
    after = replace(
        before,
        # Не откатываем уже обработанное время назад.
        last_active_timestamp=max(
            timestamp,
            before.last_active_timestamp,
        ),
    )

    self.write(after)
    state.commit(before, after)

```

Файл: **gfx/animations.py**

```

`python
from future import annotations

import math
from enum import Enum

class AnimationState(str, Enum):
    IDLE = "idle"
    RUN = "run"

```

```
ATTACK = "attack"
```

```
CAST = "cast"
```

```
HURT = "hurt"
```

```
DIE = "die"
```

```
LOOPING = {
```

```
    AnimationState.IDLE,
```

```
    AnimationState.RUN,
```

```
}
```

```
DURATIONS = {
```

```
    AnimationState.IDLE: 0.80,
```

```
    AnimationState.RUN: 0.50,
```

```
    AnimationState.ATTACK: 0.35,
```

```
    AnimationState.CAST: 0.45,
```

```
    AnimationState.HURT: 0.20,
```

```
    AnimationState.DIE: 0.60,
```

```
}
```

```
class AnimationController:
```

```
    def init(self, frame_count: int = 4) -> None:
```

```
        if type(frame_count) is not int or frame_count < 1:
```

```
            raise ValueError("frame_count должен быть целым >= 1")
```

```
        self.frame_count = frame_count
```

```
        self.state = AnimationState.IDLE
```

```
        self.elapsed = 0.0
```

```
    def set_state(self, state: AnimationState | str) -> bool:
```

```
        state = AnimationState(state)
```

```
        # Главное правило FSM: одинаковое состояние не сбрасывает кадр.
```

```
        if state == self.state:
```

```
            return False
```

```
        self.state = state
```

```
        self.elapsed = 0.0
```

```
        return True
```

```
    def advance(self, seconds: float) -> None:
```

```

    if not math.isfinite(seconds) or seconds < 0:
        raise ValueError("Некорректная дельта времени")

    duration = DURATIONS[self.state]

    if self.state in LOOPING:
        self.elapsed = (self.elapsed + seconds) % duration
    else:
        # Неповторяющиеся анимации удерживают последний кадр.
        # Переход обратно в IDLE задаёт боевой движок.
        self.elapsed = min(duration, self.elapsed + seconds)

@property
def frame_index(self) -> int:
    duration = DURATIONS[self.state]
    return min(
        self.frame_count - 1,
        int(self.elapsed / duration * self.frame_count),
    )

@property
def finished(self) -> bool:
    return (
        self.state not in LOOPING
        and self.elapsed >= DURATIONS[self.state]
    )

```

Файл: `gfx/sprite_loader.py`

```

`python
from future import annotations

import hashlib
import math
from pathlib import Path

from PySide6.QtCore import QPoint, Qt
from PySide6.QtGui import QColor, QImage, QPainter, QPixmap, QPolygon

```

```

import config as C
from gfx.animations import AnimationState

class SpriteLoader:
    def init(self, root: Path = C.SPRITES_DIR) -> None:
        self.root = Path(root)
        self._cache: dict[tuple, tuple[QPixmap, ...]] = {}

    def clear(self) -> None:
        self._cache.clear()

    def frames(
        self,
        key: str,
        state: AnimationState | str,
        *,
        size: int = 36,
        dpr: float = 1.0,
        frame_count: int = 4,
    ) -> tuple[QPixmap, ...]:
        state = AnimationState(state)

        if type(size) is not int or size < 1:
            raise ValueError("Некорректный размер спрайта")
        if type(frame_count) is not int or frame_count < 1:
            raise ValueError("Некорректное количество кадров")
        if not math.isfinite(dpr) or dpr <= 0:
            raise ValueError("Некорректный DPR")

        dpr = round(dpr, 3)
        cache_key = (key, state.value, size, dpr, frame_count)

        if cache_key in self._cache:
            return self._cache[cache_key]

        path = self.root / key / f"{state.value}.png"
        sheet = QImage(str(path)) if path.is_file() else QImage()

        valid_sheet = (
            not sheet.isNull()
            and sheet.width() >= frame_count
            and sheet.width() % frame_count == 0

```

```

    )
    physical_size = max(1, math.ceil(size * dpr))
    frames = []
    for index in range(frame_count):
        if valid_sheet:
            width = sheet.width() // frame_count
            image = sheet.copy(
                index * width, 0, width, sheet.height()
            )
        else:
            image = self._pixel_frame(
                key,
                state,
                index,
                frame_count,
                boss=size >= 48,
            )
        image = image.scaled(
            physical_size,
            physical_size,
            Qt.AspectRatioMode.IgnoreAspectRatio,
            Qt.TransformationMode.FastTransformation,
        )
        pixmap = QPixmap.fromImage(image)
        pixmap.setDevicePixelRatio(dpr)
        frames.append(pixmap)

    result = tuple(frames)
    self._cache[cache_key] = result
    return result

    @staticmethod
    def _pixel_frame(
        key: str,
        state: AnimationState,
        frame: int,
        count: int,
        *,
        boss: bool,

```

```

) -> QImage:
    image = QImage(32, 32, QImage.Format.Format_ARGB32_Premultiplied)
    image.fill(Qt.GlobalColor.transparent)

    p = QPainter(image)
    p.setRenderHint(QPainter.RenderHint.Antialiasing, False)
    p.setPen(Qt.PenStyle.NoPen)

    aliases = {
        "warlock": "pyromancer",
        "priest": "paladin",
        "fire_imp": "imp",
    }
    key = aliases.get(key, key)

    hero_keys = {
        "knight", "assassin", "pyromancer",
        "ranger", "necromancer", "paladin",
    }
    is_hero = key in hero_keys
    boss = boss and not is_hero

    phase = frame % 4
    progress = frame / max(1, count - 1)
    walking = state == AnimationState.RUN
    attacking = state == AnimationState.ATTACK
    casting = state == AnimationState.CAST

    bob = (0, -1, 0, 1)[phase] if walking else (0, 0, -1, 0)[phase]
    stride = (-2, 0, 2, 0)[phase] if walking else 0

    if state == AnimationState.DIE:
        p.setOpacity(max(0.15, 1.0 - progress * 0.80))
        p.translate(0, int(progress * 20))
        p.scale(1.0, max(0.25, 1.0 - progress * 0.75))
    else:
        p.translate(0, bob)

    def rect(x, y, w, h, color):
        p.fillRect(int(x), int(y), int(w), int(h), QColor(color))

    def poly(points, color):

```

```

        p.setBrush(QColor(color))
        p.setPen(Qt.PenStyle.NoPen)
        p.drawPolygon(QPolygon([QPoint(x, y) for x, y in points]))

```

```

def line(x1, y1, x2, y2, color, width=1):

```

```

    from PySide6.QtGui import QPen
    p.setPen(QPen(QColor(color), width))
    p.drawLine(x1, y1, x2, y2)
    p.setPen(Qt.PenStyle.NoPen)

```

```

outline = "#15121e"

```

```

skin = "#d8ae85"

```

```

bone = "#d6cfb3"

```

```

steel = "#8796ad"

```

```

shine = "#d2dde3"

```

```

gold = "#d2a65d"

```

```

# Все персонажи ориентированы вправо.

```

```

# BattleStage зеркалит врагов при рисовании.

```

```

if is_hero:

```

```

    cloth = {
        "knight": "#5b3452",
        "assassin": "#443450",
        "pyromancer": "#713b99",
        "ranger": "#3d654c",
        "necromancer": "#355d59",
        "paladin": "#ddd2ad",
    }[key]

```

```

    poly([(9, 14), (20, 14), (24, 29), (6, 29)], outline)

```

```

    poly([(10, 14), (19, 14), (22, 27), (8, 27)], cloth)

```

```

    rect(10, 16, 3, 10, QColor(cloth).lighter(135).name())

```

```

    rect(17, 16, 3, 11, QColor(cloth).darker(145).name())

```

```

    rect(10 + stride, 27, 5, 4, outline)

```

```

    rect(18 - stride, 27, 5, 4, outline)

```

```

    rect(10 + stride, 27, 4, 2, "#655663")

```

```

    rect(18 - stride, 27, 4, 2, "#655663")

```

```

    if key in ("knight", "paladin"):

```

```

        rect(10, 7, 11, 9, outline)

```

```

        rect(11, 6, 9, 9, steel if key == "knight" else gold)

```



```
rect(12, 6, 3, 7, shine)
```

```
rect(11, 11, 9, 2, "#202332")
```

```
rect(18, 11, 2, 1, "#e2b756")
```

```
rect(15, 5, 3, 3, "#a74655")
```

```
rect(9, 16, 14, 10, outline)
```

```
rect(10, 16, 12, 8, steel)
```

```
rect(11, 17, 4, 6, shine)
```

```
rect(16, 17, 4, 6, "#58677e")
```

```
rect(9, 16, 4, 4, "#b4c0cd")
```

```
rect(20, 16, 4, 4, "#aeb9c5")
```

```
rect(10, 24, 12, 2, gold)
```

```
if key == "knight":
```

```
    poly(
```

```
        [(4, 17), (10, 15), (14, 18), (12, 25), (8, 29), (4, 25)],
```

```
        outline,
```

```
    )
```

```
    poly(
```

```
        [(5, 18), (10, 17), (12, 19), (10, 25), (8, 27), (5, 24)],
```

```
        "#536b96",
```

```
    )
```

```
    rect(8, 18, 2, 8, gold)
```

```
    rect(6, 20, 5, 2, gold)
```

```
if attacking:
```

```
    rect(23, 17, 7, 2, shine)
```

```
    rect(22, 15, 2, 6, gold)
```

```
    rect(19, 17, 3, 2, "#715044")
```

```
    rect(29, 16, 2, 1, "#eff4df")
```

```
else:
```

```
    rect(25, 6, 2, 16, shine)
```

```
    rect(24, 7, 1, 13, "#748eac")
```

```
    rect(22, 21, 7, 2, gold)
```

```
    rect(25, 23, 2, 5, "#604433")
```

```
else:
```

```
    rect(26, 8, 2, 22, "#93603a")
```

```
    rect(25, 8, 1, 21, gold)
```

```
    rect(24, 5, 6, 5, "#eadbad")
```

```
    rect(26, 3, 2, 9, "#fff0c2")
```

```
    rect(23, 7, 8, 2, "#fff0c2")
```

```
    rect(14, 18, 2, 6, gold)
```

```

rect(12, 20, 6, 2, gold)
elif key in ("assassin", "ranger"):
    hood = "#342943" if key == "assassin" else "#2a4b36"
    poly([(9, 12), (10, 6), (15, 3), (21, 7), (23, 15)], outline)
    poly([(10, 12), (12, 7), (16, 5), (20, 8), (21, 14)], hood)
    rect(13, 10, 7, 4, "#11131b")
    rect(17, 10, 2, 1, "#dfbd71")
    rect(20, 13, 3, 7, hood)
    rect(10, 22, 11, 2, "#8a6540")
    rect(14, 21, 3, 3, gold)

    if key == "assassin":
        reach = (2, 5, 7, 3)[phase] if attacking else 0
        rect(21, 18, 4, 3, skin)
        poly(
            [(24, 18), (29 + min(2, reach), 15), (27, 21), (24, 21)],
            shine,
        )
        rect(23, 18, 1, 4, gold)
        rect(6, 21, 5, 2, skin)
        poly([(4, 17), (7, 20), (7, 25)], "#b6c7d9")
    else:
        line(26, 10, 29, 18, "#bb8956", 2)
        line(29, 18, 26, 27, "#bb8956", 2)
        line(26, 10, 26, 27, "#d3c4a6")
        line(21, 18, 31, 18, "#d9d7bf")
        rect(21, 17, 3, 3, skin)
        rect(7, 12, 3, 10, "#664531")
        line(7, 10, 10, 6, "#c1b797")

    else:
        # Колдун/пиромант: широкополая остроконечная шляпа.
        # Некромант: тёмная корона-капюшон и посох с черепом.
        hat = cloth if key == "pyromancer" else "#24453f"

        rect(12, 10, 8, 7, skin)
        rect(17, 12, 2, 1, "#241e2d")
        rect(11, 15, 8, 3, "#c2b8ba")
        rect(12, 17, 5, 4, "#d8ccd0")

        poly([(6, 11), (13, 8), (17, 1), (20, 9), (25, 12)], outline)

```

```

poly([(8, 10), (14, 8), (17, 3), (19, 10), (23, 11)], hat)
rect(11, 9, 11, 2, gold)
rect(16, 5, 2, 2, "#ab7acc")

rect(24, 8, 2, 22, "#75513d")
rect(25, 9, 1, 20, "#b38c59")

if key == "necromancer":
    rect(22, 5, 7, 6, bone)
    rect(23, 7, 2, 2, outline)
    rect(26, 7, 2, 2, outline)
    rect(24, 10, 3, 2, bone)
    rect(15, 22, 3, 4, "#70bf91")
else:
    poly([(21, 7), (25, 2), (29, 7), (25, 11)], "#431f68")
    poly([(23, 6), (25, 3), (27, 7), (25, 9)], "#d098fa")
    rect(25, 4, 1, 3, "#f1d2ff")

if casting:
    colors = ("#7250b8", "#a774e0", "#e1b2ff", "#b781ed")
    radius = (2, 3, 4, 3)[phase]
    poly(
        [(27, 12 - radius), (27 + radius, 12),
         (27, 12 + radius), (27 - radius, 12)],
        colors[phase],
    )
    rect(26, 11, 2, 2, "#f7ddff")
    rect(22, 4 + phase, 1, 2, "#c698ef")

elif boss:
    # Рогатый бронированный демон с крыльями.
    poly([(11, 13), (3, 7), (1, 21), (9, 19)], "#442639")
    poly([(21, 13), (29, 7), (31, 21), (23, 19)], "#442639")
    line(4, 11, 8, 17, "#875063")
    line(28, 11, 24, 17, "#875063")

    poly([(8, 15), (24, 15), (26, 27), (6, 27)], outline)
    rect(9, 15, 14, 11, "#70414c")
    rect(11, 16, 4, 8, "#a16a69")
    rect(17, 16, 4, 8, "#462c3e")
    rect(13, 7, 9, 10, "#994751")
    poly([(13, 9), (7, 3), (10, 11)], "#c3a785")

```

```

poly([(21, 9), (26, 2), (24, 12)], "#c3a785")
rect(15, 10, 2, 2, "#ffd55a")
rect(20, 10, 2, 2, "#ffd55a")
rect(17, 14, 5, 1, "#e8cfb0")
rect(8 + stride, 26, 6, 5, "#392536")
rect(20 - stride, 26, 6, 5, "#392536")
rect(28, 12, 2, 18, "#b4a385")
poly([(26, 10), (31, 7), (31, 18), (27, 16)], "#c75f50")

elif key == "bat":
    wing = (5, 10, 16, 10)[phase]
    poly(
        [(14, 16), (5, wing), (0, wing + 3),
         (4, 20), (8, 17), (11, 21)],
        "#33233f",
    )
    poly(
        [(18, 16), (27, wing), (31, wing + 3),
         (28, 20), (24, 17), (21, 21)],
        "#33233f",
    )
    line(3, wing + 3, 13, 17, "#8d5479")
    line(29, wing + 3, 19, 17, "#8d5479")
    rect(13, 12, 7, 12, "#68415f")
    poly([(13, 14), (13, 7), (16, 12)], "#8e577b")
    poly([(17, 12), (20, 7), (20, 15)], "#8e577b")
    rect(16, 14, 2, 1, "#ffb365")
    rect(19, 14, 1, 1, "#ffb365")

elif key == "spider":
    for side in (-1, 1):
        for leg in range(4):
            sy = 15 + leg * 3
            ex = 16 + side * (12 + (leg % 2))
            ey = sy - 4 + ((phase + leg) % 2) * 3
            line(16 + side * 4, sy, ex, ey, "#312a40", 2)
            line(ex, ey, ex + side, ey + 6, "#66546e")
        poly([(8, 16), (11, 10), (20, 10), (24, 17), (20, 25), (11, 25)],
            "#3c2d48")
        rect(12, 12, 7, 3, "#6a4768")
        rect(19, 17, 8, 7, "#201c2e")
        rect(23, 18, 2, 1, "#ed695b")

```

```

    rect(26, 18, 1, 1, "#ed695b")
    line(25, 22, 28, 25, "#c0a28c")
    line(22, 23, 24, 26, "#c0a28c")

elif key in ("skeleton", "sentinel"):
    armor = key == "sentinel"

    rect(12, 5, 10, 9, outline)
    rect(13, 5, 8, 8, bone if not armor else steel)
    rect(14, 8, 2, 2, "#241c28")
    rect(19, 8, 2, 2, "#241c28")
    rect(16, 12, 4, 2, "#b1a58d")

    rect(16, 14, 2, 11, bone)
    for y in (16, 19, 22):
        rect(11, y, 12, 1, bone)
        rect(11, y, 1, 2, bone)
        rect(22, y, 1, 2, bone)

    if armor:
        rect(10, 15, 14, 9, "#66718c")
        rect(12, 16, 4, 6, "#a0aec1")
        rect(18, 16, 3, 6, "#3f465e")

    line(12, 24, 10 + stride, 30, bone, 2)
    line(21, 24, 23 - stride, 30, bone, 2)
    line(22, 16, 26, 21, bone, 2)
    rect(26, 9 if not attacking else 17, 2, 13, "#b7bec5")
    rect(24, 22, 6, 2, "#856744")

elif key == "imp":
    tail = (0, 1, 0, -1)[phase]
    poly([(8, 24), (3, 23), (1, 17 + tail), (4, 20), (10, 20)], "#a54432")
    poly([(10, 14), (22, 14), (24, 25), (8, 25)], "#982f30")
    rect(12, 15, 5, 8, "#de653e")
    rect(13, 6, 10, 9, "#c34834")
    poly([(13, 8), (10, 3), (16, 7)], "#e0b477")
    poly([(20, 7), (24, 2), (23, 11)], "#e0b477")
    rect(19, 9, 3, 2, "#ffe069")
    rect(18, 13, 5, 1, "#25171e")
    rect(10 + stride, 25, 5, 5, "#6b2831")
    rect(20 - stride, 25, 5, 5, "#6b2831")

```

```

        flame_height = (5, 8, 6, 9)[phase]
        poly([(25, 19), (27, 19 - flame_height), (31, 19), (28, 23)], "#ed742e")
        poly([(27, 19), (28, 15), (30, 20)], "#ffe376")

    elif key == "wolf":
        poly([(4, 17), (10, 13), (22, 15), (27, 20), (20, 25), (7, 24)], "#454153")
        rect(8, 16, 12, 5, "#777181")
        poly([(21, 15), (24, 9), (27, 15), (31, 18), (29, 22), (22, 22)], "#595364")
        rect(27, 16, 2, 1, "#dcb66b")
        poly([(4, 18), (1, 12), (7, 17)], "#646074")
        rect(8 + stride, 23, 3, 7, "#292635")
        rect(20 - stride, 23, 3, 7, "#292635")

    elif key == "golem":
        poly([(7, 13), (13, 10), (23, 13), (27, 26), (6, 26)], "#242432")
        rect(10, 14, 13, 11, "#48424f")
        rect(12, 6, 11, 10, "#514953")
        rect(13, 6, 4, 7, "#74606b")
        rect(18, 10, 4, 2, "#f6984d")
        line(15, 16, 18, 20, "#d57840")
        line(18, 20, 16, 24, "#d57840")
        rect(4, 16, 6, 11, "#5b505b")
        rect(23, 16, 6, 11, "#3b3543")
        rect(9 + stride, 25, 6, 6, "#302c3b")
        rect(20 - stride, 25, 6, 6, "#302c3b")

    else:
        # Призраки, дриады и инквизиторы – отдельные палитры.
        palettes = {
            "wraith": ("#365d77", "#78afc2", "#d9f2eb"),
            "dryad": ("#395b42", "#72a26b", "#dac37a"),
            "inquisitor": ("#593c62", "#98618f", "#efad88"),
        }
        base, light, eyes = palettes.get(
            key, ("#584262", "#a27093", "#e6be78")
        )

        poly([(10, 11), (15, 5), (22, 10), (25, 27),
              (20, 25), (16, 30), (12, 26), (6, 29)], outline)
        poly([(11, 12), (16, 7), (21, 11), (23, 25),
              (18, 24), (15, 28), (12, 24), (8, 26)], base)

```

```

        rect(13, 14, 3, 10, light)
        rect(15, 11, 7, 4, "#1b202b")
        rect(18, 12, 3, 1, eyes)
        line(24, 15, 28, 23, light, 2)

    if key == "dryad":
        line(14, 8, 10, 2, "#8c754f", 2)
        line(20, 9, 24, 2, "#8c754f", 2)
        rect(8, 3, 4, 2, "#83ab62")
        rect(23, 4, 4, 2, "#83ab62")

    if state == AnimationState.HURT:
        p.setCompositionMode(QPainter.CompositionMode.CompositionMode_SourceAtop)
        p.fillRect(0, 0, 32, 32, QColor(255, 231, 210, 70))

    p.end()
    return image

```

Файл: ui/common.py

```

`python
from future import annotations

import hashlib
from dataclasses import fields
from html import escape

from PySide6.QtCore import QPoint, QSize, Qt
from PySide6.QtGui import (
    QColor, QCursor, QIcon, QImage, QPainter, QPen, QPixmap, QPolygon,
)
from PySide6.QtWidgets import (
    QAbstractItemView, QListWidget, QListWidgetItem, QToolTip,
)

from models.item import (
    ACT_LORE, ACT_NAMES, EquipmentSlot, Item,
    PREFIX_BY_ID, Rarity, SUFFIX_BY_ID,

```

```

)

from models.stats import StatBlock

RARITY_COLORS = {
    Rarity.COMMON: "#bdc1cf",
    Rarity.UNCOMMON: "#78d99b",
    Rarity.RARE: "#71b8ff",
    Rarity.LEGENDARY: "#ffd17b",
    Rarity.IMMORTAL: "#d29aff",
    Rarity.MYTHIC: "#ff7faf",
}

RARITY_NAMES = {
    Rarity.COMMON: "Обычный",
    Rarity.UNCOMMON: "Необычный",
    Rarity.RARE: "Редкий",
    Rarity.LEGENDARY: "Легендарный",
    Rarity.IMMORTAL: "Бессмертный",
    Rarity.MYTHIC: "Мифический",
}

SLOT_LABELS = {
    EquipmentSlot.WEAPON: "Оружие",
    EquipmentSlot.OFFHAND: "Левая рука",
    EquipmentSlot.HEAD: "Шлем",
    EquipmentSlot.CHEST: "Нагрудник",
    EquipmentSlot.GLOVES: "Перчатки",
    EquipmentSlot.BOOTS: "Сапоги",
    EquipmentSlot.BELT: "Пояс",
    EquipmentSlot.AMULET: "Амулет",
    EquipmentSlot.RING_1: "Перстень I",
    EquipmentSlot.RING_2: "Перстень II",
}

STAT_NAMES = {
    "strength": "Сила",
    "agility": "Ловкость",

```


"intelligence": "Интеллект",
"vitality": "Живучесть",
"luck": "Удача",
"damage": "Урон",
"armor": "Броня",
"max_hp": "Здоровье",
"mana": "Мана",
"physical_bonus": "Физический урон",
"magical_bonus": "Магический урон",
"hp_bonus": "Максимальное HP",
"crit_chance": "Шанс крита",
"crit_damage": "Сила крита",
"evasion": "Уклонение",
"attack_speed": "Скорость атаки",
"basic_requirement_reduction": "Снижение затрат атаки",
"cooldown_reduction": "Сокращение перезарядки",
"skill_range": "Дальность навыков",
"basic_attack_range": "Дальность атаки",
"cast_speed": "Скорость сотворения",
"fire_bonus": "Огненный урон",
"cold_bonus": "Урон холодом",
"lightning_bonus": "Урон молнией",
"chaos_bonus": "Урон хаосом",
"multistrike": "Повторные атаки",
"projectile_count": "Дополнительные снаряды",
"melee_bonus": "Урон ближнего боя",
"projectile_bonus": "Урон снарядов",
"aoe_damage": "Урон по площади",
"projectile_speed": "Скорость снарядов",
"summon_damage": "Урон призыва",
"fire_res": "Сопротивление огню",
"cold_res": "Сопротивление холоду",
"lightning_res": "Сопротивление молнии",
"chaos_res": "Сопротивление хаосу",
"block_chance": "Шанс блока",

```

"elemental_dodge": "Стихийное уклонение",
"elemental_block": "Стихийный блок",
"hp_regen": "HP в секунду",
"damage_absorption": "Поглощение урона",
"movement_speed": "Скорость передвижения",
"aoe_enhancement": "Радиус области",
"skill_duration": "Длительность навыков",
"hp_per_hit": "HP за попадание",
"hp_per_kill": "HP за убийство",
"life_leech": "Вампиризм",
"skill_heal": "Сила лечения",
"all_skill_level": "Уровни навыков",
"exp_gain": "Получаемый опыт",
"additional_exp": "Дополнительный опыт",
}

PERCENT_STATS = {
"physical_bonus", "magical_bonus", "hp_bonus",
"crit_chance", "crit_damage", "evasion",
"cooldown_reduction", "skill_range",
"fire_bonus", "cold_bonus", "lightning_bonus", "chaos_bonus",
"melee_bonus", "projectile_bonus", "aoe_damage",
"projectile_speed", "summon_damage",
"fire_res", "cold_res", "lightning_res", "chaos_res",
"block_chance", "elemental_dodge", "elemental_block",
"aoe_enhancement", "skill_duration", "life_leech",
"skill_heal", "exp_gain",
}

```

```

def stats_text(stats: StatBlock) -> str:
    result = []
    for definition in fields(stats):
        value = getattr(stats, definition.name)
        if value == 0:
            continue

```

```

    formatted = (

```

```

        f"{value * 100:+.1f}%"
        if definition.name in PERCENT_STATS
    else f"{value:+.2f}".rstrip("0").rstrip(".")
)
result.append(
    f"{STAT_NAMES.get(definition.name, definition.name)}: {formatted}"
)

return "\n".join(result) or "Без дополнительных характеристик"

```

def item_tooltip(item: Item) -> str:

color = RARITY_COLORS[item.rarity]

lore = item.lore or ACT_LORE["crypts"][0]

origin = ACT_NAMES.get(item.origin_act, "Неизвестное происхождение")

```

affixes = []
if item.prefix_id:
    affixes.append("Префикс: " + PREFIX_BY_ID[item.prefix_id].name)
if item.suffix_id:
    affixes.append("Суффикс: " + SUFFIX_BY_ID[item.suffix_id].name)

affix_html = "<br>".join(escape(line) for line in affixes)
stats_html = escape(stats_text(item.stats)).replace("\n", "<br>")

return f"""
<table width="300" cellpadding="5" cellspacing="0">
  <tr><td align="center">
    <span style="color:#a98654;">◆ — F & R — ◆</span>
  </td></tr>
  <tr><td>
    <b style="color:{color};font-size:15px;">
      {escape(item.display_name)}
    </b><br>
    <span style="color:#b8a8bf;">
      {RARITY_NAMES[item.rarity]} · {SLOT_LABELS[item.slot]}
      · Ур. {item.level}
    </span><br>
    <span style="color:#988899;">{escape(origin)}</span>
  </td></tr>
  <tr><td><hr>
    <span style="color:#c6aad8;">{affix_html}</span><br>

```

```

<span style="color:#e9e0d2;">{stats_html}</span>
</td></tr>
<tr><td>
<i style="color:#ac9caf;">«{escape(lore)}»</i>
</td></tr>
<tr><td><hr>
<span style="color:#f0c77e;">♦ Продажа: {item.sell_value:,} золота</span>
</td></tr>
</table>
"""

```

```
def show_item_card(item: Item, widget=None) -> None:
```

```

QToolTip.showText(
QCursor.pos(), item_tooltip(item),
widget, msecShowTime=12000,
)

```

```
_ICON_CACHE: dict[tuple, QIcon] = {}
```

```
def item_icon(item: Item, size: int = 40) -> QIcon:
```

```

digest = hashlib.blake2b(
item.item_id.encode("utf-8"), digest_size=8
).digest()
visual = item.visual_key
key = (visual, item.rarity, digest[:3], size)

```

```

if key in _ICON_CACHE:
    return _ICON_CACHE[key]

image = QImage(32, 32, QImage.Format.Format_ARGB32_Premultiplied)
image.fill(Qt.GlobalColor.transparent)
p = QPainter(image)
p.setRenderHint(QPainter.RenderHint.Antialiasing, False)

def rect(x, y, w, h, color):
    p.fillRect(int(x), int(y), int(w), int(h), QColor(color))

def poly(points, color):
    p.setPen(Qt.PenStyle.NoPen)
    p.setBrush(QColor(color))
    p.drawPolygon(QPolygon([QPoint(x, y) for x, y in points]))

```

```

def line(x1, y1, x2, y2, color, width=1):
    p.setPen(QPen(QColor(color), width))
    p.drawLine(x1, y1, x2, y2)

metal, shine = (
    ("76869e", "d9e6ee"),
    ("8e8192", "e5cce9"),
    ("9b876e", "eed5ae"),
)[digest[0] % 3]
gem = ("71c9ff", "ce88ff", "ff7e9d", "86efb2")[digest[1] % 4]
gold = "d5ab67"
dark = "211b30"
wood = "8f624c"
rarity = QColor(RARITY_COLORS[item.rarity])

rect(1, 1, 30, 30, "#100c17")
for inset, alpha in ((2, 70), (4, 35), (7, 22)):
    color = QColor(rarity)
    color.setAlpha(alpha)
    rect(inset, inset, 32 - inset * 2, 32 - inset * 2, color)

for x, y in ((2, 2), (27, 2), (2, 27), (27, 27)):
    rect(x, y, 3, 1, rarity)
    rect(x, y, 1, 3, rarity)

if visual in ("sword", "greatsword", "sabre", "dagger", "dual"):
    offsets = (-3, 5) if visual == "dual" else (0,)
    for offset in offsets:
        tip_y = 9 if visual == "dagger" else 4
        width = 5 if visual == "greatsword" else 3
        if visual == "sabre":
            poly([(10, 23), (19, 14), (25, 4), (25, 13), (14, 26)], metal)
            line(12, 22, 24, 7, shine)
        else:
            poly([
                (9 + offset, 23), (23 + offset, tip_y),
                (25 + offset, tip_y),
                (12 + width + offset, 25),
            ], metal)
            line(11 + offset, 22, 24 + offset, tip_y + 1, shine)
            line(7 + offset, 20, 16 + offset, 27, gold, 2)

```

```

        line(9 + offset, 25, 5 + offset, 29, wood, 3)
    elif visual in ("staff", "wand", "scythe"):
        line(9, 29, 22, 5, wood, 3)
        line(10, 27, 22, 6, gold)
        if visual == "scythe":
            poly([(18, 6), (28, 5), (30, 10), (25, 8),
                (17, 9), (7, 16), (10, 10)], shine)
        else:
            radius = 5 if visual == "staff" else 3
            poly([(22, 3), (22 + radius, 8), (22, 13),
                (22 - radius, 8)], metal)
            poly([(22, 5), (25, 8), (22, 11), (19, 8)], gem)
            rect(21, 6, 2, 2, "#fff1f6")
    elif visual in ("bow", "crossbow"):
        line(10, 5, 21, 10, wood, 3)
        line(21, 10, 24, 17, wood, 3)
        line(24, 17, 20, 24, wood, 3)
        line(20, 24, 10, 28, wood, 3)
        line(10, 5, 10, 28, "#d0c4ac")
        line(5, 17, 29, 17, shine)
        poly([(29, 17), (25, 14), (25, 20)], metal)
        if visual == "crossbow":
            line(8, 25, 24, 9, wood, 4)
            rect(10, 20, 4, 3, gold)
    elif visual in ("hammer", "mace", "flail"):
        line(8, 29, 21, 9, wood, 3)
        if visual == "hammer":
            poly([(12, 7), (19, 3), (28, 9), (23, 17)], metal)
            line(14, 7, 22, 13, shine, 2)
        elif visual == "mace":
            poly([(17, 5), (23, 4), (28, 9), (24, 16), (17, 14)], metal)
            rect(20, 6, 3, 7, shine)
        else:
            line(20, 10, 26, 14, gold)
            line(26, 14, 23, 20, gold)
            poly([(22, 17), (27, 18), (29, 23), (24, 27), (19, 23)], metal)
            rect(23, 20, 2, 2, shine)
    elif visual in ("shield", "tower", "buckler"):

```

```

    if visual == "buckler":
        poly([(10, 7), (22, 7), (27, 13), (27, 21),
              (21, 27), (11, 27), (5, 20), (5, 13)], metal)
    else:
        bottom = 29
        poly([(6, 7), (16, 3), (26, 7), (24, 23),
              (16, bottom), (8, 23)], metal)
        poly([(8, 8), (16, 5), (23, 8), (21, 21),
              (16, 26), (10, 21)], "#43547b")
        line(16, 8, 16, 24, gold, 2)
        line(10, 14, 22, 14, gold, 2)
        rect(14, 12, 4, 4, gem)

elif visual == "book":
    rect(6, 6, 20, 23, dark)
    rect(8, 7, 17, 20, "#633e69")
    rect(6, 7, 3, 21, gold)
    rect(24, 8, 2, 19, "#d8cbaa")
    rect(11, 10, 11, 1, gold)
    poly([(16, 13), (21, 18), (16, 23), (11, 18)], gem)

elif visual == "skull":
    poly([(10, 5), (22, 5), (27, 11), (25, 21),
          (21, 23), (21, 28), (11, 28), (11, 23),
          (6, 20), (5, 11)], "#c8c2a7")
    rect(9, 12, 6, 6, dark)
    rect(19, 12, 6, 6, dark)
    rect(11, 14, 2, 2, gem)
    rect(21, 14, 2, 2, gem)
    poly([(17, 17), (14, 22), (19, 22)], dark)
    for x in (12, 15, 18):
        rect(x, 25, 1, 3, dark)

elif visual == "orb":
    poly([(11, 5), (21, 5), (27, 11), (27, 21),
          (21, 27), (11, 27), (5, 21), (5, 11)], metal)
    poly([(12, 8), (20, 8), (24, 12), (24, 20),
          (20, 24), (12, 24), (8, 20), (8, 12)], gem)
    rect(10, 10, 6, 3, "#eef4ff")
    rect(9, 13, 2, 5, "#cce7ff")
    rect(12, 28, 10, 2, gold)

```

```

elif visual in ("quiver", "arrows"):
    for x in (10, 15, 20):
        line(x, 5, x - 3, 25, shine)
        poly([(x, 3), (x - 3, 8), (x + 2, 7)], "#a9bfd8")
    if visual == "quiver":
        poly([(7, 13), (22, 16), (19, 29), (5, 26)], wood)
        line(7, 14, 21, 17, gold, 2)
        line(8, 18, 17, 27, "#bc9067")

elif visual in ("plate", "leather", "robe"):
    color = {
        "plate": metal,
        "leather": "#826044",
        "robe": "#70518a",
    }[visual]
    poly([(7, 7), (12, 5), (16, 9), (21, 5),
          (26, 8), (29, 16), (23, 18),
          (25 if visual == "robe" else 22, 29),
          (7 if visual == "robe" else 10, 29),
          (9, 18), (3, 16)], dark)
    poly([(8, 9), (12, 7), (16, 11), (21, 7),
          (24, 10), (26, 15), (21, 16),
          (22, 27), (10, 27), (11, 16), (6, 15)], color)
    rect(12, 13, 3, 11, shine if visual == "plate" else gold)
    rect(10, 24, 12, 2, gold)
    rect(16, 14, 3, 3, gem)

elif visual in ("helm", "hood", "crown"):
    if visual == "hood":
        poly([(6, 25), (8, 10), (16, 3), (24, 10), (27, 25)], "#493758")
        rect(11, 12, 11, 9, dark)
        rect(17, 14, 3, 1, gem)
    elif visual == "crown":
        poly([(6, 24), (4, 8), (11, 14), (16, 4),
              (21, 14), (28, 8), (25, 24)], gold)
        rect(8, 23, 16, 4, metal)
        rect(14, 15, 4, 5, gem)
    else:
        poly([(10, 10), (5, 5), (5, 12), (10, 17)], gold)
        poly([(22, 10), (27, 5), (27, 12), (22, 17)], gold)
        rect(10, 8, 13, 18, metal)
        rect(12, 9, 4, 13, shine)

```



```

    rect(10, 16, 13, 3, dark)
    rect(16, 14, 2, 13, gold)

elif visual == "gloves":
    for x, y in ((5, 9), (18, 5)):
        rect(x, y + 7, 9, 13, metal)
        for finger in range(3):
            rect(x + finger * 3, y + finger % 2, 2, 10, shine)
            rect(x, y + 17, 9, 3, gold)

elif visual == "boots":
    for x, y in ((5, 8), (18, 5)):
        rect(x, y, 8, 18, wood)
        rect(x, y + 15, 11, 6, metal)
        rect(x, y, 8, 3, gold)
        rect(x + 1, y + 4, 2, 9, shine)

elif visual == "belt":
    rect(4, 11, 25, 11, wood)
    rect(4, 11, 25, 2, "#bb9667")
    rect(12, 9, 11, 15, gold)
    rect(14, 11, 7, 11, dark)
    rect(14, 16, 9, 2, shine)

elif visual == "amulet":
    line(7, 4, 16, 15, gold)
    line(25, 4, 16, 15, gold)
    poly([(16, 12), (25, 20), (16, 29), (7, 20)], gold)
    poly([(16, 15), (21, 20), (16, 26), (11, 20)], gem)
    rect(15, 16, 2, 3, "#fff2ff")

else:
    for x, y, w, h in (
        (10, 9, 12, 3), (7, 12, 3, 11),
        (22, 12, 3, 11), (10, 23, 12, 3),
    ):
        rect(x, y, w, h, gold)

    if visual == "signet":
        rect(10, 5, 12, 11, metal)
        rect(15, 7, 2, 7, gem)
        rect(12, 9, 7, 2, gem)

```

```

    else:
        poly([(16, 4), (23, 10), (16, 17), (9, 10)], metal)
        poly([(16, 6), (20, 10), (16, 14), (12, 10)], gem)
        rect(11, 24, 2, 1, dark)
        rect(17, 24, 2, 1, dark)

p.end()

image = image.scaled(
    size * 2, size * 2,
    Qt.AspectRatioMode.IgnoreAspectRatio,
    Qt.TransformationMode.FastTransformation,
)
pixmap = QPixmap.fromImage(image)
pixmap.setDevicePixelRatio(2.0)
icon = QIcon(pixmap)

if len(_ICON_CACHE) >= 1024:
    _ICON_CACHE.clear()
_ICON_CACHE[key] = icon
return icon

```

```
class ItemGrid(QListWidget):
```

```
def init(self, parent=None) -> None:
```

```
    super().init(parent)
```

```
    self._items_by_id: dict[str, Item] = {}
```

```
    self._signature = None
```

```

        self.setViewMode(QListWidget.ViewMode.IconMode)
        self.setResizeMode(QListWidget.ResizeMode.Adjust)
        self.setMovement(QListWidget.Movement.Static)
        self.setSelectionMode(QAbstractItemView.SelectionMode.ExtendedSelection)
        self.setIconSize(QSize(44, 44))
        self.setGridSize(QSize(62, 68))
        self.setSpacing(3)
        self.setUniformItemSizes(True)
        self.setMinimumHeight(110)

        self.itemClicked.connect(self._show_card)

def selected_ids(self) -> tuple[str, ...]:

```

```

    return tuple(
        entry.data(Qt.ItemDataRole.UserRole)
        for entry in self.selectedItems()
    )

def _show_card(self, entry: QListWidgetItem) -> None:
    item = self._items_by_id.get(entry.data(Qt.ItemDataRole.UserRole))
    if item is not None:
        show_item_card(item, self)

def set_items(self, items: tuple[Item, ...]) -> None:
    if self._signature == items:
        return

    selected = set(self.selected_ids())
    scroll = self.verticalScrollBar().value()

    self._signature = items
    self._items_by_id = {item.item_id: item for item in items}
    self.clear()

    for item in items:
        entry = QListWidgetItem(item_icon(item, 44), str(item.level))
        entry.setData(Qt.ItemDataRole.UserRole, item.item_id)
        entry.setToolTip(item_tooltip(item))
        entry.setForeground(QColor(RARITY_COLORS[item.rarity]))
        entry.setTextAlignment(Qt.AlignmentFlag.AlignCenter)
        self.addItem(entry)
        entry.setSelected(item.item_id in selected)

    self.verticalScrollBar().setValue(scroll)

```

```
APP_STYLESHEET = """
```

```

QWidget {
    color:#e6dcc9;
    font-family:"Segoe UI","DejaVu Sans",sans-serif;
    font-size:11px;
}
QLabel { background:transparent; }
QPushButton, QToolButton {

```

```

background:#261c2c;
color:#dec8a5;
border:1px solid #735039;
border-radius:3px;
padding:4px 6px;
}
QPushButton:hover, QToolButton:hover {
background:#3b253b;
border-color:#d8ab62;
}
QPushButton:checked, QToolButton:checked {
background:#532e46;
border-color:#edc77d;
}
QPushButton:disabled, QToolButton:disabled {
background:#201a25;
color:#706674;
border-color:#403343;
}
QListWidget, QGraphicsView {
background:#100e18;
border:1px solid #543b52;
border-radius:3px;
}
QListWidget::item:selected {
background:#49304e;
border:1px solid #deb373;
}
QProgressBar {
background:#19141f;
border:1px solid #56424c;
border-radius:2px;
text-align:center;
min-height:13px;
max-height:16px;

```

```

font-size:10px;
}
QProgressBar::chunk { background:#963f53; }
QTabBar::tab {
background:#241a2b;
color:#bba488;
border:1px solid #51394d;
padding:5px 12px;
}
QTabBar::tab:selected {
background:#493044;
color:#edc784;
}
QToolTip {
background:#170f20;
color:#eee2cc;
border:2px solid #987246;
padding:7px;
}
QDialog, QMessageBox { background:#1c1523; }
QScrollArea { background:transparent; border:none; }
QScrollBar:vertical { background:#18121e; width:9px; }
QScrollBar::handle:vertical {
background:#705063;
min-height:24px;
}
"""
`

```

Файл: ui/gothic_frame.py

```

`python
from future import annotations

from PySide6.QtCore import QPointF, QRectF, Qt, Signal

```

```

from PySide6.QtGui import (
    QColor,
    QLinearGradient,
    QPainter,
    QPainterPath,
    QPen,
    QPolygonF,
)
from PySide6.QtWidgets import QVBoxLayout, QWidget

```

```

class GothicFrame(QWidget):

```

```

    """

```

Прозрачный QWidget с фигурной областью рисования.

```

top_level=True включает перетаскивание окна за заголовок.

```

```

    """

```

```

drag_finished = Signal()

```

```

def __init__(

```

```

    self,

```

```

    title: str,

```

```

    parent=None,

```

```

    *,

```

```

    top_level: bool = False,

```

```

) -> None:

```

```

    super().__init__(parent)

```

```

    self.title = title

```

```

    self.top_level = top_level

```

```

    self._drag_offset = None

```

```

    self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)

```

```

    self.setAutoFillBackground(False)

```

```

    self.body = QVBoxLayout(self)

```

```

    self.body.setContentsMargins(13, 46, 13, 13)

```

```

    self.body.setSpacing(8)

```

```

def _shape(self) -> QPainterPath:

```

```

    w = float(self.width())

```

```

    h = float(self.height())

```

```

c = w / 2

path = QPainterPath(QPointF(1, 35))
path.lineTo(9, 27)
path.lineTo(9, 15)
path.lineTo(19, 24)
path.lineTo(c - 43, 24)
path.lineTo(c - 31, 13)
path.lineTo(c - 19, 20)
path.lineTo(c - 10, 4)
path.lineTo(c, 11)
path.lineTo(c + 10, 4)
path.lineTo(c + 19, 20)
path.lineTo(c + 31, 13)
path.lineTo(c + 43, 24)
path.lineTo(w - 19, 24)
path.lineTo(w - 9, 15)
path.lineTo(w - 9, 27)
path.lineTo(w - 1, 35)
path.lineTo(w - 1, h - 15)
path.lineTo(w - 15, h - 1)
path.lineTo(15, h - 1)
path.lineTo(1, h - 15)
path.closeSubpath()
return path

def _draw_demon(self, painter: QPainter) -> None:
    c = self.width() / 2

    painter.save()
    painter.translate(c, 21)

    # Крылья/рога барельефа.
    painter.setPen(QPen(QColor("#c19550"), 1))
    painter.setBrush(QColor("#51333c"))

    for direction in (-1, 1):
        painter.drawPolygon(QPolygonF([
            QPointF(direction * 5, 1),
            QPointF(direction * 25, -8),
            QPointF(direction * 18, 4),
            QPointF(direction * 31, 8),

```

```

        QPointF(direction * 10, 9),
    ]))

    face = QPainterPath(QPointF(-9, -1))
    face.lineTo(-5, -8)
    face.lineTo(0, -4)
    face.lineTo(5, -8)
    face.lineTo(9, -1)
    face.lineTo(6, 10)
    face.lineTo(0, 14)
    face.lineTo(-6, 10)
    face.closeSubpath()

    gradient = QLinearGradient(-8, -7, 8, 14)
    gradient.setColorAt(0, QColor("#a87f49"))
    gradient.setColorAt(0.5, QColor("#584149"))
    gradient.setColorAt(1, QColor("#221923"))

    painter.setBrush(gradient)
    painter.drawPath(face)

    painter.setPen(QPen(QColor("#d95b69"), 1.8))
    painter.drawLine(QPointF(-5, 2), QPointF(-2, 4))
    painter.drawLine(QPointF(5, 2), QPointF(2, 4))
    painter.setPen(QPen(QColor("#c7a679"), 1))
    painter.drawLine(QPointF(-2, 9), QPointF(0, 11))
    painter.drawLine(QPointF(2, 9), QPointF(0, 11))

    painter.restore()

def paintEvent(self, event) -> None:
    painter = QPainter(self)
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)

    gradient = QLinearGradient(0, 0, 0, self.height())
    gradient.setColorAt(0, QColor(43, 27, 40, 248))
    gradient.setColorAt(0.4, QColor(18, 17, 24, 240))
    gradient.setColorAt(1, QColor(25, 18, 29, 248))

    painter.setBrush(gradient)
    painter.setPen(QPen(QColor("#976d43"), 1.4))
    painter.drawPath(self._shape())

```



```

painter.setPen(QPen(QColor("#572633"), 3))
painter.drawLine(16, 41, self.width() - 16, 41)

painter.setPen(QPen(QColor("#bc9454"), 0.8))
painter.drawLine(18, 43, self.width() - 18, 43)

# Кованые нижние уголки.
painter.setPen(QPen(QColor("#9e7949"), 2))
for x, direction in ((13, 1), (self.width() - 13, -1)):
    y = self.height() - 13
    painter.drawLine(x, y, x + direction * 22, y)
    painter.drawLine(x, y, x, y - 23)
    painter.drawLine(x, y - 9, x + direction * 10, y - 9)

self._draw_demon(painter)

painter.setPen(QColor("#d3b47c"))
painter.drawText(
    QRectF(25, 24, self.width() - 50, 17),
    Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.AlignVCenter,
    self.title,
)

painter.setPen(QColor("#8c6654"))
painter.drawText(
    QRectF(self.width() - 110, 25, 84, 15),
    Qt.AlignmentFlag.AlignRight,
    "R f + x",
)
painter.end()

def mousePressEvent(self, event) -> None:
    if (
        self.top_level
        and event.button() == Qt.MouseButton.LeftButton
        and event.position().y() < 44
    ):
        self._drag_offset = (
            event.globalPosition().toPoint() - self.window().pos()
        )
        event.accept()

```

```

        return
    super().mousePressEvent(event)

def mouseMoveEvent(self, event) -> None:
    if self._drag_offset is not None:
        self.window().move(
            event.globalPosition().toPoint() - self._drag_offset
        )
    event.accept()
    return
    super().mouseMoveEvent(event)

def mouseReleaseEvent(self, event) -> None:
    if self._drag_offset is not None:
        self._drag_offset = None
        self.drag_finished.emit()
    event.accept()
    return
    super().mouseReleaseEvent(event)

```

Файл: ui/detailed_stats_window.py

```

`python
from future import annotations

from PySide6.QtCore import QRectF, Qt
from PySide6.QtGui import QColor, QPainter
from PySide6.QtWidgets import (
    QFrame,
    QHBoxLayout,
    QLabel,
    QScrollArea,
    QToolButton,
    QVBoxLayout,
    QWidget,
)

```

```

from gfx.sprite_loader import SpriteLoader
from models.hero import Hero, SecondaryStats
from models.stats import StatBlock
from ui.gothic_frame import GothicFrame

```

```

class StatsBackground(QWidget):

```

```

    def init(self, parent=None) -> None:

```

```

        super().init(parent)

```

```

        self.sprite = None

```

```

    def paintEvent(self, event) -> None:

```

```

        painter = QPainter(self)

```

```

        painter.fillRect(self.rect(), QColor("#121018"))

```

```

        if self.sprite is not None:

```

```

            painter.setOpacity(0.08)

```

```

            width = min(260, self.width() - 20)

```

```

            rect = QRectF(

```

```

                (self.width() - width) / 2,

```

```

                max(100, (self.height() - width) / 2),

```

```

                width,

```

```

                width,

```

```

            )

```

```

            painter.drawPixmap(

```

```

                rect,

```

```

                self.sprite,

```

```

                QRectF(self.sprite.rect()),

```

```

            )

```

```

        painter.end()

```

```

class DetailedStatsWindow(GothicFrame):

```

```

    def init(self, parent=None) -> None:

```

```

        super().init("", parent, top_level=True)

```

```

        self.setWindowFlags(

```

```

            Qt.WindowType.Tool

```

```

            | Qt.WindowType.FramelessWindowHint

```

```

            | Qt.WindowType.WindowStaysOnTopHint

```

```

        )

```

```

        self.setAttribute(Qt.WidgetAttribute.WA_DeleteOnClose, False)

```

```
self.resize(380, 560)
```

```
self.setMinimumSize(350, 320)
```

```
self.loader = SpriteLoader()  
self._sprite_key = None  
self._rows: dict[str, QLabel] = {}
```

```
self.title_label = QLabel("Detailed Stats", self)  
self.title_label.setAlignment(Qt.AlignmentFlag.AlignCenter)  
self.title_label.setStyleSheet(  
    "color:#e2ba70; font-size:14px; font-weight:bold;"  
)  
self.title_label.setAttribute(  
    Qt.WidgetAttribute.WA_TransparentForMouseEvents
```

```
self.close_button = QPushButton(self)  
self.close_button.setText("x")  
self.close_button.setFixedSize(23, 22)  
self.close_button.clicked.connect(self.close)
```

```
self.hero_label = QLabel()  
self.hero_label.setWordWrap(True)  
self.body.addWidget(self.hero_label)
```

```
self.scroll = QScrollArea()  
self.scroll.setWidgetResizable(True)  
self.scroll.setFrameShape(QFrame.Shape.NoFrame)  
self.body.addWidget(self.scroll, 1)
```

```
self.content = StatsBackground()  
self.content_layout = QVBoxLayout(self.content)  
self.content_layout.setContentsMargins(8, 8, 8, 8)  
self.content_layout.setSpacing(7)  
self.scroll.setWidget(self.content)
```

```
from models.hero import DETAILED_REGISTRY
```

```
for section, rows in DETAILED_REGISTRY.items():  
    label = QLabel(section)  
    label.setStyleSheet(  
        "color:#d5a360; font-size:14px; font-weight:bold;"
```

```

    )
    self.content_layout.addWidget(label)

    line = QFrame()
    line.setFixedHeight(1)
    line.setStyleSheet("background:#745038;")
    self.content_layout.addWidget(line)

    for name, _, _ in rows:
        row = QHBoxLayout()
        title = QLabel(name)
        title.setWordWrap(True)
        title.setStyleSheet("color:#cfc5b8; font-size:10px;")

        value = QLabel()
        value.setAlignment(
            Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter
        )
        value.setMinimumWidth(67)
        value.setStyleSheet("color:#e9d5ab; font-size:11px;")

        row.addWidget(title, 1)
        row.addWidget(value)
        self.content_layout.addLayout(row)
        self._rows[name] = value

    self.content_layout.addStretch()

def resizeEvent(self, event) -> None:
    super().resizeEvent(event)
    self.title_label.setGeometry(45, 24, self.width() - 90, 18)
    self.close_button.move(self.width() - 37, 19)
    self.close_button.raise_()

def refresh(
    self,
    hero: Hero,
    bonuses: StatBlock = StatBlock(),
    *,
    secondary: SecondaryStats | None = None,
) -> None:
    self.hero_label.setText(

```

```

        f"{hero.name} · {hero.definition.name} Lv.{hero.level}"
    )

    sections = hero.detailed_stats(bonuses, secondary=secondary)
    for rows in sections.values():
        for name, value in rows:
            if self._rows[name].text() != value:
                self._rows[name].setText(value)

    key = hero.hero_class.value
    if key != self._sprite_key:
        self._sprite_key = key
        self.content.sprite = self.loader.frames(
            key, "idle", size=256, dpr=self.devicePixelRatioF()
        )[0]
        self.content.update()

def closeEvent(self, event) -> None:
    self.hide()
    event.ignore()

```

Файл: ui/battle_stage.py

```

`python
from future import annotations

import math
from collections import deque
from dataclasses import dataclass

from PySide6.QtCore import (
    QElapsedTimer, QPointF, QRectF, Qt, QTimer,
)
from PySide6.QtGui import (
    QColor, QFont, QLinearGradient, QPainter,
    QPainterPath, QPen, QPolygonF, QRadialGradient,
)

```

```

from PySide6.QtWidgets import QWidget

from engine.combat_manager import (
    CombatFrame, CombatManager, DamageEvent, LootEvent, VfxEvent,
)

from gfx.animations import AnimationController
from gfx.sprite_loader import SpriteLoader
from models.enemy import Act
from ui.common import RARITY_COLORS, RARITY_NAMES

ACT_TITLES = {
    Act.CRYPTS: "Катакомбы Падших",
    Act.FOREST: "Осквернённая Чаща",
    Act.CALDERA: "Пепельная Кальдера",
    Act.CITADEL: "Цитадель Бездны",
}

ELEMENT_COLORS = {
    "physical": "#e5e8ee",
    "fire": "#ff9b49",
    "cold": "#9edfff",
    "lightning": "#ffe4a0",
    "chaos": "#94efb5",
    "magic": "#c991ed",
}

TEXT_COLORS = {
    "physical": "#f0ece4",
    "magical": "#c789f3",
    "heal": "#88e9a4",
    "dodge": "#a9a3b2",
    "barrier": "#98d9fb",
    "block": "#a9d7ff",
}

@dataclass
class FloatingText:
    x: float

```

```
y: float
text: str
color: QColor
age: float = 0.0
lifetime: float = 0.85
```

```
@dataclass
class Particle:
    x: float
    y: float
    vx: float
    vy: float
    color: QColor
    size: float
    age: float = 0.0
    lifetime: float = 0.45
    gravity: float = 65.0
```

```
@dataclass
class Effect:
    kind: str
    x: float
    y: float
    color: QColor
    age: float = 0.0
    lifetime: float = 0.25
```

```
@dataclass
class LootOrb:
    event: LootEvent
    offset: float
    age: float = 0.0
    landed: bool = False
    lifetime: float = 1.45
```

```
@dataclass
class LootNotice:
```


event: LootEvent

age: float = 0.0

lifetime: float = 2.5

class BattleStage(QWidget):

def **init**(

self,

combat: CombatManager,

parent: QWidget | None = None,

*,

drive_combat: bool = True,

) -> None:

super().**init**(parent)

self.combat = combat

self.drive_combat = drive_combat

self.loader = SpriteLoader()

self._frame = combat.frame()

self._animations: dict[str, AnimationController] = {}

self._texts: list[FloatingText] = []

self._particles: list[Particle] = []

self._effects: list[Effect] = []

self._loot: list[LootOrb] = []

self._notices: deque[LootNotice] = deque(maxlen=24)

self._last_simulation_time = self._frame.time

self._visual_time = 0.0

self._barrier_flash = 0.0

self._event_serial = 0

self.setFixedSize(520, 144)

self.setAttribute(Qt.WidgetAttribute.WA_OpaquePaintEvent)

combat.frame_changed.connect(self._receive_frame)

combat.damage_event.connect(self._receive_damage)

combat.vfx_event.connect(self._receive_vfx)

combat.loot_event.connect(self._receive_loot)

self._clock = QElapsedTimer()

self._clock.start()

```

self._timer = QTimer(self)
self._timer.setInterval(16)
self._timer.timeout.connect(self._tick)
self._timer.start()

```

```

def _receive_frame(self, frame: CombatFrame) -> None:

```

```

    self._frame = frame
    ids = {actor.actor_id for actor in frame.actors}

```

```

    for actor_id in tuple(self._animations):
        if actor_id not in ids:
            del self._animations[actor_id]

```

```

    for actor in frame.actors:
        animator = self._animations.setdefault(
            actor.actor_id, AnimationController()
        )
        animator.set_state(actor.animation)

```

```

def _receive_damage(self, event: DamageEvent) -> None:

```

```

    labels = {
        "dodge": "YBOPOT",
        "barrier": "БАРЬЕР",
        "block": "БЛОК",
    }

```

```

    if event.kind in labels:
        text = labels[event.kind]
    elif event.kind == "heal":
        text = f"+{max(1, round(event.amount))}"
    else:
        text = str(max(1, round(event.amount)))

```

```

    color = QColor(
        "#ffe09a" if event.critical
        else TEXT_COLORS.get(event.kind, "#ffffff")
    )

```

```

    self._texts.append(FloatingText(
        event.x + (len(self._texts) % 5 - 2) * 3,
        event.y,

```

```

        text,
        color,
    ))
    if len(self._texts) > 100:
        del self._texts[:-100]

    if event.critical:
        self._burst(event.x, event.y + 8, QColor("#ffda7f"), 14)
    elif event.kind == "block":
        self._burst(event.x, event.y + 8, QColor("#91d9ff"), 9)
        self._effects.append(Effect(
            "shield", event.x, event.y + 7,
            QColor("#9bdfff"), lifetime=0.30,
        ))

def _receive_vfx(self, event: VfxEvent) -> None:
    color = QColor(ELEMENT_COLORS.get(event.element, "#c6b6df"))

    if event.kind == "barrier":
        self._barrier_flash = 0.35
        self._effects.append(Effect(
            "ripple", event.x, event.y,
            QColor("#bcefff"), lifetime=0.35,
        ))
    elif event.kind == "slash":
        self._effects.append(Effect(
            "slash", event.x, event.y,
            color, lifetime=0.15,
        ))
    elif event.kind == "holy":
        self._effects.append(Effect(
            "holy", event.x, event.y,
            QColor("#ffe5a0"), lifetime=0.32,
        ))
    elif event.kind == "impact":
        self._burst(event.x, event.y, color, 5)

    if len(self._effects) > 100:
        del self._effects[:-100]

def _receive_loot(self, event: LootEvent) -> None:
    self._event_serial += 1

```

```

offset = ((self._event_serial % 5) - 2) * 13
self._loot.append(LootOrb(event, offset))
self._notices.append(LootNotice(event))

if len(self._loot) > 32:
    del self._loot[:-32]

def _burst(
    self,
    x: float,
    y: float,
    color: QColor,
    count: int,
) -> None:
    self._event_serial += 1
    phase = self._event_serial * 0.71

    for index in range(count):
        angle = phase + index * math.tau / count
        speed = 25 + (index % 4) * 15
        self._particles.append(Particle(
            x, y,
            math.cos(angle) * speed,
            math.sin(angle) * speed - 16,
            QColor(color),
            1.0 + index % 2,
            lifetime=0.30 + (index % 4) * 0.08,
        ))

    if len(self._particles) > 320:
        del self._particles[:-320]

def _tick(self) -> None:
    dt = min(max(0.0, self._clock.restart() / 1000.0), 0.25)

    if self.drive_combat:
        self.combat.advance(dt)

    simulation_dt = max(
        0.0, self._frame.time - self._last_simulation_time
    )
    self._last_simulation_time = self._frame.time

```

```

self._visual_time += dt
for animation in self._animations.values():
    animation.advance(simulation_dt)

for text in self._texts:
    text.age += dt
self._texts = [
    text for text in self._texts if text.age < text.lifetime
]

for particle in self._particles:
    particle.age += dt
    particle.x += particle.vx * dt
    particle.y += particle.vy * dt
    particle.vy += particle.gravity * dt
self._particles = [
    particle for particle in self._particles
    if particle.age < particle.lifetime
]

for effect in self._effects:
    effect.age += dt
self._effects = [
    effect for effect in self._effects
    if effect.age < effect.lifetime
]

for orb in self._loot:
    orb.age += dt
    if orb.age >= 0.80 and not orb.landed:
        orb.landed = True
        self._burst(
            orb.event.x + orb.offset,
            orb.event.y - 2,
            QColor(RARITY_COLORS[orb.event.item.rarity]),
            10,
        )
self._loot = [orb for orb in self._loot if orb.age < orb.lifetime]

if self._notices:
    self._notices[0].age += dt

```

```

        if self._notices[0].age >= self._notices[0].lifetime:
            self._notices.popleft()

    self._barrier_flash = max(0.0, self._barrier_flash - dt)
    self.update()

@staticmethod
def _polygon(painter: QPainter, points, color) -> None:
    painter.setPen(Qt.PenStyle.NoPen)
    painter.setBrush(QColor(color))
    painter.drawPolygon(QPolygonF([
        QPointF(float(x), float(y)) for x, y in points
    ]))

@staticmethod
def _line(
    painter: QPainter,
    x1: float, y1: float, x2: float, y2: float,
    color, width: float = 1,
) -> None:
    painter.setPen(QPen(QColor(color), width))
    painter.drawLine(QPointF(x1, y1), QPointF(x2, y2))

@staticmethod
def _arch(x: float, y: float, width: float, height: float) -> QPainterPath:
    path = QPainterPath(QPointF(x, y + height))
    path.lineTo(x, y + height * 0.42)
    path.quadTo(x, y + height * 0.15, x + width / 2, y)
    path.quadTo(
        x + width, y + height * 0.15,
        x + width, y + height * 0.42,
    )
    path.lineTo(x + width, y + height)
    path.closeSubpath()
    return path

def _flame(
    self, painter: QPainter,
    x: float, y: float, color: str, seed: float,
) -> None:
    t = self._visual_time
    h = 10 + 3 * math.sin(t * 6 + seed)

```

```

self._polygon(painter, [
    (x - 6, y), (x - 4, y - 8),
    (x - 1, y - h - 5), (x + 2, y - 6),
    (x + 5, y - h), (x + 6, y),
], color)

self._polygon(painter, [
    (x - 3, y), (x, y - h),
    (x + 3, y - 3), (x + 2, y),
], "#d1f7b7" if color == "#58a68b" else "#ffe3a1")

def _draw_background(self, painter: QPainter) -> None:
    t = self._visual_time
    act = self._frame.act

    colors = {
        Act.CRYPTS: ("#15121e", "#252330"),
        Act.FOREST: ("#0b1919", "#213832"),
        Act.CALDERA: ("#3d1520", "#160f1a"),
        Act.CITADEL: ("#201334", "#100d1c"),
    }

    sky = QLinearGradient(0, 0, 0, 144)
    sky.setColorAt(0, QColor(colors[act][0]))
    sky.setColorAt(1, QColor(colors[act][1]))
    painter.fillRect(QRectF(0, 0, 520, 144), sky)

    if act == Act.CRYPTS:
        offset = (t * 2.2) % 126

        for index in range(-1, 6):
            x = index * 126 - offset

            painter.setBrush(QColor("#100f19"))
            painter.setPen(QPen(QColor("#454052"), 4))
            painter.drawPath(self._arch(x + 13, 24, 78, 92))

            painter.setPen(QPen(QColor("#625563"), 1))
            painter.drawPath(self._arch(x + 20, 31, 64, 82))

            painter.fillRect(QRectF(x + 1, 38, 10, 87), QColor("#393342"))
            painter.fillRect(QRectF(x - 2, 35, 16, 5), QColor("#514552"))
            painter.fillRect(QRectF(x - 2, 119, 16, 5), QColor("#514552"))

```

```

        for crack in range(3):
            cx = x + 100 + crack * 6
            cy = 35 + crack * 23
            self._line(painter, cx, cy, cx - 5, cy + 9, "#17131f")
            self._line(painter, cx - 5, cy + 9, cx + 1, cy + 17, "#17131f")

        near = (t * 5) % 156
        for index in range(-1, 5):
            x = index * 156 - near

        painter.save()
        painter.translate(x + 33, 123)
        painter.rotate(-8 if index % 2 else 7)
        painter.setPen(QPen(QColor("#73666f"), 1))
        painter.setBrush(QColor("#48404b"))
        painter.drawRoundedRect(QRectF(-10, -30, 21, 32), 6, 6)
        self._line(painter, 0, -24, 0, -8, "#b2a387")
        self._line(painter, -5, -19, 5, -19, "#b2a387")
        self._line(painter, -4, -8, 4, -11, "#28212e")
        painter.restore()

        for link in range(8):
            painter.setPen(QPen(QColor("#645665"), 1))
            painter.setBrush(Qt.BrushStyle.NoBrush)
            painter.drawEllipse(QRectF(x + 101, 17 + link * 5, 3, 7))

        self._polygon(painter, [
            (x + 91, 84), (x + 110, 84),
            (x + 105, 91), (x + 96, 91),
        ], "#65515d")
        self._flame(painter, x + 101, 83, "#58a68b", index)

    elif act == Act.FOREST:
        offset = (t * 2.7) % 118
        for index in range(-1, 7):
            x = index * 118 - offset
            trunk = QPainterPath(QPointF(x + 28, 128))
            trunk.cubicTo(x + 50, 90, x + 18, 69, x + 38, 18)
            painter.setPen(QPen(QColor("#142724"), 15))
            painter.setBrush(Qt.BrushStyle.NoBrush)
            painter.drawPath(trunk)

```



```

        for branch in range(4):
            by = 42 + branch * 17
            direction = -1 if branch % 2 else 1
            self._line(
                painter, x + 34, by,
                x + 34 + direction * 30, by - 20,
                "#19352d", 5,
            )
            self._line(
                painter, x + 34 + direction * 23, by - 15,
                x + 34 + direction * 37, by - 38,
                "#19352d", 2,
            )

        near = (t * 5.5) % 142
        for index in range(-1, 6):
            x = index * 142 - near

        self._polygon(painter, [
            (x + 4, 132), (x + 13, 113),
            (x + 39, 110), (x + 54, 129),
        ], "#334b42")
        self._line(painter, x + 15, 116, x + 39, 113, "#6f8b55", 3)

        for mushroom in range(3):
            mx = x + 73 + mushroom * 10
            my = 127 - mushroom % 2 * 6
            self._line(painter, mx, my, mx, my - 9, "#70969b", 2)
            painter.setPen(Qt.PenStyle.NoPen)
            painter.setBrush(QColor("#4faeaa"))
            painter.drawEllipse(QRectF(mx - 6, my - 14, 13, 7))
            painter.setBrush(QColor("#b3f2d7"))
            painter.drawEllipse(QRectF(mx - 2, my - 13, 3, 2))

        vine = QPainterPath(QPointF(x + 98, 14))
        vine.cubicTo(x + 78, 30, x + 119, 39, x + 94, 66)
        painter.setPen(QPen(QColor("#3c6650"), 2))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawPath(vine)

    elif act == Act.CALDERA:

```

```
glow = QRadialGradient(QPointF(390, 46), 160)
```

```
glow.setColorAt(0, QColor(216, 65, 35, 100))
```

```
glow.setColorAt(1, QColor(216, 65, 35, 0))
```

```
painter.fillRect(QRectF(0, 0, 520, 144), glow)
```

```
offset = (t * 2.4) % 110
```

```
for index in range(-1, 7):
```

```
    x = index * 110 - offset
```

```
    self._polygon(painter, [
```

```
        (x - 12, 130), (x + 13, 62), (x + 20, 77),
```

```
        (x + 43, 22), (x + 51, 76), (x + 67, 52),
```

```
        (x + 101, 130),
```

```
    ], "#211b2b")
```

```
    self._line(painter, x + 43, 25, x + 35, 92, "#55323d", 2)
```

```
near = (t * 5.0) % 174
```

```
pulse = 0.65 + 0.35 * math.sin(t * 2.2)
```

```
for index in range(-1, 5):
```

```
    x = index * 174 - near
```

```
    self._polygon(painter, [
```

```
        (x + 26, 125), (x + 26, 75), (x + 34, 70),
```

```
        (x + 39, 79), (x + 45, 71), (x + 57, 84),
```

```
        (x + 60, 126),
```

```
    ], "#3b2935")
```

```
    self._line(painter, x + 31, 90, x + 53, 89, "#5c3b42")
```

```
    self._line(painter, x + 30, 108, x + 58, 106, "#5c3b42")
```

```
magma = QColor("#f47c3b")
```

```
magma.setAlphaF(pulse)
```

```
points = [
```

```
    (x + 75, 125), (x + 93, 115),
```

```
    (x + 108, 123), (x + 123, 110),
```

```
    (x + 147, 126),
```

```
]
```

```
for a, b in zip(points, points[1:]):
```

```
    self._line(painter, *a, *b, "#6e302c", 6)
```

```
    self._line(painter, *a, *b, magma, 2)
```

```
else:
```

```
    offset = (t * 1.8) % 136
```

```

for index in range(-1, 6):
    x = index * 136 - offset
    arch = self._arch(x + 15, 24, 78, 95)

    painter.setPen(QPen(QColor("#655072"), 3))
    painter.setBrush(QColor("#312143"))
    painter.drawPath(arch)

    painter.save()
    painter.setClipPath(arch)

    for pane in range(5):
        color = QColor(105 + pane * 15, 57, 160 + pane * 12)
        color.setAlpha(120 + int(35 * math.sin(t * 2 + pane + index)))
        self._polygon(painter, [
            (x + 12 + pane * 17, 27),
            (x + 55, 67),
            (x + 13 + pane * 17, 121),
        ], color)
    painter.restore()

    self._line(painter, x + 54, 30, x + 54, 119, "#b18b99", 1)
    self._line(painter, x + 17, 73, x + 91, 73, "#b18b99", 1)

    oy = 64 + math.sin(t * 1.2 + index) * 5
    self._polygon(painter, [
        (x + 111, oy - 23), (x + 121, oy - 7),
        (x + 120, oy + 23), (x + 110, oy + 31),
        (x + 104, oy + 16), (x + 104, oy - 10),
    ], "#100d19")
    self._line(painter, x + 111, oy - 15, x + 111, oy + 20, "#7c5eae")

    near = (t * 4.5) % 192
    for index in range(-1, 4):
        x = index * 192 - near
        painter.fillRect(QRectF(x + 8, 61, 17, 74), QColor("#423149"))
        painter.fillRect(QRectF(x + 5, 59, 23, 7), QColor("#6c5165"))
        for notch in range(5):
            self._line(
                painter, x + 12, 72 + notch * 11,
                x + 21, 76 + notch * 11,

```

```

        "#a77b94",
    )

    painter.setPen(QPen(QColor("#9864c2"), 1))
    painter.setBrush(Qt.BrushStyle.NoBrush)
    painter.drawEllipse(QRectF(x + 72, 116, 73, 14))
    painter.drawEllipse(QRectF(x + 83, 119, 51, 8))
    painter.setPen(QColor("#c9a1e8"))
    painter.drawText(QPointF(x + 89, 125), "f R x")

    ground = QLinearGradient(0, 109, 0, 144)
    ground.setColorAt(0, QColor(10, 9, 16, 0))
    ground.setColorAt(1, QColor(9, 7, 14, 225))
    painter.fillRect(QRectF(0, 105, 520, 39), ground)

    self._draw_atmosphere(painter)

def _draw_atmosphere(self, painter: QPainter) -> None:
    t = self._visual_time
    act = self._frame.act

    if act == Act.CRYPTS:
        for layer in range(3):
            x = ((t * (6 + layer * 2)) + layer * 180) % 750 - 220
            fog = QRadialGradient(QPointF(x + 160, 126), 175)
            fog.setColorAt(0, QColor(161, 159, 170, 22))
            fog.setColorAt(1, QColor(161, 159, 170, 0))
            painter.fillRect(QRectF(x, 107, 370, 35), fog)
        return

    for index in range(23):
        x = (index * 73.7 + math.sin(t * 0.3 + index) * 15) % 520

        if act == Act.CALDERA:
            y = 143 - ((index * 19 + t * (9 + index % 7)) % 125)
            color = QColor("#ffae68" if index % 3 else "#82767d")
            radius = 0.8 + index % 2 * 0.5
        else:
            y = 35 + (index * 37 % 100) + math.sin(t * 0.6 + index) * 7
            color = QColor("#82dec5" if act == Act.FOREST else "#c193fa")
            radius = 0.7 + index % 3 * 0.3

```

```

        color.setAlpha(60 + int(60 * (0.5 + 0.5 * math.sin(t + index))))
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(color)
        painter.drawEllipse(QPointF(x, y), radius, radius)

def _draw_barrier(self, painter: QPainter) -> None:
    if self._frame.barrier_max <= 0:
        return

    ratio = max(0.0, min(
        1.0, self._frame.barrier / self._frame.barrier_max
    ))
    if ratio <= 0 and self._barrier_flash <= 0:
        return

    t = self._visual_time
    pulse = 0.8 + 0.2 * math.sin(t * 3)
    flash = min(1.0, self._barrier_flash / 0.35)
    strength = min(1.0, ratio * pulse + flash * 0.7)

    rect = QRectF(36, 39, 173, 95)

    glow = QRadialGradient(QPointF(123, 104), 98)
    glow.setColorAt(0.0, QColor(112, 152, 225, 5))
    glow.setColorAt(0.65, QColor(111, 142, 232, int(13 * strength)))
    glow.setColorAt(1.0, QColor(121, 196, 244, int(85 * strength)))

    painter.setBrush(glow)
    painter.setPen(Qt.PenStyle.NoPen)
    painter.drawEllipse(rect)

    for width, alpha in ((6, 16), (3, 35), (1, 150)):
        color = QColor("#a3d7ff")
        color.setAlpha(int(alpha * strength))
        painter.setPen(QPen(color, width))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawEllipse(rect)

    for index in range(17):
        angle = math.pi + index * math.pi / 16
        x = 122.5 + math.cos(angle) * 86.5
        y = 86.5 + math.sin(angle) * 47.5

```

```

        color = QColor("#e3c58b" if index % 3 == 0 else "#98dbf0")
        color.setAlpha(int(160 * strength))
        painter.save()
        painter.translate(x, y)
        painter.rotate(math.degrees(angle) + 90)
        self._line(painter, 0, -3, 0, 3, color, 1)
        self._line(painter, -2, -1, 2, 1, color, 1)
        if index % 2:
            self._line(painter, -2, 2, 0, 0, color, 1)
        painter.restore()

    if flash > 0:
        color = QColor("#e9fbff")
        color.setAlpha(int(180 * flash))
        painter.setPen(QPen(color, 1.2))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        inset = (1 - flash) * 10
        painter.drawEllipse(rect.adjusted(inset, inset / 2, -inset, -inset / 2))

def _draw_actors(self, painter: QPainter) -> None:
    dpr = self.devicePixelRatioF()

    for actor in sorted(self._frame.actors, key=lambda entry: entry.y):
        size = 48 if actor.boss else 36
        animator = self._animations.setdefault(
            actor.actor_id, AnimationController()
        )
        animator.set_state(actor.animation)

        frames = self.loader.frames(
            actor.sprite_key, actor.animation,
            size=size, dpr=dpr,
        )
        pixmap = frames[animator.frame_index % len(frames)]

        painter.save()
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QColor(0, 0, 0, 75))
        painter.drawEllipse(QRectF(actor.x - 12, actor.y - 3, 24, 5))

        painter.translate(actor.x, actor.y)

```

```

        if actor.enemy:
            painter.scale(-1, 1)

        painter.drawPixmap(QPointF(-size / 2, -size), pixmap)
        painter.restore()

        if actor.hp <= 0:
            continue

        width = 45 if actor.boss else 24
        left = actor.x - width / 2
        top = actor.y - size - 5
        ratio = max(0.0, min(1.0, actor.hp / actor.max_hp))

        painter.fillRect(QRectF(left, top, width, 3), QColor("#211727"))
        painter.fillRect(
            QRectF(left, top, width * ratio, 3),
            QColor("#c45e62" if actor.enemy else "#79c19b"),
        )

        if actor.elite:
            painter.setPen(QColor("#efc776"))
            painter.drawText(QPointF(actor.x - 4, top - 2), "◆")

        if actor.boss:
            painter.fillRect(
                QRectF(left, top + 5, width, 2), QColor("#3c2531")
            )
            painter.fillRect(
                QRectF(left, top + 5, width * actor.rage, 2),
                QColor("#ef9653"),
            )

def _draw_projectiles(self, painter: QPainter) -> None:
    for projectile in self._frame.projectiles:
        painter.save()
        painter.translate(projectile.x, projectile.y)
        painter.rotate(math.degrees(projectile.angle))

        if projectile.kind == "arrow":
            self._line(painter, -19, 0, -4, 0, QColor(225, 238, 255, 80), 2)
            self._line(painter, -10, 0, 8, 0, "#d7d3bc", 1.5)

```

```

        self._polygon(painter, [(10, 0), (5, -3), (5, 3)], "#e2edf2")
        self._line(painter, -10, 0, -14, -3, "#95b5cb")
        self._line(painter, -10, 0, -14, 3, "#95b5cb")

    else:
        fire = projectile.kind == "fireball"
        color = QColor("#ff9347" if fire else "#84e7ad")
        tail = QColor(color)
        tail.setAlpha(85)

        self._polygon(painter, [
            (-22, 0), (-3, -5), (2, 0), (-3, 5),
        ], tail)

        gradient = QRadialGradient(QPointF(0, 0), 9)
        gradient.setColorAt(0, QColor("#fff1c1" if fire else "#e9ffe8"))
        gradient.setColorAt(0.35, color)
        gradient.setColorAt(1, QColor(color.red(), color.green(), color.blue(), 0))
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(gradient)
        painter.drawEllipse(QRectF(-9, -9, 18, 18))

        if projectile.kind == "necrotic":
            painter.fillRect(QRectF(-3, -3, 6, 5), QColor("#d7efd2"))
            painter.fillRect(QRectF(0, -2, 2, 2), QColor("#31554c"))
            painter.fillRect(QRectF(-2, 2, 4, 2), QColor("#a8d6b7"))

        painter.restore()

def _draw_effects(self, painter: QPainter) -> None:
    for effect in self._effects:
        progress = effect.age / effect.lifetime
        color = QColor(effect.color)
        color.setAlphaF(max(0.0, 1 - progress))

        painter.save()
        painter.translate(effect.x, effect.y)

        if effect.kind == "slash":
            painter.rotate(-25 + progress * 65)
            path = QPainterPath()
            path.moveTo(-18, 15)

```



```

        path.cubicTo(-6, -20, 20, -24, 31, -1)
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.setPen(QPen(color, 5 * (1 - progress) + 1))
        painter.drawPath(path)

    bright = QColor("#ffffff")
    bright.setAlphaF(max(0.0, 1 - progress))
    painter.setPen(QPen(bright, 1))
    painter.drawPath(path)

    elif effect.kind == "holy":
        gradient = QLinearGradient(0, -100, 0, 20)
        gradient.setColorAt(0, QColor(255, 230, 154, 0))
        gradient.setColorAt(0.55, color)
        gradient.setColorAt(1, QColor(255, 235, 181, 0))
        painter.fillRect(QRectF(-9, -100, 18, 120), gradient)
        self._line(painter, 0, -85, 0, 10, color, 3)
        self._line(painter, -15, -14, 15, -14, color, 2)

    elif effect.kind == "shield":
        painter.setPen(QPen(color, 1.5))
        painter.setBrush(QColor(110, 184, 243, int(55 * (1 - progress))))
        painter.drawPolygon(QPolygonF([
            QPointF(-10, -9), QPointF(0, -13), QPointF(10, -9),
            QPointF(8, 5), QPointF(0, 12), QPointF(-8, 5),
        ]))
        self._line(painter, 0, -7, 0, 6, color)
        self._line(painter, -5, -2, 5, -2, color)

    elif effect.kind == "ripple":
        radius = 5 + progress * 25
        painter.setPen(QPen(color, 1))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawEllipse(
            QRectF(-radius, -radius * 0.7, radius * 2, radius * 1.4)
        )

    painter.restore()

    for particle in self._particles:
        color = QColor(particle.color)
        color.setAlphaF(max(0.0, 1 - particle.age / particle.lifetime))

```

```

        painter.fillRect(
            QRectF(particle.x, particle.y, particle.size, particle.size),
            color,
        )

def _draw_loot(self, painter: QPainter) -> None:
    for orb in self._loot:
        t = min(1.0, orb.age / 0.80)
        x = orb.event.x + orb.offset * t
        y = orb.event.y - 9 - 39 * math.sin(math.pi * t)

        fade = (
            1.0 if orb.age < 0.9
            else max(0.0, (orb.lifetime - orb.age) / (orb.lifetime - 0.9))
        )
        color = QColor(RARITY_COLORS[orb.event.item.rarity])

        painter.save()
        painter.setOpacity(fade)

        glow = QRadialGradient(QPointF(x, y), 14)
        glow.setColorAt(0, color)
        glow.setColorAt(0.4, QColor(color.red(), color.green(), color.blue(), 100))
        glow.setColorAt(1, QColor(color.red(), color.green(), color.blue(), 0))

        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(glow)
        painter.drawEllipse(QRectF(x - 14, y - 14, 28, 28))

        self._polygon(painter, [
            (x, y - 5), (x + 5, y), (x, y + 5), (x - 5, y),
        ], color)
        painter.fillRect(QRectF(x - 1, y - 3, 2, 3), QColor("#fff5dc"))
        painter.restore()

def _draw_texts(self, painter: QPainter) -> None:
    painter.setFont(QFont("Sans Serif", 8, QFont.Weight.Bold))

    for text in self._texts:
        alpha = max(0.0, 1 - text.age / text.lifetime)
        color = QColor(text.color)
        color.setAlphaF(alpha)

```

```

        y = text.y - text.age * 27
        rect = QRectF(text.x - 48, y - 10, 96, 20)

        painter.setPen(QColor(0, 0, 0, int(180 * alpha)))
        painter.drawText(
            rect.translated(1, 1),
            Qt.AlignmentFlag.AlignCenter, text.text,
        )
        painter.setPen(color)
        painter.drawText(rect, Qt.AlignmentFlag.AlignCenter, text.text)

def _draw_notice(self, painter: QPainter) -> None:
    if not self._notices:
        return

    notice = self._notices[0]
    event = notice.event
    color = QColor(RARITY_COLORS[event.item.rarity])

    age = notice.age
    alpha = min(1.0, age / 0.15, (notice.lifetime - age) / 0.30)
    alpha = max(0.0, alpha)

    painter.save()
    painter.setOpacity(alpha)
    rect = QRectF(230, 25, 281, 31)

    path = QPainterPath(QPointF(rect.left() + 7, rect.top()))
    path.lineTo(rect.right() - 7, rect.top())
    path.lineTo(rect.right(), rect.top() + 7)
    path.lineTo(rect.right(), rect.bottom() - 7)
    path.lineTo(rect.right() - 7, rect.bottom())
    path.lineTo(rect.left() + 7, rect.bottom())
    path.lineTo(rect.left(), rect.bottom() - 7)
    path.lineTo(rect.left(), rect.top() + 7)
    path.closeSubpath()

    painter.setBrush(QColor(23, 14, 31, 235))
    painter.setPen(QPen(color.darker(135), 1))
    painter.drawPath(path)

```

```

painter.setFont(QFont("Sans Serif", 8, QFont.Weight.Bold))
title = (
    f"♦ {RARITY_NAMES[event.item.rarity]}: "
    f"{event.item.display_name}"
)
title = painter.fontMetrics().elidedText(
    title, Qt.TextElideMode.ElideRight, 265
)

painter.setPen(color)
painter.drawText(QPointF(238, 37), title)

painter.setFont(QFont("Sans Serif", 7))
painter.setPen(QColor("#c2b3c7"))
message = (
    "Отправлено в сундук"
    if event.stored
    else f"Сундук полон · продано за {event.sale_gold:,}"
)
painter.drawText(QPointF(238, 49), message)
painter.restore()

def _draw_hud(self, painter: QPainter) -> None:
    painter.fillRect(QRectF(0, 0, 520, 23), QColor(11, 8, 17, 215))
    painter.setFont(QFont("Sans Serif", 8))
    painter.setPen(QColor("#ded0bd"))

    title = f"{ACT_TITLES[self._frame.act]} · {self._frame.wave}"
    painter.drawText(QPointF(8, 15), title)

    if self._frame.boss_remaining is not None:
        painter.setPen(QColor("#efc675"))
        painter.drawText(
            QRectF(350, 2, 160, 18),
            Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter,
            f"BOCC · {self._frame.boss_remaining:04.1f} c",
        )

    elif self._frame.phase == "defeat":
        painter.setPen(QColor("#e88e98"))
        painter.drawText(QPointF(380, 15), "Отступление")

    elif self._frame.phase == "between":
        painter.drawText(QPointF(379, 15), "Волна отражена")

```

```

painter.fillRect(QRectF(8, 138, 504, 3), QColor("#342437"))
painter.fillRect(
    QRectF(8, 138, 504 * self._frame.progress, 3),
    QColor("#bd9258"),
)

painter.setPen(QPen(QColor("#765143"), 1))
painter.setBrush(Qt.BrushStyle.NoBrush)
painter.drawRect(QRectF(0.5, 0.5, 519, 143))

def paint_scene(self, painter: QPainter) -> None:
    self._draw_background(painter)
    self._draw_barrier(painter)
    self._draw_actors(painter)
    self._draw_projectiles(painter)
    self._draw_loot(painter)
    self._draw_effects(painter)
    self._draw_texts(painter)
    self._draw_hud(painter)
    self._draw_notice(painter)

def paintEvent(self, event) -> None:
    painter = QPainter(self)
    painter.fillRect(self.rect(), QColor("#121018"))
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)
    painter.setRenderHint(
        QPainter.RenderHint.SmoothPixmapTransform, False
    )
    painter.scale(self.width() / 520.0, self.height() / 144.0)
    painter.setClipRect(QRectF(0, 0, 520, 144))

    self.paint_scene(painter)
    painter.end()

```

Файл: ui/actions.py

```

`python
from future import annotations

```

```

import time
from dataclasses import replace
from typing import Callable

from PySide6.QtCore import QObject, Signal

import config as C
from engine.combat_manager import CombatManager
from engine.cube_synth import CubeSynth
from engine.runes_tree import RunesTree
from engine.save_manager import SaveError, SaveManager
from engine.state import GameData, GameState
from models.hero import Hero, HeroBuild, PRIMARY_STAT_KEYS, default_build
from models.item import EquipmentSlot, RARITY_ORDER

```

```

class UiActions(QObject):

```

```

    error = Signal(str)

```

```

    notice = Signal(str)

```

```

def __init__(
    self,
    state: GameState,
    saves: SaveManager,
    combat: CombatManager,
    parent=None,
) -> None:
    super().__init__(parent)
    self.state = state
    self.saves = saves
    self.combat = combat
    self.runes = RunesTree()
    self.cube = CubeSynth(self.runes)

def run(self, operation: Callable) -> bool:
    """Граница обработки пользовательских ошибок."""
    try:
        operation()
        return True
    except (ValueError, SaveError, RuntimeError) as exc:
        self.error.emit(str(exc))
        return False

```

```
def _transaction(self, transform: Callable[[GameData], GameData]) -> GameData:
```

```
    self.state.assert_owner_thread()
```

```
    if self.combat.running:
```

```
        self.combat.flush()
```

```
    before = self.state.data
```

```
    after = transform(before)
```

```
    after = replace(
```

```
        after,
```

```
        last_active_timestamp=max(
```

```
            time.time(),
```

```
            before.last_active_timestamp,
```

```
        ),
```

```
    )
```

```
    self.saves.write(after)
```

```
    self.state.commit(before, after)
```

```
    return after
```

```
def move_items(self, ids: tuple[str, ...], to_backpack: bool) -> None:
```

```
    selected = set(ids)
```

```
    if not selected:
```

```
        raise ValueError("Сначала выберите предметы")
```

```
    def transform(data: GameData) -> GameData:
```

```
        source = data.stash if to_backpack else data.backpack
```

```
        target = data.backpack if to_backpack else data.stash
```

```
        capacity = (
```

```
            C.DEFAULT_BACKPACK_CAPACITY
```

```
            if to_backpack else data.stash_capacity
```

```
        )
```

```
        available = {item.item_id for item in source}
```

```
        if not selected <= available:
```

```
            raise ValueError("Часть выбранных предметов уже перемещена")
```

```
        moving = tuple(item for item in source if item.item_id in selected)
```

```
        if len(target) + len(moving) > capacity:
```

```
            raise ValueError("В целевом контейнере недостаточно места")
```

```

        remaining = tuple(
            item for item in source if item.item_id not in selected
        )

        if to_backpack:
            return replace(data, stash=remaining, backpack=target + moving)
        return replace(data, backpack=remaining, stash=target + moving)

    self._transaction(transform)
    self.notice.emit(f"Перемещено предметов: {len(selected)}")

def move_all(self, to_backpack: bool) -> None:
    data = self.state.data
    source = data.stash if to_backpack else data.backpack
    target = data.backpack if to_backpack else data.stash
    capacity = (
        C.DEFAULT_BACKPACK_CAPACITY
        if to_backpack else data.stash_capacity
    )
    free = max(0, capacity - len(target))

    ids = tuple(item.item_id for item in source[:free])
    if not ids:
        raise ValueError("Нет предметов для переноса или контейнер заполнен")

    self.move_items(ids, to_backpack)

def sort_stash(self) -> None:
    self._transaction(lambda data: replace(
        data,
        stash=tuple(sorted(
            data.stash,
            key=lambda item: (
                -RARITY_ORDER.index(item.rarity),
                -item.level,
                item.slot.value,
                item.item_id,
            )),
        )),
    ))
    self.notice.emit("Тайник отсортирован по редкости и уровню")

```



```

@staticmethod
def _build(data: GameData, hero_id: str) -> HeroBuild:
    if hero_id not in {hero.hero_id for hero in data.party}:
        raise ValueError("Герой больше не входит в активный отряд")

    return next(
        (build for build in data.hero_builds if build.hero_id == hero_id),
        default_build(hero_id),
    )

@staticmethod
def _replace_build(data: GameData, build: HeroBuild):
    return tuple(
        entry for entry in data.hero_builds
        if entry.hero_id != build.hero_id
    ) + (build,)

def equip(self, hero_id: str, item_id: str) -> None:
    def transform(data: GameData) -> GameData:
        item = next(
            (entry for entry in data.backpack if entry.item_id == item_id),
            None,
        )
        if item is None:
            raise ValueError("Предмет отсутствует в рюкзаке")

        build = self._build(data, hero_id)
        updated, previous = build.equip(item)

        backpack = tuple(
            entry for entry in data.backpack if entry.item_id != item_id
        )
        if previous is not None:
            backpack += (previous,)

    return replace(
        data,
        backpack=backpack,
        hero_builds=self._replace_build(data, updated),
    )

    self._transaction(transform)

```

```

        self.notice.emit("Экипировка обновлена")

def unequip(self, hero_id: str, slot: EquipmentSlot) -> None:
    def transform(data: GameData) -> GameData:
        build = self._build(data, hero_id)
        item = next(
            (entry for entry in build.equipment if entry.slot == slot),
            None,
        )
        if item is None:
            raise ValueError("Слот пуст")
        if len(data.backpack) >= C.DEFAULT_BACKPACK_CAPACITY:
            raise ValueError("Рюкзак заполнен")

        updated = replace(
            build,
            equipment=tuple(
                entry for entry in build.equipment if entry.slot != slot
            ),
        )

    return replace(
        data,
        backpack=data.backpack + (item,),
        hero_builds=self._replace_build(data, updated),
    )

self._transaction(transform)
self.notice.emit("Предмет снят в рюкзак")

def buy_rune(self, node_id: str) -> None:
    self._transaction(
        lambda data: self.runes.calculate_purchase(data, node_id)
    )
    self.notice.emit(f"Открыта руна: {self.runes.nodes[node_id].name}")

def synthesize(self, ids: tuple[str, ...]) -> None:
    result_holder = []

    def transform(data: GameData) -> GameData:
        result = self.cube.calculate(data, ids)
        result_holder.append(result)

```

```

        return result.after
    )

    self._transaction(transform)
    item = result_holder[0].item
    self.notice.emit(
        f"Синтез: {item.display_name} · {item.rarity.value} · Lv.{item.level}"
    )

def allocate_stat(self, hero_id: str, stat_key: str) -> None:
    if stat_key not in PRIMARY_STAT_KEYS:
        raise ValueError("Неизвестная характеристика")

    def transform(data: GameData) -> GameData:
        build = self._build(data, hero_id)
        snapshot = next(
            hero for hero in data.party
            if hero.hero_id == hero_id
        )
        hero = Hero.from_state(snapshot, build)

        if hero.free_stat_points <= 0:
            raise ValueError("Нет свободных очков характеристик")

        allocation = list(build.allocated_stats)
        allocation[PRIMARY_STAT_KEYS.index(stat_key)] += 1

        updated = replace(
            build,
            allocated_stats=tuple(allocation),
        )
        return replace(
            data,
            hero_builds=self._replace_build(data, updated),
        )

    self._transaction(transform)
    self.notice.emit("Очко характеристики распределено")

```

Файл: ui/panels/hero_panel.py

```

`python
from future import annotations

from dataclasses import replace

from PySide6.QtCore import QSize, Qt, Signal
from PySide6.QtGui import QIcon
from PySide6.QtWidgets import (
    QApplication,
    QGridLayout,
    QHBoxLayout,
    QLabel,
    QProgressBar,
    QPushButton,
    QToolButton,
    QVBoxLayout,
)

import config as C
from gfx.sprite_loader import SpriteLoader
from models.hero import PRIMARY_STAT_KEYS, total_exp_for_level
from models.item import EquipmentSlot
from ui.actions import UiActions
from ui.common import (
    ItemGrid, RARITY_COLORS, SLOT_LABELS,
    item_icon, item_tooltip, show_item_card,
)
from ui.detailed_stats_window import DetailedStatsWindow
from ui.gothic_frame import GothicFrame

ATTRIBUTE_LABELS = {
    "strength": "СИЛА · STR",
    "agility": "ЛОВКОСТЬ · AGI",
    "intelligence": "ИНТЕЛЛЕКТ · INT",
    "vitality": "ЖИВУЧЕСТЬ · VIT",
    "luck": "УДАЧА · LUK",
}

```

```
class HeroPanel(GothicFrame):
```

```
    navigate = Signal(str)
```

```
    def __init__(self, actions: UiActions, parent=None) -> None:
```

```
        super().__init__("HERO · ГЕРОЙ И ОТРЯД", parent)
```

```
        self.actions = actions
```

```
        self.active_id = actions.state.data.party[0].hero_id
```

```
        self.loader = SpriteLoader()
```

```
        self._selector_signature = None
```

```
        self._last_frame_time = -1.0
```

```
        self._equipment = {}
```

```
        self._member = None
```

```
        self.details_window = DetailedStatsWindow(self)
```

```
        self.selector = QHBoxLayout()
```

```
        self.body.addLayout(self.selector)
```

```
        top = QHBoxLayout()
```

```
        self.portrait = QLabel()
```

```
        self.portrait.setFixedSize(72, 80)
```

```
        self.portrait.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
        top.addWidget(self.portrait)
```

```
        summary = QVBoxLayout()
```

```
        self.name = QLabel()
```

```
        self.name.setWordWrap(True)
```

```
        summary.addWidget(self.name)
```

```
        self.hp = QProgressBar()
```

```
        self.mp = QProgressBar()
```

```
        self.xp = QProgressBar()
```

```
        for bar in (self.hp, self.mp, self.xp):
```

```
            bar.setRange(0, 1000)
```

```
            summary.addWidget(bar)
```

```
        self.mp.setStyleSheet("QProgressBar::chunk { background:#4b70ad; }")
```

```
        self.xp.setStyleSheet("QProgressBar::chunk { background:#a58242; }")
```

```
        top.addLayout(summary, 1)
```

```
        self.body.addLayout(top)
```

```

self.points = QLabel()
self.body.addWidget(self.points)

attributes = QGridLayout()
self.attribute_values = {}
self.attribute_buttons = {}

for row, key in enumerate(PRIMARY_STAT_KEYS):
    value = QLabel()
    value.setAlignment(Qt.AlignmentFlag.AlignRight)

    plus = QPushButton("+")
    plus.setFixedSize(26, 24)
    plus.clicked.connect(
        lambda checked=False, stat=key:
            self.actions.run(
                lambda: self.actions.allocate_stat(self.active_id, stat)
            )
    )

    attributes.addWidget(QLabel(ATTRIBUTE_LABELS[key]), row, 0)
    attributes.addWidget(value, row, 1)
    attributes.addWidget(plus, row, 2)
    self.attribute_values[key] = value
    self.attribute_buttons[key] = plus

self.body.addLayout(attributes)

detailed_button = QPushButton(
    "ПОДРОБНАЯ СТАТИСТИКА / DETAILED STATS"
)
detailed_button.setMinimumHeight(32)
detailed_button.clicked.connect(self._open_details)
self.body.addWidget(detailed_button)

self.secondary = QLabel()
self.secondary.setWordWrap(True)
self.body.addWidget(self.secondary)

self.body.addWidget(QLabel("ЭКИПИРОВКА • ПКМ: снять предмет"))
equipment = QGridLayout()
equipment.setSpacing(4)

```

```

self.slots = {}

order = (
    EquipmentSlot.HEAD, EquipmentSlot.AMULET,
    EquipmentSlot.WEAPON, EquipmentSlot.OFFHAND,
    EquipmentSlot.CHEST, EquipmentSlot.GLOVES,
    EquipmentSlot.BELT, EquipmentSlot.BOOTS,
    EquipmentSlot.RING_1, EquipmentSlot.RING_2,
)

for index, slot in enumerate(order):
    button = QPushButton()
    button.setToolButtonStyle(Qt.ToolButtonStyle.ToolButtonTextBesideIcon)
    button.setIconSize(QSize(30, 30))
    button.setMinimumHeight(37)
    button.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)

    button.clicked.connect(
        lambda checked=False, selected=slot: self._inspect_slot(selected)
    )
    button.customContextMenuRequested.connect(
        lambda point, selected=slot:
            self.actions.run(
                lambda: self.actions.unequip(self.active_id, selected)
            )
    )
    equipment.addWidget(button, index // 2, index % 2)
    self.slots[slot] = button

self.body.addLayout(equipment)

self.skills = QLabel()
self.skills.setWordWrap(True)
self.body.addWidget(self.skills)

self.bag_title = QLabel()
self.body.addWidget(self.bag_title)

self.bag = ItemGrid()
self.bag.setMinimumHeight(140)
self.bag.itemDoubleClicked.connect(self._equip)
self.body.addWidget(self.bag, 1)

```

```

controls = QHBoxLayout()
equip = QPushButton("Надеть выбранное")
store = QPushButton("В сундук")

equip.clicked.connect(lambda: self._equip())
store.clicked.connect(
    lambda: self.actions.run(
        lambda: self.actions.move_items(self.bag.selected_ids(), False)
    )
)

controls.addWidget(equip)
controls.addWidget(store)
self.body.addLayout(controls)

dock = QHBoxLayout()
for caption, page in (
    ("Сундук", "stash"), ("Куб", "cube"), ("Руны", "runes")
):
    button = QPushButton(caption)
    button.clicked.connect(
        lambda checked=False, target=page: self.navigate.emit(target)
    )
    dock.addWidget(button)
self.body.addLayout(dock)

self.actions.state.state_changed.connect(self.refresh)
self.actions.combat.frame_changed.connect(self._combat_frame)
self.refresh()

def _select(self, hero_id: str) -> None:
    self.active_id = hero_id
    self.refresh()

def _inspect_slot(self, slot: EquipmentSlot) -> None:
    item = self._equipment.get(slot)
    if item is not None:
        show_item_card(item, self.slots[slot])

def _equip(self, entry=None) -> None:
    ids = self.bag.selected_ids()

```



```

    if entry is not None:
        ids = (entry.data(Qt.ItemDataRole.UserRole),)
        if len(ids) != 1:
            self.actions.error.emit("Выберите один предмет в рюкзаке")
            return
        self.actions.run(lambda: self.actions.equip(self.active_id, ids[0]))

    @staticmethod
    def _set_bar(bar, value: float, maximum: float, title: str) -> None:
        ratio = value / maximum if maximum > 0 else 0.0
        bar.setValue(round(max(0.0, min(1.0, ratio)) * 100))
        bar.setFormat(f"{title} {value:,.0f}/{maximum:,.0f}")

    def _refresh_selector(self, members) -> None:
        signature = tuple(
            (m.hero.hero_id, m.hero.hero_class, m.hero.level, m.hero.name)
            for m in members
        )

        if signature != self._selector_signature:
            self._selector_signature = signature
            while self.selector.count():
                widget = self.selector.takeAt(0).widget()
                if widget:
                    widget.deleteLater()

            for member in members:
                hero = member.hero
                button = QPushButton()
                button.setProperty("hero_id", hero.hero_id)
                button.setCheckable(True)
                button.setIcon(QIcon(self.loader.frames(
                    hero.hero_class.value, "idle",
                    size=32, dpr=self.devicePixelRatioF(),
                )[0]))
                button.setIconSize(QSize(32, 32))
                button.setToolTip(
                    f"{hero.name} · {hero.definition.name} Lv.{hero.level}"
                )
                button.clicked.connect(
                    lambda checked=False, hero_id=hero.hero_id:
                        self._select(hero_id)

```

```

    )
    self.selector.addWidget(button)

    for index in range(self.selector.count()):
        button = self.selector.itemAt(index).widget()
        button.setChecked(button.property("hero_id") == self.active_id)

def refresh(self, *_args) -> None:
    data = self.actions.state.data
    members = self.actions.runes.squad(data)

    if self.active_id not in {m.hero.hero_id for m in members}:
        self.active_id = members[0].hero.hero_id

    self._refresh_selector(members)
    self._member = next(
        m for m in members if m.hero.hero_id == self.active_id
    )
    hero = self._member.hero
    secondary = self._member.stats
    bonuses = self.actions.runes.effects(data.unlocked_runes).stats
    primary = hero.primary_stats(bonuses)

    self.name.setText(
        f"<b>{hero.definition.name} Lv.{hero.level}</b><br>{hero.name}"
    )
    self.portrait.setPixmap(self.loader.frames(
        hero.hero_class.value, "idle",
        size=70, dpr=self.devicePixelRatioF(),
    )[0])

    start = total_exp_for_level(hero.level)
    end = total_exp_for_level(hero.level + 1)
    self.xp.setValue(round((hero.total_exp - start) / (end - start) * 1000))
    self.xp.setFormat(f"XP · до уровня {hero.exp_to_next_level:,}")

    self.points.setText(
        f"<b>Свободные очки: {hero.free_stat_points}</b>"
        f" · распределено {sum(hero.allocated_stats)}"
    )

    for index, key in enumerate(PRIMARY_STAT_KEYS):

```

```

        self.attribute_values[key].setText(f"{getattr(primary, key):,.1f}")
        self.attribute_values[key].setToolTip(
            f"Вложено очков: {hero.allocated_stats[index]}"
        )
        self.attribute_buttons[key].setEnabled(hero.free_stat_points > 0)

    self.secondary.setText(
        f"Урон: {secondary.physical_damage:,.1f} физ. / "
        f"{secondary.magical_damage:,.1f} маг.<br>"
        f"Броня: {secondary.armor:,.1f} · Блок: {secondary.block_chance:.1%}<br>"
        f"Крит: {secondary.crit_chance:.1%} ×{secondary.crit_multiplier:.2f}"
        f" · Вампиризм: {secondary.lifesteal:.1%}"
    )

    self._equipment = {item.slot: item for item in hero.equipment}
    for slot, button in self.slots.items():
        item = self._equipment.get(slot)
        button.setText(SLOT_LABELS[slot])
        button.setEnabled(item is not None)
        if item is None:
            button.setIcon(QIcon())
            button.setToolTip("Пустой слот")
            button.setStyleSheet("")
        else:
            button.setIcon(item_icon(item, 30))
            button.setToolTip(item_tooltip(item))
            button.setStyleSheet(
                f"QPushButton {{ border-color:{RARITY_COLORS[item.rarity]}; }}"
            )

    self.skills.setText(
        "<b>Навыки:</b> " + " · ".join(skill.name for skill in hero.skills)
    )
    self.bag_title.setText(
        f"РЮКЗАК · {len(data.backpack)}/{C.DEFAULT_BACKPACK_CAPACITY}"
    )
    self.bag.set_items(data.backpack)

    snapshot = next(h for h in data.party if h.hero_id == self.active_id)
    self._set_bar(
        self.hp,
        secondary.max_hp * snapshot.hp / snapshot.max_hp,

```

```

        secondary.max_hp, "HP",
    )
    self._set_bar(self.mp, secondary.max_mana, secondary.max_mana, "MP")
    self._combat_frame(self.actions.combat.frame(), force=True)
    self._refresh_details()

def _open_details(self) -> None:
    self._refresh_details()
    window = self.details_window

    screen = self.screen() or QApplication.primaryScreen()
    if screen:
        area = screen.availableGeometry()
        x = self.window().geometry().right() + 6
        y = self.window().y()
        x = max(area.left(), min(x, area.right() - window.width() + 1))
        y = max(area.top(), min(y, area.bottom() - window.height() + 1))
        window.move(x, y)

    window.show()
    window.raise_()

def _refresh_details(self) -> None:
    if self._member is None:
        return

    secondary = self._member.stats
    runtime = next(
        (
            h for h in self.actions.combat.heroes
            if h.hero.hero_id == self.active_id
        ),
        None,
    )

    # Базовые значения берём из актуальной модели, а не из устаревшего
    # runtime-снимка между покупкой предмета и следующим combat tick.
    if runtime is not None:
        now = self.actions.combat.time
        damage_bonus = runtime.buff("damage", now)
        if (
            "berserker" in self.actions.runes.effects(

```

```

        self.actions.state.data.unlocked_runes
    ).keystones
    and runtime.hp < runtime.stats.max_hp * 0.40
):
    damage_bonus = (1 + damage_bonus) * 1.8 - 1

    secondary = replace(
        secondary,
        physical_damage=secondary.physical_damage * (1 + damage_bonus),
        magical_damage=secondary.magical_damage * (1 + damage_bonus),
        evasion=min(
            0.95, secondary.evasion + runtime.buff("evasion", now)
        ),
        damage_absorption=(
            secondary.damage_absorption
            + (runtime.shield if runtime.shield_until > now else 0.0)
        ),
    )

    self.details_window.refresh(
        self._member.hero,
        secondary=secondary,
    )

def _combat_frame(self, frame, force: bool = False) -> None:
    if not force and not self.isVisible() and not self.details_window.isVisible():
        return
    if not force and frame.time - self._last_frame_time < 0.10:
        return

    self._last_frame_time = frame.time
    actor = next(
        (
            actor for actor in frame.actors
            if not actor.enemy and actor.actor_id == self.active_id
        ),
        None,
    )
    if actor:
        self._set_bar(self.hp, actor.hp, actor.max_hp, "HP")
        self._set_bar(self.mp, actor mana, actor.max_mana, "MP")

```

```

    if self.details_window.isVisible():
        self._refresh_details()

def hideEvent(self, event) -> None:
    self.details_window.hide()
    super().hideEvent(event)

```

Файл: ui/panels/stash_panel.py

```

`python
from future import annotations

import math

from PySide6.QtCore import Qt
from PySide6.QtWidgets import (
    QHBoxLayout,
    QLabel,
    QPushButton,
    QTabBar,
)

from ui.actions import UiActions
from ui.common import ItemGrid
from ui.gothic_frame import GothicFrame

PAGE_SIZE = 40

class StashPanel(GothicFrame):
    def __init__(self, actions: UiActions, parent=None) -> None:
        super().__init__("STASH · СУНДУК БЕЗДНЫ", parent)
        self.actions = actions

        self.counter = QLabel()
        self.body.addWidget(self.counter)

        self.tabs = QTabBar()
        self.tabs.setExpanding(False)

```

```
self.tabs.currentChanged.connect(self._show_page)
```

```
self.body.addWidget(self.tabs)
```

```
self.grid = ItemGrid()
```

```
self.grid.setMinimumHeight(250)
```

```
self.grid.itemDoubleClicked.connect(self._take_one)
```

```
self.body.addWidget(self.grid, 1)
```

```
hint = QLabel()
```

```
    "Наведение / клик – карточка предмета.\n"
```

```
    "Двойной клик – забрать в рюкзак. Ctrl/Shift – выбор."
```

```
)
```

```
hint.setWordWrap(True)
```

```
self.body.addWidget(hint)
```

```
selected = QPushButton("Забрать выбранное")
```

```
selected.clicked.connect(
```

```
    lambda: self.actions.run(
```

```
        lambda: self.actions.move_items(
```

```
            self.grid.selected_ids(), True
```

```
        )
```

```
    )
```

```
)
```

```
self.body.addWidget(selected)
```

```
row = QHBoxLayout()
```

```
take_all = QPushButton("Забрать всё")
```

```
store_all = QPushButton("Сохранить всё")
```

```
take_all.setToolTip(
```

```
    "Перенести предметы из всех вкладок в рюкзак, пока есть место"
```

```
)
```

```
store_all.setToolTip(
```

```
    "Перенести рюкзак в сундук, пока есть место"
```

```
)
```

```
take_all.clicked.connect(
```

```
    lambda: self.actions.run(
```

```
        lambda: self.actions.move_all(True)
```

```
    )
```

```
)
```

```
store_all.clicked.connect(
```

```

        lambda: self.actions.run(
            lambda: self.actions.move_all(False)
        )
    )

    row.addWidget(take_all)
    row.addWidget(store_all)
    self.body.addLayout(row)

    sort = QPushButton("Сортировка · редкость / уровень")
    sort.clicked.connect(
        lambda: self.actions.run(self.actions.sort_stash)
    )
    self.body.addWidget(sort)

    self.actions.state.state_changed.connect(self.refresh)
    self.refresh()

def _take_one(self, entry) -> None:
    item_id = entry.data(Qt.ItemDataRole.UserRole)
    self.actions.run(
        lambda: self.actions.move_items((item_id,), True)
    )

def refresh(self, *_args) -> None:
    data = self.actions.state.data
    self.counter.setText(
        f"Сундук: {len(data.stash)}/{data.stash_capacity}"
        f" · Рюкзак: {len(data.backpack)}"
    )

    pages = max(1, math.ceil(data.stash_capacity / PAGE_SIZE))
    current = max(0, self.tabs.currentIndex())

    self.tabs.blockSignals(True)
    while self.tabs.count() > pages:
        self.tabs.removeTab(self.tabs.count() - 1)
    while self.tabs.count() < pages:
        self.tabs.addTab(str(self.tabs.count() + 1))
    self.tabs.setCurrentIndex(min(current, pages - 1))
    self.tabs.blockSignals(False)

```



```

        self._show_page(self.tabs.currentIndex())
    def _show_page(self, index: int) -> None:
        index = max(0, index)
        start = index * PAGE_SIZE
        self.grid.set_items(
            self.actions.state.data.stash[start:start + PAGE_SIZE]
        )

```

Файл: ui/panels/cube_panel.py

```

`python
from future import annotations

import math

from PySide6.QtCore import QPointF, QRectF, Qt
from PySide6.QtGui import QColor, QPainter, QPen
from PySide6.QtWidgets import (
    QDialog,
    QDialogButtonBox,
    QGridLayout,
    QHBoxLayout,
    QLabel,
    QPushButton,
    QToolButton,
    QVBoxLayout,
    QWidget,
)

from models.item import Rarity
from ui.actions import UiActions
from ui.common import ItemGrid, RARITY_NAMES, item_icon, item_tooltip
from ui.gothic_frame import GothicFrame

class TransmutationCircle(QWidget):
    def __init__(self, parent=None) -> None:

```

```
super().init(parent)
```

```
self.setFixedHeight(102)
```

```
def paintEvent(self, event) -> None:
    painter = QPainter(self)
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)

    cx = self.width() / 2
    cy = self.height() / 2

    painter.setBrush(QColor(42, 21, 50, 170))
    painter.setPen(QPen(QColor("#9b4dca"), 1.3))

    for radius in (28, 40, 47):
        painter.drawEllipse(QPointF(cx, cy), radius, radius)

    painter.setPen(QPen(QColor("#c8963e"), 1))
    points = [
        QPointF(
            cx + math.cos(index * math.tau / 6) * 40,
            cy + math.sin(index * math.tau / 6) * 40,
        )
        for index in range(6)
    ]

    for index in range(6):
        painter.drawLine(points[index], points[(index + 2) % 6])

    painter.setPen(QColor("#d2afd9"))
    painter.drawText(
        QRectF(cx - 25, cy - 14, 50, 28),
        Qt.AlignmentFlag.AlignCenter,
        "F x R",
    )
    painter.end()
```

```
class CubePanel(GothicFrame):
```

```
def init(self, actions: UiActions, parent=None) -> None:
```

```
super().init("CUBE · КУБ БЕЗДНЫ", parent)
```

```
self.actions = actions
```

```
self.selected_ids: tuple[str, ...] = ()
```

```
self.body.addWidget(TransmutationCircle())
```

```
self.summary = QLabel("Поместите 4-12 предметов одного ранга")
```

```
self.summary.setWordWrap(True)
```

```
self.body.addWidget(self.summary)
```

```
layout = QGridLayout()
```

```
self.slots = []
```

```
for index in range(12):
```

```
    button = QToolButton()
```

```
    button.setText("+")
```

```
    button.setMinimumSize(42, 42)
```

```
    button.clicked.connect(
```

```
        lambda checked=False, slot_index=index:
```

```
            self._slot_clicked(slot_index)
```

```
    )
```

```
    layout.addWidget(button, index // 4, index % 4)
```

```
    self.slots.append(button)
```

```
self.body.addLayout(layout)
```

```
self.odds = QLabel()
```

```
self.odds.setWordWrap(True)
```

```
self.odds.setMinimumHeight(80)
```

```
self.body.addWidget(self.odds)
```

```
choose = QPushButton("Выбрать ингредиенты из тайника")
```

```
choose.clicked.connect(self._choose)
```

```
self.body.addWidget(choose)
```

```
row = QHBoxLayout()
```

```
auto = QPushButton("Автозаполнение")
```

```
clear = QPushButton("Очистить")
```

```
auto.clicked.connect(self._autofill)
```

```
clear.clicked.connect(self._clear)
```

```
row.addWidget(auto)
```

```
row.addWidget(clear)
```

```
self.body.addLayout(row)
```

```

self.synthesize = QPushButton("◆ СИНТЕЗ ◆")
self.synthesize.setMinimumHeight(38)
self.synthesize.clicked.connect(self._synthesize)
self.body.addWidget(self.synthesize)

explanation = QLabel(
    "Ингредиенты остаются в тайнике до подтверждения.\n"
    "Синтез необратимо расходует выбранные предметы."
)
explanation.setWordWrap(True)
self.body.addWidget(explanation)
self.body.addStretch()

self.actions.state.state_changed.connect(self.refresh)
self.refresh()

def _slot_clicked(self, index: int) -> None:
    if index < len(self.selected_ids):
        self.selected_ids = tuple(
            item_id for position, item_id in enumerate(self.selected_ids)
            if position != index
        )
        self.refresh()
    else:
        self._choose()

def _choose(self) -> None:
    dialog = QDialog(self)
    dialog.setWindowTitle("Ингредиенты Куба")
    dialog.resize(540, 430)

    layout = QVBoxLayout(dialog)
    layout.addWidget(QLabel(
        "Выберите 4-12 предметов. Ctrl/Shift – множественный выбор."
    ))

    grid = ItemGrid()
    grid.set_items(self.actions.state.data.stash)

    selected = set(self.selected_ids)
    for index in range(grid.count()):
        entry = grid.item(index)

```

```

        entry.setSelected(
            entry.data(Qt.ItemDataRole.UserRole) in selected
        )

    layout.addWidget(grid, 1)

    buttons = QDialogButtonBox(
        QDialogButtonBox.StandardButton.Ok
        | QDialogButtonBox.StandardButton.Cancel
    )
    buttons.accepted.connect(dialog.accept)
    buttons.rejected.connect(dialog.reject)
    layout.addWidget(buttons)

    if dialog.exec() == QDialog.DialogCode.Accepted:
        ids = grid.selected_ids()
        if len(ids) > 12:
            self.actions.error.emit("В Куб помещается не более 12 предметов")
            return
        self.selected_ids = ids
        self.refresh()

def _autofill(self) -> None:
    self.selected_ids = self.actions.cube.autofill(
        self.actions.state.data.stash
    )
    self.refresh()
    if not self.selected_ids:
        self.actions.notice.emit("Подходящая группа ингредиентов не найдена")

def _clear(self) -> None:
    self.selected_ids = ()
    self.refresh()

def _synthesize(self) -> None:
    ids = self.selected_ids
    if self.actions.run(lambda: self.actions.synthesize(ids)):
        self.selected_ids = ()
        self.refresh()

def refresh(self, *_args) -> None:
    from PySide6.QtGui import QIcon

```

```

data = self.actions.state.data
by_id = {item.item_id: item for item in data.stash}

self.selected_ids = tuple(
    item_id for item_id in self.selected_ids if item_id in by_id
)
ingredients = tuple(by_id[item_id] for item_id in self.selected_ids)

for index, button in enumerate(self.slots):
    if index < len(ingredients):
        item = ingredients[index]
        button.setIcon(item_icon(item))
        button.setText("")
        button.setToolTip(item_tooltip(item) + "<br>Нажать: убрать из Куба")
    else:
        button.setIcon(QIcon())
        button.setText("+")
        button.setToolTip("Выбрать ингредиенты")

self.summary.setText(f"Ингредиенты: {len(ingredients)}/12")

try:
    bonus = self.actions.runes.effects(
        data.unlocked_runes
    ).synthesis_bonus
    odds = self.actions.cube.probabilities(ingredients, bonus)
except ValueError as exc:
    self.odds.setText(str(exc))
    self.synthesize.setEnabled(False)
    return

lines = [
    f"{RARITY_NAMES[rarity]}: {chance:.1%}"
    for rarity, chance in odds.items()
]

for rarity in (Rarity.LEGENDARY, Rarity.IMMORTAL):
    if rarity not in odds:
        lines.append(f"{RARITY_NAMES[rarity]}: 0.0%")

self.odds.setText("\n".join(lines))

```

```
self.synthesize.setEnabled(True)
```

Файл: ui/panels/runes_panel.py

```
`python
from future import annotations

from html import escape

from PySide6.QtCore import QPointF, Qt, Signal
from PySide6.QtGui import QColor, QBrush, QPainter, QPen
from PySide6.QtWidgets import (
    QGraphicsEllipseItem,
    QGraphicsScene,
    QGraphicsView,
    QHBoxLayout,
    QLabel,
    QPushButton,
)

from engine.runes_tree import RuneNode, Sector
from ui.actions import UiActions
from ui.common import stats_text
from ui.gothic_frame import GothicFrame

SECTOR_COLORS = {
    Sector.WAR: "#d86b55",
    Sector.ETHER: "#639cd9",
    Sector.VITALITY: "#70b780",
    Sector.GREED: "#d3af56",
    Sector.CHRONOMANCY: "#ac79db",
    None: "#ead8a5",
}

def effect_description(node: RuneNode) -> str:
    effect = node.effects
```

```
lines = []
```

```
stat_lines = stats_text(effect.stats)
if stat_lines != "Без дополнительных характеристик":
    lines.append(stat_lines)

for value, caption in (
    (effect.gold_bonus, "Золото"),
    (effect.sale_bonus, "Цена продажи"),
    (effect.drop_bonus, "Шанс предмета"),
    (effect.synthesis_bonus, "Критический синтез"),
    (effect.active_speed_bonus, "Активная скорость"),
):
    if value:
        lines.append(f"{caption}: +{value:.1%}")

if effect.cap_steps:
    lines.append("Следующая ступень потолка оффлайна")
if effect.efficiency_steps:
    lines.append("Следующая ступень эффективности оффлайна")
if effect.keystones:
    lines.append("Великая руна: " + ", ".join(effect.keystones))

return "\n".join(lines) or "Центр созвездия"
```

```
class RuneView(QGraphicsView):
```

```
    node_selected = Signal(str)
```

```
    node_activated = Signal(str)
```

```
    def __init__(self, scene: QGraphicsScene, parent=None) -> None:
        super().__init__(scene, parent)
        self.setRenderHint(QPainter.RenderHint.Antialiasing)
        self.setDragMode(QGraphicsView.DragMode.ScrollHandDrag)
        self.setTransformationAnchor(
            QGraphicsView.ViewportAnchor.AnchorUnderMouse
        )
        self.setResizeAnchor(QGraphicsView.ViewportAnchor.AnchorViewCenter)
        self.setBackgroundBrush(QColor("#100e19"))
        self._press_position = None

    def wheelEvent(self, event) -> None:
        factor = 1.18 if event.angleDelta().y() > 0 else 1 / 1.18
```



```

        current = self.transform().m11()
        target = max(0.22, min(2.8, current * factor))
        self.scale(target / current, target / current)
        event.accept()

def mousePressEvent(self, event) -> None:
    self._press_position = event.position()
    super().mousePressEvent(event)

def mouseReleaseEvent(self, event) -> None:
    if (
        self._press_position is not None
        and (event.position() - self._press_position).manhattanLength() < 5
    ):
        item = self.itemAt(event.position().toPoint())
        if item is not None and isinstance(item.data(0), str):
            self.node_selected.emit(item.data(0))

    self._press_position = None
    super().mouseReleaseEvent(event)

def mouseDoubleClickEvent(self, event) -> None:
    item = self.itemAt(event.position().toPoint())
    if item is not None and isinstance(item.data(0), str):
        self.node_activated.emit(item.data(0))
        event.accept()
    return
    super().mouseDoubleClickEvent(event)

def fit_tree(self) -> None:
    self.resetTransform()
    self.fitInView(
        self.sceneRect(),
        Qt.AspectRatioMode.KeepAspectRatio,
    )

```

```

class RunesPanel(GothicFrame):
    def init(self, actions: UiActions, parent=None) -> None:
        super().init("RUNES · СОЗВЕЗДИЯ БЕЗДНЫ", parent)
        self.actions = actions
        self.selected_id = "nexus"

```

```

self.header = QLabel()
self.body.addWidget(self.header)

self.scene = QGraphicsScene(self)
self.scene.setSceneRect(-760, -760, 1520, 1520)
self.view = RuneView(self.scene)
self.view.setMinimumHeight(290)
self.body.addWidget(self.view, 1)

self._nodes: dict[str, QGraphicsEllipseItem] = {}
self._edges = []

tree = self.actions.runes

for parent_id, child_id in tree.edges:
    parent = tree.nodes[parent_id]
    child = tree.nodes[child_id]

    line = self.scene.addLine(
        *parent.position,
        *child.position,
        QPen(QColor("#3f334c"), 2),
    )
    line.setZValue(-1)
    self._edges.append((parent_id, child_id, line))

for node in tree.nodes.values():
    radius = 14 if node.keystone or node.node_id == "nexus" else 9

    item = QGraphicsEllipseItem(
        -radius, -radius, radius * 2, radius * 2
    )
    item.setPos(QPointF(*node.position))
    item.setData(0, node.node_id)
    item.setAcceptHoverEvents(True)
    item.setZValue(1)

    description = escape(effect_description(node)).replace("\n", "<br>")
    item.setToolTip(
        f"<b>{escape(node.name)}</b><br>"
        f"{description}<hr>Цена: {node.cost:,} золота"
    )

```

```

        self.scene.addItem(item)
        self._nodes[node.node_id] = item

    self.details = QLabel()
    self.details.setWordWrap(True)
    self.details.setMinimumHeight(65)
    self.body.addWidget(self.details)

    row = QHBoxLayout()
    self.buy = QPushButton("Купить руну")
    self.buy.clicked.connect(self._buy)

    fit = QPushButton("Показать всё")
    fit.clicked.connect(self.view.fit_tree)

    row.addWidget(self.buy)
    row.addWidget(fit)
    row.addWidget(QLabel("Колесо: зум · ЛКМ: перемещение"))
    self.body.addLayout(row)

    self.view.node_selected.connect(self._select)
    self.view.node_activated.connect(self._activate)
    self.actions.state.state_changed.connect(self.refresh)

    self.refresh()

def _select(self, node_id: str) -> None:
    self.selected_id = node_id
    self.refresh()

def _activate(self, node_id: str) -> None:
    # Двойной щелчок выбирает узел; покупка остаётся явной кнопкой.
    self._select(node_id)
    self.buy.setFocus()

def _buy(self) -> None:
    self.actions.run(
        lambda: self.actions.buy_rune(self.selected_id)
    )

def refresh(self, *_args) -> None:

```

```

data = self.actions.state.data
tree = self.actions.runes
owned = set(data.unlocked_runes) | {"nexus"}

self.header.setText(
    f"Открыто {len(data.unlocked_runes)}/150  · "
    f"Оффлайн {data.offline_cap_seconds // 3600} ч · "
    f"Эффективность {data.offline_efficiency:.0%}"
)

```

```

for node_id, item in self._nodes.items():
    node = tree.nodes[node_id]
    unlocked = node_id in owned
    connected = set(node.prerequisites) <= owned
    affordable = data.gold >= node.cost
    color = QColor(SECTOR_COLORS[node.sector])

```

```

    if unlocked:
        item.setBrush(QBrush(color))
        item.setPen(QPen(color.lighter(155), 2.5))
    elif connected:
        item.setBrush(QBrush(color.darker(280)))
        item.setPen(QPen(
            color if affordable else QColor("#8b776c"),
            2,
        ))
    else:
        item.setBrush(QBrush(QColor("#201b2b")))
        item.setPen(QPen(QColor("#494050"), 1))

```

```

    if node_id == self.selected_id:
        item.setPen(QPen(QColor("#fff1c4"), 3.5))

```

```

for parent_id, child_id, line in self._edges:
    if parent_id in owned and child_id in owned:
        color = QColor("#ae8c58")
    elif parent_id in owned:
        color = QColor("#735675")
    else:
        color = QColor("#342c40")
    line.setPen(QPen(color, 2))

```

```

node = tree.nodes[self.selected_id]
connected = set(node.prerequisites) <= owned
purchased = self.selected_id in owned

status = (
    "Открыта"
    if purchased else
    "Сначала откройте предыдущую руну"
    if not connected else
    "Недостаточно золота"
    if data.gold < node.cost else
    "Доступна для покупки"
)

self.details.setText(
    f"{node.name} · {node.cost:,} золота · {status}\n"
    f"{effect_description(node)}"
)

self.buy.setEnabled(
    not purchased and connected and data.gold >= node.cost
)

self.buy.setText(
    "Открыта" if purchased else f"Купить · {node.cost:,}"
)

```

Файл: ui/main_window.py

```

python
from future import annotations

import sys

from PySide6.QtCore import (
    QEvent,
    QPoint,
    QRectF,
    QSettings,
    Qt,

```

```

    QTimer,
    Signal,
)
from PySide6.QtGui import QColor, QPainter, QPainterPath, QPen
from PySide6.QtWidgets import (
    QApplication,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QScrollArea,
    QStackedWidget,
    QToolButton,
    QVBoxLayout,
    QWidget,
)

import config as C
from engine.combat_manager import CombatManager
from engine.save_manager import SaveError, SaveManager
from engine.state import GameState
from ui.actions import UiActions
from ui.battle_stage import BattleStage
from ui.common import APP_STYLESHEET
from ui.gothic_frame import GothicFrame
from ui.panels.cube_panel import CubePanel
from ui.panels.hero_panel import HeroPanel
from ui.panels.runes_panel import RunesPanel
from ui.panels.stash_panel import StashPanel

class CompactBattleStage(BattleStage):
    def init(self, combat, parent=None) -> None:
        super().init(combat, parent)
        self.setFixedSize(520, 108)

class PanelDrawer(GothicFrame):
    closed = Signal()

```

```

def __init__(self, owner: QWidget) -> None:
    super().__init__("ПАНЕЛЬ БЕЗДНЫ", owner, top_level=True)

    self.setWindowFlags(
        Qt.WindowType.Tool
        | Qt.WindowType.FramelessWindowHint
        | Qt.WindowType.WindowStaysOnTopHint
    )
    self.setAttribute(Qt.WidgetAttribute.WA_DeleteOnClose, False)

    # Внутри находится уже оформленная GothicFrame-панель.
    self.stack = QStackedWidget()
    self.body.addWidget(self.stack)

    self.close_button = QPushButton(self)
    self.close_button.setText("x")
    self.close_button.setFixedSize(25, 22)
    self.close_button.setToolTip("Заккрыть панель – бой продолжится")
    self.close_button.clicked.connect(self.close)

    self._pages = {}

def add_panel(
    self,
    page: str,
    panel: QWidget,
    minimum_height: int,
) -> None:
    panel.setMinimumHeight(minimum_height)

    scroll = QScrollArea()
    scroll.setWidgetResizable(True)
    scroll.setFrameShape(QScrollArea.Shape.NoFrame)
    scroll.setWidget(panel)

    self.stack.addWidget(scroll)
    self._pages[page] = scroll

def select(self, page: str) -> None:
    self.stack.setCurrentWidget(self._pages[page])

def resizeEvent(self, event) -> None:

```

```

        super().resizeEvent(event)
        self.close_button.move(self.width() - 39, 17)
        self.close_button.raise_()

def closeEvent(self, event) -> None:
    self.hide()
    self.closed.emit()
    event.ignore()

```

```
class MainWindow(QWidget):
```

```
    WIDTH = 540
```

```
    HEIGHT = 168
```

```

    PANEL_SIZES = {
        "hero": (430, 720),
        "stash": (420, 550),
        "cube": (390, 640),
        "runes": (790, 650),
    }

    PANEL_TITLES = {
        "hero": "ГЕРОЙ",
        "stash": "СУНДУК",
        "cube": "КУБ СИНТЕЗА",
        "runes": "ДРЕВО РУН",
    }

    def __init__(
        self,
        state: GameState,
        saves: SaveManager,
        parent=None,
    ) -> None:
        super().__init__(parent)

        self.setWindowFlags(
            Qt.WindowType.Window
            | Qt.WindowType.FramelessWindowHint
            | Qt.WindowType.WindowStaysOnTopHint
        )

        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        self.setWindowTitle(C.APP_NAME)

```



```

self.setStyleSheet(APP_STYLESHEET)
self.setFixedSize(self.WIDTH, self.HEIGHT)

self.state = state
self.saves = saves
self.settings = QSettings()
self.combat = CombatManager(state, saves=saves, parent=self)
self.actions = UiActions(state, saves, self.combat, self)

```

```

self._closed = False
self._active_panel = None
self._drag_offset = None

```

```

layout = QVBoxLayout(self)
layout.setContentsMargins(10, 4, 10, 8)
layout.setSpacing(2)

```

```

self.header = QWidget()
self.header.setFixedHeight(20)
self.header.installEventFilter(self)

```

```

header_layout = QHBoxLayout(self.header)
header_layout.setContentsMargins(0, 0, 0, 0)
header_layout.setSpacing(4)

```

```

title = QLabel("f DESKTOP ABYSS")
title.setStyleSheet("color: #ad895f; font-size: 9px;")
title.setAttribute(
    Qt.WidgetAttribute.WA_TransparentForMouseEvents
)
header_layout.addWidget(title)

```

```

self.gold = QLabel()
self.gold.setAlignment(Qt.AlignmentFlag.AlignRight)
self.gold.setStyleSheet(
    "color: #edc579; font-weight: bold; font-size: 11px;"
)
self.gold.setAttribute(
    Qt.WidgetAttribute.WA_TransparentForMouseEvents
)
header_layout.addWidget(self.gold, 1)

```

```

self.pin = QPushButton()
self.pin.setText("◆")
self.pin.setCheckable(True)
self.pin.setChecked(True)
self.pin.setFixedSize(22, 18)
self.pin.setToolTip("Закрепить поверх других окон")
self.pin.toggled.connect(self._pin)

```

```

minimize = QPushButton()
minimize.setText("-")
minimize.setFixedSize(22, 18)
minimize.setToolTip("Свернуть игру")
minimize.clicked.connect(self.showMinimized)

```

```

close = QPushButton()
close.setText("x")
close.setFixedSize(22, 18)
close.setToolTip("Сохранить и закрыть игру")
close.clicked.connect(self.close)

```

```

header_layout.addWidget(self.pin)
header_layout.addWidget(minimize)
header_layout.addWidget(close)
layout.addWidget(self.header)

```

```

self.battle = CompactBattleStage(self.combat)
layout.addWidget(self.battle)

```

```

dock = QHBoxLayout()
dock.setContentsMargins(0, 0, 0, 0)
dock.setSpacing(4)

```

```

self.dock_buttons = {}

```

```

for caption, page in (
    ("ГЕРОЙ", "hero"),
    ("СУНДУК", "stash"),
    ("КУБ", "cube"),
    ("РУНЫ", "runes"),
):
    button = QPushButton(caption)
    button.setCheckable(True)

```

```

        button.setFixedHeight(24)
        button.clicked.connect(
            lambda checked=False, target=page:
                self.toggle_panel(target)
        )
        dock.addWidget(button, 1)
        self.dock_buttons[page] = button

    layout.addLayout(dock)

    # Один drawer, один QStackedWidget, одна видимая панель.
    self.drawer = PanelDrawer(self)
    self.drawer.closed.connect(self._drawer_closed)

    self.hero_panel = HeroPanel(self.actions)
    self.stash_panel = StashPanel(self.actions)
    self.cube_panel = CubePanel(self.actions)
    self.runes_panel = RunesPanel(self.actions)

    self.drawer.add_panel("hero", self.hero_panel, 840)
    self.drawer.add_panel("stash", self.stash_panel, 470)
    self.drawer.add_panel("cube", self.cube_panel, 550)
    self.drawer.add_panel("runes", self.runes_panel, 460)

    self.hero_panel.navigate.connect(self.navigate)

    self.actions.error.connect(self._show_error)
    self.actions.notice.connect(self._show_notice)
    self.state.state_changed.connect(self._refresh_header)
    self.combat.party_defeated.connect(self._show_notice)
    self.combat.combat_error.connect(self._show_notice)

    self.autosave = QTimer(self)
    self.autosave.setInterval(C.AUTOSAVE_INTERVAL_MS)
    self.autosave.timeout.connect(self._autosave)
    self.autosave.start()

    self._refresh_header()
    self._restore_position()

def paintEvent(self, event) -> None:
    painter = QPainter(self)

```

```

painter.setRenderHint(QPainter.RenderHint.Antialiasing)

w, h = self.width(), self.height()

path = QPainterPath()
path.moveTo(1, 12)
path.lineTo(9, 3)
path.lineTo(25, 3)
path.lineTo(29, 0)
path.lineTo(w - 29, 0)
path.lineTo(w - 25, 3)
path.lineTo(w - 9, 3)
path.lineTo(w - 1, 12)
path.lineTo(w - 1, h - 10)
path.lineTo(w - 11, h - 1)
path.lineTo(11, h - 1)
path.lineTo(1, h - 10)
path.closeSubpath()

painter.setBrush(QColor(22, 15, 27, 247))
painter.setPen(QPen(QColor("#a27949"), 1.2))
painter.drawPath(path)

painter.setPen(QPen(QColor("#6b3045"), 2))
painter.drawLine(9, h - 4, w - 9, h - 4)

for x, direction in ((5, 1), (w - 5, -1)):
    painter.setPen(QPen(QColor("#d0a469"), 1))
    painter.drawLine(x, 15, x, 35)
    painter.drawLine(x, 15, x + direction * 7, 8)
    painter.drawLine(x, h - 14, x + direction * 8, h - 6)

painter.end()

def eventFilter(self, watched, event) -> bool:
    if watched is self.header:
        if (
            event.type() == QEvent.Type.MouseButtonPress
            and event.button() == Qt.MouseButton.LeftButton
        ):
            self._drag_offset = (
                event.globalPosition().toPoint() - self.pos()

```

```

    )
    return True

    if (
        event.type() == QEvent.Type.MouseMove
        and self._drag_offset is not None
    ):
        self.move(
            event.globalPosition().toPoint() - self._drag_offset
        )
    return True

    if event.type() == QEvent.Type.MouseButtonRelease:
        self._drag_offset = None
        self.settings.setValue("widget/position", self.pos())
        return True

    return super().eventFilter(watched, event)

def _restore_position(self) -> None:
    screen = QApplication.primaryScreen()
    if screen is None:
        return

    area = screen.availableGeometry()
    saved = self.settings.value("widget/position")

    if isinstance(saved, QPoint):
        candidate = QApplication.screenAt(
            saved + QPoint(self.width() // 2, self.height() // 2)
        )
        if candidate is not None:
            area = candidate.availableGeometry()
            screen = candidate

    x, y = saved.x(), saved.y()
    else:
        x = area.right() - self.width() - 12
        y = area.bottom() - self.height() - 8

    x = max(area.left(), min(x, area.right() - self.width() + 1))
    y = max(area.top(), min(y, area.bottom() - self.height() + 1))

```

```

self.move(x, y)

def _position_drawer(self) -> None:
    if self._active_panel is None:
        return

    screen = QApplication.screenAt(self.frameGeometry().center())
    screen = screen or QApplication.primaryScreen()
    if screen is None:
        return

    area = screen.availableGeometry().adjusted(6, 6, -6, -6)
    preferred_w, preferred_h = self.PANEL_SIZES[self._active_panel]

    width = min(preferred_w, area.width())
    above = self.y() - area.top() - 7

    if above >= 280:
        height = min(preferred_h, above)
        y = self.y() - height - 7
    else:
        height = min(preferred_h, area.height())
        y = area.top()

    self.drawer.resize(width, height)

    x = self.x() + self.width() // 2 - width // 2
    x = max(area.left(), min(x, area.right() - width + 1))
    y = max(area.top(), min(y, area.bottom() - height + 1))

    self.drawer.move(x, y)

def toggle_panel(self, page: str) -> None:
    if self._active_panel == page and self.drawer.isVisible():
        self.drawer.close()
    else:
        self.navigate(page)

def navigate(self, page: str) -> None:
    if page not in self.PANEL_SIZES:
        return

```

```

self._active_panel = page
self.drawer.title = self.PANEL_TITLES[page]
self.drawer.select(page)
self._position_drawer()

for key, button in self.dock_buttons.items():
    button.setChecked(key == page)

self.drawer.show()
self.drawer.raise_()
self.drawer.update()

if page == "hero":
    self.hero_panel.refresh()
elif page == "runes":
    QTimer.singleShot(0, self.runes_panel.view.fit_tree)

def _drawer_closed(self) -> None:
    self._active_panel = None
    for button in self.dock_buttons.values():
        button.setChecked(False)

def moveEvent(self, event) -> None:
    super().moveEvent(event)
    if hasattr(self, "drawer") and self.drawer.isVisible():
        self._position_drawer()

def hideEvent(self, event) -> None:
    if hasattr(self, "drawer"):
        self.drawer.hide()
        self._drawer_closed()
    super().hideEvent(event)

def _pin(self, enabled: bool) -> None:
    position = self.pos()
    self.setWindowFlag(
        Qt.WindowType.WindowStaysOnTopHint, enabled
    )
    self.drawer.setWindowFlag(
        Qt.WindowType.WindowStaysOnTopHint, enabled
    )
    self.show()

```

```

self.move(position)

def _refresh_header(self, *_args) -> None:
    data = self.state.data
    self.gold.setText(f"♦ {data.gold:,}")
    self.header.setToolTip(
        f"Золото: {data.gold:,}\n"
        f"Волна: {data.current_wave:,}\n"
        "Перетащите заголовок, чтобы переместить виджет"
    )

def _show_notice(self, message: str) -> None:
    # Сообщения не увеличивают размер базового виджета.
    self.header.setToolTip(message)
    self.gold.setToolTip(message)

def _show_error(self, message: str) -> None:
    parent = self.drawer if self.drawer.isVisible() else self
    QMessageBox.warning(parent, "Предупреждение Бездны", message)

def start_combat(self) -> None:
    self.combat.start()

def save_now(self) -> None:
    self.combat.flush()
    self.saves.checkpoint(self.state)
    self.settings.setValue("widget/position", self.pos())

def _autosave(self) -> None:
    try:
        self.save_now()
    except (SaveError, ValueError, RuntimeError) as exc:
        self._show_notice(f"Ошибка сохранения: {exc}")
        print(f"Ошибка автосохранения: {exc}", file=sys.stderr)

def show_offline_report(self, result) -> None:
    if result.elapsed_seconds < 1:
        return

    hours, remainder = divmod(int(result.elapsed_seconds), 3600)
    minutes = remainder // 60

```



```

text = (
    f"Вы отсутствовали: {hours} ч {minutes} мин\n"
    f"Отражено волн: {result.waves_cleared:,}\n\n"
    f"Золото: +{result.total_gold:,}\n"
    f"Опыт отряду: +{result.total_exp:,}\n"
    f"Предметов в сундук: {result.items_stored}\n"
    f"Автопродано: {result.items_sold}"
)

if result.clock_rollback:
    text += "\n\nОбнаружен перевод часов назад."

QMessageBox.information(self, "Приветствие Бездны", text)

def closeEvent(self, event) -> None:
    if self._closed:
        event.accept()
        return

    try:
        self.save_now()
    except (SaveError, ValueError, RuntimeError) as exc:
        answer = QMessageBox.warning(
            self,
            "Не удалось сохранить игру",
            f"{exc}\n\nВыйти без нового сохранения?",
            QMessageBox.StandardButton.Yes
            | QMessageBox.StandardButton.No,
            QMessageBox.StandardButton.No,
        )
        if answer != QMessageBox.StandardButton.Yes:
            event.ignore()
            return

    self._closed = True
    self.autosave.stop()
    self.drawer.hide()
    self.combat.stop()
    event.accept()

def final_checkpoint(self) -> None:
    if self._closed:

```

```

        return
    try:
        self.save_now()
    except (SaveError, ValueError, RuntimeError) as exc:
        print(f"Ошибка финального сохранения: {exc}", file=sys.stderr)
    self._closed = True
    self.autosave.stop()
    self.drawer.hide()
    self.combat.stop()

```

Файл: main.py

```

`python
from future import annotations

import signal
import sys

from PySide6.QtCore import QLockFile, QTimer
from PySide6.QtWidgets import QApplication, QMessageBox

import config as C
from engine.offline_manager import OfflineManager
from engine.save_manager import SaveError, SaveManager
from engine.state import GameState
from ui.common import APP_STYLESHEET
from ui.main_window import MainWindow

def main() -> int:
    app = QApplication(sys.argv)
    app.setApplicationName(C.APP_NAME)
    app.setOrganizationName(C.ORGANIZATION_NAME)
    app.setStyleSheet(APP_STYLESHEET)

    try:
        C.DATA_DIR.mkdir(parents=True, exist_ok=True)

```

```

except OSError as exc:
    QMessageBox.critical(
        None,
        "Ошибка запуска",
        f"Не удалось создать каталог данных:\n{exc}",
    )
    return 1

lock = QLockFile(str(C.LOCK_PATH))
lock.setStaleLockTime(0)

if not lock.tryLock(0):
    QMessageBox.warning(
        None,
        C.APP_NAME,
        "Не удалось получить блокировку сохранения.\n"
        "Возможно, игра уже запущена.",
    )
    return 1

try:
    state = GameState.instance()
    saves = SaveManager()

    try:
        state.load(saves.load())

        # Проверяем новые данные героев до выдачи и записи наград.
        from engine.runes_tree import RunesTree
        RunesTree().squad(state.data)

        if not 3 <= len(state.data.party) <= 4:
            raise ValueError("Для боя необходим отряд из 3-4 героев")

        offline = OfflineManager().claim(state, saves)

    except (SaveError, ValueError, RuntimeError) as exc:
        QMessageBox.critical(
            None,
            "Не удалось загрузить игру",
            f"{exc}\n\nСохранение не сброшено.",
        )

```

```

        return 1

window = MainWindow(state, saves)
window.show()

# Никакая панель автоматически не открывается.
# CombatManager ещё остановлен, пока читается оффлайн-отчёт.
window.show_offline_report(offline)
window.start_combat()

signal_timer = QTimer(app)
signal_timer.setInterval(200)
signal_timer.timeout.connect(lambda: None)
signal_timer.start()

signal.signal(signal.SIGINT, lambda _: window.close())
signal.signal(signal.SIGTERM, lambda _: window.close())

app.aboutToQuit.connect(window.final_checkpoint)

return app.exec()

finally:
    lock.unlock()

```

```

if name == "main":
    raise SystemExit(main())

```

,